



A Correctness and Incorrectness Program Logic

ROBERTO BRUNI, University of Pisa, Italy

ROBERTO GIACOBAZZI, University of Verona, Italy

ROBERTA GORI, University of Pisa, Italy

FRANCESCO RANZATO, University of Padova, Italy

Abstract interpretation is a well-known and extensively used method to extract over-approximate program invariants by a sound program analysis algorithm. Soundness means that no program errors are lost and it is, in principle, guaranteed by construction. Completeness means that the abstract interpreter reports no false alarms for all possible inputs, but this is extremely rare because it needs a very precise analysis. We introduce a weaker notion of completeness, called *local completeness*, which requires that no false alarms are produced only relatively to some fixed program inputs. Based on this idea, we introduce a program logic, called Local Completeness Logic for an abstract domain A , for proving both the correctness and incorrectness of program specifications. Our proof system, which is parameterized by an abstract domain A , combines over- and under-approximating reasoning. In a provable triple $\vdash_A [p] c [q]$, c is a program, q is an under-approximation of the strongest post-condition of c on input p such that their abstractions in A coincide. This means that q is never too coarse, namely, under some mild assumptions, *the abstract interpretation of c does not yield false alarms for the input p iff q has no alarm*. Therefore, proving $\vdash_A [p] c [q]$ not only ensures that all the alarms raised in q are true ones, but also that if q does not raise alarms, then c is correct. We also prove that if A is the straightforward abstraction making all program properties equivalent, then our program logic coincides with O'Hearn's incorrectness logic, while for any other abstraction, contrary to the case of incorrectness logic, our logic can also establish program correctness.

CCS Concepts: • **Theory of computation** → **Logic and verification**; **Abstraction**; **Programming logic**; **Semantics and reasoning**; **Program analysis**; *Hoare logic*; Axiomatic semantics; **Abstraction**; **Program reasoning**;

Additional Key Words and Phrases: Abstract interpretation, abstract domain, program analysis, program verification, program logic, local completeness, best correct approximation, incorrectness logic

ACM Reference format:

Roberto Bruni, Roberto Giacobazzi, Roberta Gori, and Francesco Ranzato. 2023. A Correctness and Incorrectness Program Logic. *J. ACM* 70, 2, Article 15 (March 2023), 45 pages.
<https://doi.org/10.1145/3582267>

The authors have been funded by the *Italian MIUR*, under the PRIN2017 project no. 201784YSZ5 “Analysis of Program Analyses (ASPRA)” and by a *Meta research* gift. Roberto Giacobazzi and Francesco Ranzato have been partially funded by *Facebook Research*, under a “Probability and Programming Research Award,” by an *Amazon Research Award* for “AWS Automated Reasoning,” and by a *WhatsApp Research Award* on “Privacy-aware Program Analysis.”

Authors' addresses: R. Bruni and R. Gori, University of Pisa, Pisa, Italy; emails: bruni@di.unipi.it, gori@di.unipi.it; R. Giacobazzi, University of Verona, Verona, Italy; email: roberto.giacobazzi@univr.it; F. Ranzato, University of Padova, Padova, Italy; email: ranzato@math.unipd.it.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

© 2023 Copyright held by the owner/author(s).

0004-5411/2023/03-ART15 \$15.00

<https://doi.org/10.1145/3582267>

1 INTRODUCTION

“The only effective way to raise the confidence level of a program significantly is to give a convincing proof of its correctness” [Dijkstra 1972b]. This statement, given by E. W. Dijkstra in 1972 in his Turing Award lecture [Dijkstra 1972c], is universally valid and nowadays felt as an indispensable necessity and still a major challenge in modern information societies [Hoare 2003; Jones et al. 2006]. The idea of having programs that certify other programs is due to A. M. Turing [Turing 1989] and put forward by McCarthy [1962], Floyd [1967], and Hoare [1969] in the 1960s. The past 50 years have witnessed an incredible flourishing of formal methods and tools for achieving this ambitious goal. These include, among others, certified compilers [Leroy 2006], certified analyzers [Jourdan et al. 2015], advanced type checkers [Pierce 2002], software model checkers [Ball et al. 2005], and abstract interpretation-based methods for static program analysis [Cousot 2021; Cousot and Cousot 1977; Giacobazzi and Ranzato 2022]. Proving program correctness means inferring an adequate invariant that has to be strong enough to imply the desired correctness property for our code. In program correctness proofs, the ambition to automate the inference of a “good” invariant immediately leads us to the need of some sort of over-approximation, making tractable (e.g., decidable) problems that are otherwise intractable. The well-known and inherent undecidability of all non-straightforward extensional properties of programs provides an intrinsic limitation in the use of approximated and decidable formal methods for proving program properties [Rice 1953] (see also Cousot et al. [2018]). This is particularly clear in program analysis, where the necessary termination of the analysis algorithm may introduce false alarms. The soundness of a program analyser, which is guaranteed by construction in abstract interpretation, means that all true alarms (also called true positives) are caught, but it is often the case that false alarms (also called false positives) are reported. There exists a range of successful applications of formal methods for proving the absence of bugs in programs [Calcagno et al. 2015; Cousot 2021; Distefano et al. 2019; Hoare 1969; O’Hearn 2018; Rival and Yi 2020; Sadowski et al. 2018]. Of course, as in all verification systems, program analysis is credible when few false alarms are reported, ideally none. Completeness holds when no false alarm is ever raised, and this represents in many ways the *holy grail* of program analysis and verification [Jones et al. 2006].

Understanding whether an alarm corresponds to a true bug, i.e., indirectly whether our approximation method is complete, boils down to the challenge of proving some sort of program incorrectness. The same E. W. Dijkstra asserted in 1972 that “Program testing can be used to show the presence of bugs, but never to show their absence” [Dijkstra 1972a]. Proving that our program contains a true bug therefore means isolating those states that will eventually trigger the fault, i.e., will make our program violating a correctness assertion. This means propagating along the computation properties of states that are strong enough to isolate those states that will produce a fault. This is the key idea in O’Hearn [2020]’s incorrectness logic (IL) and its subsequent application to separation logic in Raad et al. [2020]. By exploiting under-approximations, any violation exposed by such program analysis corresponds to a true alarm. This makes IL a credible support for code-review and test-driven software development, although a correctness condition can still be violated even when no alarm is reported by the analysis.

The tension between proving the absence of bugs and exhibiting their actual presence shaped the research in programming languages and software engineering for decades. Nevertheless, the problem is far from being solved and static reasoning should be extended to bug catching, rather than only for proving absence of bugs, as advocated by O’Hearn [2020].

The Problem

In this work, we consider abstract interpretation [Cousot and Cousot 1977] as the reference theory to design and validate algorithms capable to infer over-approximate program invariants. The

problem is how to guarantee that the approximation is precise enough to avoid false alarms. Next we explain why completeness is crucial to ensure precision of the analysis and why it is hard to achieve it. The ingredients are as follows: an abstract domain A of abstract program properties, e.g., some properties of stores, which is connected with the concrete domain of program properties C by a pair of monotone functions $\alpha : C \rightarrow A$ and $\gamma : A \rightarrow C$, respectively the abstraction and concretization map, such that $A = \alpha(C)$ and for any concrete property $p \in C$, $p \subseteq A(p) \triangleq \gamma\alpha(p)$.¹ Program verification by abstract interpretation means that instead of verifying whether the strongest post-condition $\text{post}[c](p)$ for a program c and a pre-condition p , satisfies a correctness specification spec , we verify whether a sound abstract over-approximation $\text{post}_A[c] : A \rightarrow A$ satisfies spec . Using denotational semantics symbols, the original verification problem can be stated more concisely as checking if $\llbracket c \rrbracket p \subseteq \text{spec}$ holds, while its abstract counterpart becomes the property $\gamma(\llbracket c \rrbracket_A^\# \alpha(p)) \subseteq \text{spec}$, where $\llbracket c \rrbracket$ denotes the strongest post-condition function for c and $\llbracket c \rrbracket_A^\#$ is called the *abstract interpretation of c on A* .

As observed above, the problem here is that computing $\llbracket c \rrbracket_A^\#$ may degrade the approximation and make the computed abstract program property too weak to imply the desired specification. This may be a consequence of the fact that abstract interpretation analysis is done by composing functions. Consider the abstract interpretation of c on A defined as $\llbracket c \rrbracket^A \triangleq \alpha \circ \llbracket c \rrbracket \circ \gamma$, commonly called best correct approximation (bca) of c in A . This is the best possible approximate semantics, namely any abstract interpretation $\llbracket c \rrbracket_A^\#$ is sound if and only if $\llbracket c \rrbracket^A \subseteq \llbracket c \rrbracket_A^\#$ [Cousot and Cousot 1979]. It is known that, in general, the composition of two bcas is not necessarily a bca itself. Consider the case of the abstract domain of integer intervals Int whose elements approximate any property $p \in \wp(\mathbb{Z})$ of the integer values that a variable x may assume by the least interval $\text{Int}(p) = [a, b]$ such that $p \subseteq [a, b]$, where $a \leq b$, $a \in \mathbb{Z} \cup \{-\infty\}$ and $b \in \mathbb{Z} \cup \{+\infty\}$ and the commands

$$c_1 \triangleq \text{if } \text{even}(x) \text{ then } 0 \text{ else } 1 \quad \text{and} \quad c_2 \triangleq \text{if } \text{even}(x) \text{ then } x \text{ else } x + 1.$$

In this case, it turns out that $\llbracket c_2 \rrbracket^{\text{Int}}([0, 1]) = [0, 2]$ and $\llbracket c_1 \rrbracket^{\text{Int}} \circ \llbracket c_2 \rrbracket^{\text{Int}}([0, 1]) = \llbracket c_1 \rrbracket^{\text{Int}}([0, 2]) = [0, 1]$, while $(\llbracket c_1 \rrbracket \circ \llbracket c_2 \rrbracket)^{\text{Int}}([0, 1]) = [0, 0]$ and $[0, 0] \subsetneq [0, 1]$. The lack of compositionality is the main cause of the interdependence between the precision of an abstract interpretation and the way programs are written [Bruni et al. 2020]. While it is obvious that if $\llbracket c \rrbracket_A^\# \alpha(p)$ satisfies spec (i.e., $\gamma(\llbracket c \rrbracket_A^\# \alpha(p)) \subseteq \text{spec}$ holds), then the program is correct, it may happen that $\llbracket c \rrbracket_A^\# \alpha(p)$ does not satisfy spec even if the program is correct, hence producing a false alarm. In this case, if $\gamma(\llbracket c \rrbracket_A^\# \alpha(p)) \not\subseteq \text{spec}$, then we cannot conclude that the Hoare triple $\{p\} c \{ \text{spec} \}$ is not valid, because any witness in $\gamma(\llbracket c \rrbracket_A^\# \alpha(p)) \setminus \text{spec}$ could be a *false alarm*.

Complete abstract interpretations, instead, do not raise false alarms. Technically, this also requires the assumption that the specification spec is expressible in A , namely that $\text{spec} = A(\text{spec})$ holds. For example, the property $x \geq 0$ is expressible by the infinite interval $[0, +\infty]$. By contrast, $x \neq 0$ is not expressible in Int , since the least over-approximating interval is $\text{Int}(x \neq 0) = \mathbb{Z} \supsetneq \mathbb{Z} \setminus \{0\}$. If spec is expressible in A , then completeness guarantees that a Hoare triple $\{p\} c \{ \text{spec} \}$ is valid iff $\gamma(\llbracket c \rrbracket_A^\# \alpha(p)) \subseteq \text{spec}$ holds. Thus, any complete abstract verification of spec by means of the (complete) abstract semantics $\llbracket c \rrbracket_A^\#$ is the same as verifying spec with respect to the concrete semantics. Notably, when the abstract interpretation is complete, then $\llbracket c \rrbracket_A^\#$ always coincides with the bca $\llbracket c \rrbracket^A$ [Giacobazzi et al. 2000]. According to a well-established definition [Cousot and Cousot

¹Abstract interpretation is more general than this [Cousot and Cousot 1977]. There are weaker abstract interpretation frameworks where only the concretization function is assumed to exist [Cousot and Cousot 1992]. We consider the stronger formulation based on Galois insertions because it is the most widely known and, as observed later, this provides enough mathematical structure to define the key notion of completeness in abstract interpretation.

[1977, 1979], completeness is a *global notion*, meaning that it involves *all* possible pre-conditions. More precisely, A is complete for a program c when $\llbracket c \rrbracket_A^\# \circ \alpha = \alpha \circ \llbracket c \rrbracket$, namely

$$\text{for all } p \in C, \llbracket c \rrbracket_A^\# \alpha(p) = \alpha(\llbracket c \rrbracket p) \text{ holds.}$$

This form of completeness is extremely hard to achieve. Giacobazzi et al. [2015] proved that for *any* non-straightforward abstraction A there always exists a program c for which any sound abstract interpretation of c on A yields at least one false alarm. Later, Bruni et al. [2020] showed that any program equivalence induced by an abstract interpreter built over a non-straightforward abstraction violates extensionality. Here, non-straightforward abstractions correspond to those abstract domains A that are able to distinguish at least two programs, i.e., there exist two programs c_1 and c_2 such that $\llbracket c_1 \rrbracket_A^\# \neq \llbracket c_2 \rrbracket_A^\#$, and A does not coincide with the identical abstraction, i.e., $\llbracket \cdot \rrbracket_A^\# \neq \llbracket \cdot \rrbracket$. In particular, it is known [Giacobazzi et al. 2015] that the main sources of incompleteness lie in the abstract interpretation of store assignments and Boolean guards. The case of Boolean guards is striking. The semantics of a Boolean guard is a predicate transformer $\llbracket b? \rrbracket : \wp(\Sigma) \rightarrow \wp(\Sigma)$, where Σ denotes the set of concrete stores. Completeness of a Boolean guard $b?$ in the abstract domain A means that for all $p \in \wp(\Sigma)$, $A(\llbracket b? \rrbracket p) = A(\llbracket b? \rrbracket A(p))$. Of course, because both branches of a conditional or loop statement guarded by b must be taken into account, the same condition applies to the negative test $\neg b?$. For example, Int is complete for a simple rectifier program, known as ReLU in neural networks,

$$\text{ReLU}(x) \triangleq \text{if } (x < 0) \text{ then } x := 0 \text{ else skip,}$$

even if its Boolean guards are not complete in Int . In fact, for the pre-condition $p \triangleq x \in \{-1, 1\}$, we have that

$$\text{Int}(\llbracket x \geq 0? \rrbracket p) = \text{Int}(\{1\}) = [1, 1] \subsetneq [0, 1] = \text{Int}(\llbracket x \geq 0? \rrbracket [-1, 1]) = \text{Int}(\llbracket x \geq 0? \rrbracket \text{Int}(p)),$$

meaning that Int is not complete for the guard $x \geq 0?$. This happens even if both guards $x < 0?$ and $x \geq 0?$ are expressible in Int as $x \in [-\infty, -1]$ and $x \in [0, +\infty]$, respectively. However, Int is instead trivially complete for both commands $x := 0$ and **skip** of ReLU. Hence, it is the lack of completeness of the abstract interpretation of the two guards $x < 0?$ and $x \geq 0?$ that prevents us to *inductively prove completeness* for the simple program ReLU. Abstraction refinement does not help either. The complete shell of Int , as described in Giacobazzi et al. [1998, 2000], for the guards $x < 0?$ and $x \geq 0?$ blows up to nearly the concrete domain of all program stores. In fact, we should refine the interval abstraction by adding denotations that describe any pair of intervals (I_1, I_2) such that $I_1 \cap I_2 = \emptyset$ and $I_1 \cup I_2$ is not an interval where $I_1 \subseteq \mathbb{Z}_{<0}$ and $I_2 \subseteq \mathbb{Z}_{\geq 0}$. This would give us a complete abstract domain for $x < 0?$ and $x \geq 0?$ that is barely equivalent to the concrete domain and therefore useless for a practical program verification.

Roadmap to Main Contributions

The main theme of the article is to combine under- and over-approximations via abstract interpretation and the novel notion of local completeness to define a program logic whose triples either prove correctness or incorrectness.

After some background on abstract interpretation in Section 2, in Section 3 we give necessary and sufficient conditions that guarantee completeness of Boolean guards on an abstract domain A . These conditions require that both $b?$ and $\neg b?$ are expressible in the abstract domain and the same has to apply to the join of any two concretizations of abstract points below $\alpha(\llbracket b? \rrbracket)$ and $\alpha(\llbracket \neg b? \rrbracket)$. This requirement turns out to be very strong: For example, this condition allows us to prove that any Boolean guard on the interval abstraction Int is incomplete (cf. Example 3.7).

Since completeness is rare, in Section 4 we introduce *locally complete abstract interpretations*, a natural weakening of completeness. Instead of requiring completeness on all possible inputs, a

locally complete abstract interpretation is complete just for some given subset of possible inputs. In the case of a guard $b?$ with input p , local completeness amounts to checking whether $\llbracket b? \rrbracket_A^\# \alpha(p) = \alpha(\llbracket b? \rrbracket p)$ holds for that particular $p \in C$. For example, any guard is locally complete for any input p that is expressible in the abstract domain. In our example of ReLU, it is easy to check that when the input is any interval or any set of integers all having the same sign, e.g., either all nonnegative or all negative, then both guards $x < 0?$ and $x \geq 0?$ turn out to be locally complete in Int , so that we can inductively infer that ReLU is locally complete in Int with respect to any such input.

By means of local completeness we can prove the absence of false alarms for programs that are globally incomplete on a given abstract domain. As a simple example, consider the following program `AbsVal` for computing the absolute value of integer variables:

AbsVal(x) $\triangleq$ if ($x < 0$) then $x := -x$ else skip.

`AbsVal` is globally incomplete on the abstract domain Int . For instance, for a pre-condition $p \triangleq x \in \{-7, 7\}$, we have that $\text{Int}(\llbracket \text{AbsVal} \rrbracket p) = [7, 7]$ while $\text{Int}(\llbracket \text{AbsVal} \rrbracket \text{Int}(p)) = \text{Int}(\llbracket \text{AbsVal} \rrbracket [-7, 7]) = [0, 7]$. Therefore, even if the pre-condition p does not include zero, an interval analysis of `AbsVal` yields a false alarm when checking the expressible specification $\text{spec} \triangleq x > 0$. Similarly to ReLU, this is not the case when the inputs all have the same sign, no matter if positive or negative. Instead, a true alarm for spec can be raised only if the set of input values truly contains zero. This means that to use `AbsVal` in our code and not having false alarms in an interval analysis, e.g., within a loop, we need to make sure that `AbsVal( $x$ )` will be called with x always having the same sign.

Our main contribution is in Section 5, where we present a logical proof system $\vdash_A$ called Local Completeness Logic on A (LCL_A) for locally complete abstract interpretations parameterized on an arbitrary abstract domain A . Our assertions are Hoare-like triples $\vdash_A [p] \text{ c } [q]$ establishing that

- (i) q is an *under-approximation* of $\text{post}[c]p$ (i.e., of $\llbracket c \rrbracket p$);
- (ii) $\text{post}[c]$ is *locally complete* for input p on A ;
- (iii) q and $\text{post}[c]p$ have the same *over-approximation* in A .

These properties of any provable triple $\vdash_A [p] \text{ c } [q]$ allow us to distinguish between true and false alarms raised by an abstract interpreter $\llbracket c \rrbracket_A^\# \alpha(p)$ for verifying any correctness specification spec that is expressible in A . The key rules in LCL_A are as follows:

$$\frac{\text{post}[e] \text{ locally complete for } p \text{ on } A}{\vdash_A [p] \text{ e } [\text{post}[e](p)]} \text{ (transfer)}$$

$$\frac{p' \Rightarrow p \Rightarrow A(p') \quad \vdash_A [p'] \text{ c } [q'] \quad q \Rightarrow q' \Rightarrow A(q)}{\vdash_A [p] \text{ c } [q]} \text{ (relax)}.$$

The rule (transfer) checks that a basic expression e , such as a Boolean test $b?$ or an assignment $x := a$ is locally complete for p before inferring the output of $\text{post}[e]$ on p as post-condition. The consequence rule (relax) is the key principle of LCL_A and combines an over- and under-approximating reasoning: (relax) allows us to infer a post-condition that defines an under-approximation q of the exact behavior as well as a sound over-approximation $A(q)$ of it, i.e., such that $q \Rightarrow \text{post}[c](p) \Rightarrow A(\text{post}[c](p)) = A(q)$ holds. Likewise, in the consequence rules of the reverse Hoare logic by de Vries and Koutavas [2011], incorrectness logic by O'Hearn [2020], and incorrectness separation logic by Raad et al. [2020], the logical ordering between pre-conditions $p' \Rightarrow p$ and post-conditions $q \Rightarrow q'$ in the premises of (relax) is reversed w.r.t. the canonical consequence rule of classical Hoare logic, and this is needed because our post-conditions q are always under-approximations.

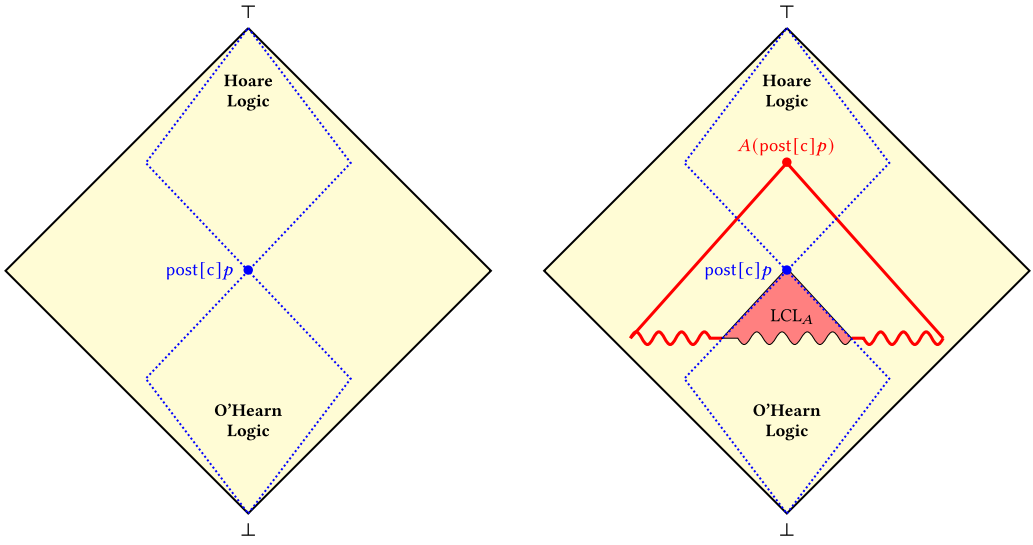


Fig. 1. Relation among LCL_A , O'Hearn's Incorrectness Logic and Hoare's partial correctness logic.

The key feature of (relax) is to constrain the under-approximating post-condition q to have *the same abstraction* as the strongest post-condition, which is needed for preserving local completeness. This twist is fundamental to guarantee that any triple derivable in LCL_A either proves correctness or incorrectness. Figure 1 illustrates the relations between different proof systems. On the left, it is shown that Hoare logic for correctness derives any over-approximation of $\text{post}[c](p)$ (i.e., any provable triple $\{p\} c \{q\}$ is such that q belongs to the blue dotted upper diamond region) while O'Hearn logic for incorrectness is able to derive any under-approximation of $\text{post}[c](p)$ (i.e., any provable triple $\vdash_{\text{IL}} [p] c [q]$ is such that q belongs to the blue dotted lower diamond region). On the right, we show the interplay between approximations derived using locally complete abstract interpretation A (the red bordered region, comprising any q such that $A(q) = A(\text{post}[c]p)$) and LCL_A under-approximations (i.e., any provable triple $\vdash_A [p] c [q]$ is such that q belongs to the red-filled region, still with $A(q) = A(\text{post}[c]p)$). We show that by the (relax) rule we can shrink the post-condition of any triples, up to some boundary that fringes the assertions of O'Hearn logic. This is made without much loss of precision: Under mild conditions on *spec*, any approximation q in the red-filled region guarantees that if the over-approximation $A(q)$ reports some alarm, then q contains a *true* alarm and, conversely, any alarm in q is a true one. The key point is that any derivable triple $\vdash_A [p] c [q]$ of LCL_A provides an under-approximation q that is not too coarse. More precisely, given a correctness specification *spec* expressible in A , the following two scenarios can occur:

- (a) *spec* is satisfied: Abstract interpretation in A , as well as any triple $\vdash_A [p] c [q]$ derivable in LCL_A , allow to conclude that *spec* holds. This is not true in Hoare logic: Although $\{p\} c \{spec\}$ is a valid triple, since post-conditions in Hoare triples can always be weakened, it is also possible to derive some triple $\{p\} c \{q\}$ whose post-condition q includes some false alarm. Using IL, no true alarm can be found and no conclusion can be drawn about the validity of *spec*.
- (b) *spec* is violated: Since *spec* is expressible in A , then *any* derivable triple $\vdash_A [p] c [q]$ of LCL_A will expose a true alarm witnessing that *spec* is violated. On the contrary, abstract interpretation in A , as well as Hoare logic, can also expose false alarms that cannot be distinguished

from true ones. In IL, although it is possible to derive triples $\vdash_{\text{IL}} [p] \text{ c } [q]$ where q exhibits some (true) alarms, since post-conditions can always be strengthened, other triples for p and c may have no alarm at all, e.g., $\vdash_{\text{IL}} [p] \text{ c } [\text{ff}]$.

We prove in Theorem 5.5 that LCL_A is sound with respect to the above properties (i)–(iii). To prove a result of logical completeness² of LCL_A for a program c , we add two more ingredients:

- (1) as in incorrectness logic [O’Hearn 2020], an infinitary rule (limit) for iteration; and
- (2) the assumption that all the basic instructions occurring in the program c of a provable triple $\vdash_A [p] \text{ c } [q]$ are globally complete on A .

In Section 6, we show that IL coincides with LCL_A when the straightforward abstraction A_{tr} that is unable to distinguish any two programs is considered. In fact A_{tr} is globally complete for every transfer function, hence for every program, and therefore the premises of the rule (transfer) are always satisfied. We also prove that A_{tr} is the only abstraction A for which the proof system LCL_A can be logically complete for a Turing complete programming language.

In Section 7, we observe that for iterative commands guarded by a Boolean guard $b?$, e.g., in while loops, it is not necessary to require the proof obligations of local completeness for $b?$ at every iteration provided that local completeness is met when the loop invariant is reached. Therefore, to improve the expressiveness of the logical system in handling while programs, we introduce two additional sound rules that are specific to while loops.

In Section 8, we consider the possibility of refining the abstract domain to complete the proof when some proof obligations of local completeness are not met by the current abstraction. Proving a triple $\vdash_A [p] \text{ c } [q]$ in a refined domain can no longer guarantee the local completeness of $\llbracket c \rrbracket_A^\#$ on input p , but we can still prove the local completeness of the best correct approximation $\llbracket c \rrbracket^A$ so that $\llbracket c \rrbracket^A \alpha(p) = \alpha(\llbracket c \rrbracket p) = \alpha(q)$ holds. From the viewpoint of program verification w.r.t. a correctness specification $spec$ that is expressible in A , this means that we retain all the potential of LCL_A for finding bugs (i.e., any element in $q \setminus spec$) or proving correctness (when $q \subseteq spec$ holds) even if the abstract interpretation $\llbracket c \rrbracket_A^\# \alpha(p)$ is not as precise as the bca $\llbracket c \rrbracket^A \alpha(p)$.

Finally, Section 9 discusses some related work, and Section 10 concludes by outlining some directions of future work expanding the ideas of this article.

This article is a full and revised version of the LICS 2021 paper [Bruni et al. 2021], extended to include all the technical proofs, further examples, and some entirely novel contributions reported in Section 4.1 concerning a local completeness characterization for Boolean guards, Section 7 concerning a relaxation of local completeness requirements for while loops, and Section 8 concerning a program logic for best correct approximations.

2 BACKGROUND

As a matter of notation, given two sets X and Y , $X \setminus Y$ denotes the set-difference between X and Y , while $X \subsetneq Y$ denotes strict inclusion. Given two functions $f : X \rightarrow Y$ and $g : Y \rightarrow Z$, we denote with $g \circ f$, or simply gf , their composition. For $f : X \rightarrow X$ and $n \in \mathbb{N}$, we let $f^n : X \rightarrow X$ be defined inductively as usual: $f^0 \triangleq id_X$ and $f^{n+1} \triangleq f \circ f^n$, where id_X denotes the identity function on X (often abbreviated id).

In ordered structures such as posets and complete lattices over a set of elements C , we typically use $\leq_C$ to denote a partial order relation, $\vee_C$ for least upper bound (lub), $\wedge_C$ for greatest lower bound (glb), $\top_C$ and $\perp_C$ for, respectively, greatest and least elements. We use $\wp(X)$ to denote the powerset complete lattice over a set X ordered by inclusion. If $f : X \rightarrow Y$, then f is overloaded

²The term *logical* is used here to distinguish the standard notion of completeness for a proof system from the notion of completeness in abstract interpretation.

to denote its powerset lifting $f : \wp(X) \rightarrow \wp(Y)$, where $f(S) \triangleq \{f(x) \mid x \in S\}$. If C is a complete lattice and $X \subseteq C$, then the Moore closure of X is defined as $\mathcal{M}(X) \triangleq \{\wedge_C Y \mid Y \subseteq X\}$, that is, $\mathcal{M}(X)$ is the least superset of X closed under glbs of its subsets (also called Moore closed); in particular, notice that $X \subseteq \mathcal{M}(X)$ and $\wedge_C \emptyset = \top_C \in \mathcal{M}(X)$. If C is a poset and $l, u \in C$, then $[l, u] \triangleq \{x \in C \mid l \leq_C x \leq_C u\}$ denotes the segment between l and u in C . For $f, g : C_1 \rightarrow C_2$ between posets, $f \leq g$ denotes that for all $x \in C_1$, $f(x) \leq_{C_2} g(x)$, while f is monotone when $x \leq_{C_1} y$ implies $f(x) \leq_{C_2} f(y)$. A function f between complete lattices is additive (respectively co-additive) when f preserves arbitrary lubs (respectively, glbs). The least fixpoint of a function $f : C \rightarrow C$ on a poset C is denoted, when it exists, by $\text{lfp}(f)$. Let us recall that if f is (Scott) continuous on a complete lattice then $\text{lfp}(f) = \bigvee_C \{f^n(\perp) \mid n \in \mathbb{N}\}$.

2.1 Abstract Interpretation

2.1.1 Abstract Domains. If the concrete semantics of our programming language is specified on a given *concrete domain* of properties C , then abstract interpretation [Cousot 2021; Cousot and Cousot 1977] is the method to specify abstract semantics, namely approximate semantics defined on an *abstract domain* of approximate program properties A . Concrete and abstract domains are typically complete lattices. This guarantees the existence of the basic lattice operators of join and meet used in the definition of concrete and abstract semantics. Since several abstractions are possible, we use subscripts such as $\leq_A$ and $\vee_A$ to disambiguate the underlying carrier set A and omit the subscripts in the case of C . Given complete lattices C and A , a pair of functions $\alpha : C \rightarrow A$ and $\gamma : A \rightarrow C$ forms a Galois connection (GC, a special case of an adjunction) when for all $c \in C$, $a \in A$, $\alpha(c) \leq_A a \Leftrightarrow c \leq \gamma(a)$ holds. In a GC the lattices C and A are called, respectively, concrete and abstract domain, and α and γ are called, respectively, abstraction and concretization maps. We only consider GCs such that $\alpha\gamma = \text{id}_A$, called Galois insertions (GIs), where α is surjective and γ is injective. Let us recall that α is additive, γ is co-additive, and $\gamma\alpha$ is an (upper) closure operator, that is, $\gamma\alpha : C \rightarrow C$ is a monotone, idempotent and extensive (i.e., $\text{id}_C \leq_C \gamma\alpha$ holds) function, and $\gamma(A) \subseteq C$ is Moore closed. Moreover, if $X \subseteq C$ is Moore closed, then X can be viewed as an abstraction of C through the maps $\alpha = \lambda c. \wedge_C \{x \in X \mid c \leq_C x\}$ and $\gamma = \text{id}_X$. The class of abstract domains of C is given by $\text{Abs}(C) \triangleq \{\langle A, \leq_A, \alpha, \gamma \rangle \mid \alpha : C \rightarrow A, \gamma : A \rightarrow C \text{ is a GI}\}$, and we write $A_{\alpha, \gamma} \in \text{Abs}(C)$ to mean that A is an abstract domain related to C by the abstraction and concretization maps α and γ . When convenient, we simply use A in place of the function $\gamma\alpha : C \rightarrow C$, e.g., $\text{Int}(\{-7, 7\}) = [-7, 7]$. For example, since γ is injective, a condition such as $\alpha(c) \leq_A \alpha(d)$ can be written as (and is equivalent to) $A(c) \leq A(d)$. An abstract domain $A_{\alpha, \gamma} \in \text{Abs}(C)$ is called *strict* when $\gamma(\perp_A) = \perp$, and a concrete value $c \in C$ is *expressible* in A when $A(c) = c$, while if $c < A(c)$ holds, then c is (strictly) *approximated* in A . Notice that $\gamma(A)$ and $C \setminus \gamma(A)$ are the sets of concrete values that are, respectively, expressible and approximated in A . Given two abstract domains $A_{\alpha_A, \gamma_A}, B_{\alpha_B, \gamma_B} \in \text{Abs}(C)$, B is a *refinement* of A , denoted by $B \leq A$, when $\gamma_A(A) \subseteq \gamma_B(B)$ holds, i.e., when B is at least as expressive as A . An abstract domain $A_{\alpha, \gamma} \in \text{Abs}(C)$ is *trivial* if either (a) $\gamma\alpha = \text{id}_C$ holds, i.e., all the concrete values are expressible in A or (b) $\gamma\alpha = \lambda x. \top_C$ holds, i.e., A is a singleton domain that can express the greatest element $\top_C$ only. A is called the *identity abstraction* in the case (a) and the *top abstraction* in the case (b).

2.1.2 Correctness. Given an abstract domain $A_{\alpha, \gamma} \in \text{Abs}(C)$ and a concrete operation $f : C \rightarrow C$ (a generalization to n -ary functions of type $C^n \rightarrow C$ can be easily done pointwise), an abstract function $f^\# : A \rightarrow A$ is a *correct* (or *sound*) approximation (or abstract interpretation) of f when $\alpha f \leq f^\# \alpha$ holds. It is known that if $f^\#$ is a correct approximation of f , then we also have *fixpoint correctness* when least fixpoints exist, i.e., $\alpha(\text{lfp}(f)) \leq \text{lfp}(f^\#)$ holds. The bca of f in

A is defined as the abstract function $f^A \triangleq \alpha f \gamma : A \rightarrow A$. The term “best correct approximation” is justified by the well-known fact [Cousot and Cousot 1979] that an abstract function $f^\# : A \rightarrow A$ is a correct approximation of f iff $f^A \leq f^\#$ holds, so that f^A is the most precise, w.r.t. the pointwise ordering $\leq_A$, among the correct approximations of f on A .

2.1.3 Completeness. The abstract function $f^\#$ is a *complete* approximation of f (or just complete) if $\alpha f = f^\# \alpha$ holds. The abstract domain A is called a *complete abstraction* for f if there exists a complete approximation $f^\# : A \rightarrow A$ of f on the abstract domain A . Completeness intuitively encodes the greatest possible precision for an abstract function $f^\#$ defined on A , meaning that the abstract behaviour of $f^\#$ on A , i.e., $f^\# \alpha$, exactly matches the abstraction in A of the concrete behaviour of f , i.e., αf . In a complete approximation $f^\#$ the only loss of precision is due to the abstract domain and not to the definition of the abstract function itself. Analogously to soundness, completeness transfers to fixpoints, meaning that if $f^\#$ is complete for f , then *fixpoint completeness* $\alpha(\text{lfp}(f)) = \text{lfp}(f^\#)$ holds. It turns out that there exists an abstract function $f^\# : A \rightarrow A$ such that completeness $\alpha f = f^\# \alpha$ holds iff $\alpha f = \alpha f \gamma \alpha$ iff $(\gamma \alpha) f = (\gamma \alpha) f (\gamma \alpha)$ iff $A f = A f A$ iff $\gamma \alpha f = \gamma f^A \alpha$. Hence, the chance of defining a complete approximation $f^\#$ of f on some abstract domain A only depends upon the bca f^A of f in A , i.e., completeness is a property of the abstract domain only. Moreover, let us observe that any trivial abstract domain is always complete for any f . In the following, we say both “ A is complete for f ” and “ f is complete on A ,” and we write $\mathbb{C}^A(f)$ to denote that A is complete for f :

$$\mathbb{C}^A(f) \stackrel{\Delta}{\Leftrightarrow} A \circ f = A \circ f \circ A. \quad (1)$$

2.2 Regular Commands

Following O’Hearn [2020] for incorrectness logic, we consider a language of *regular commands*:

$$\text{Reg} \ni r ::= e \mid r; r \mid r \oplus r \mid r^*,$$

which is general enough to cover deterministic imperative languages as well as other programming paradigms that include, e.g., nondeterministic and probabilistic computations. The language is parametric on the syntax of basic expressions $e \in \text{Exp}$, which provide the basic commands and can be instantiated with different kinds of instructions such as (nondeterministic or parallel) assignments, (Boolean) guards or assumptions, error generation primitives, local variable primitives, and so on. The term $r_1; r_2$ represents sequential composition, the term $r_1 \oplus r_2$ represents a nondeterministic choice command, and the term r^* is the Kleene iteration of r where r can be executed 0 or any finite number of times. As a shorthand, we write r^n for the sequence $r; \dots; r$ of n instances of r and let $\text{Exp}(r)$ denote the set of basic expressions occurring in $r \in \text{Reg}$.

2.2.1 Concrete Semantics. We assume that basic expressions have a semantics $\langle \cdot \rangle : \text{Exp} \rightarrow C \rightarrow C$ on a complete lattice C such that $\langle e \rangle$ is an additive function. This assumption can be done w.l.o.g. in Hoare-like (or collecting) program semantics, since their basic functions are always defined by an additive lifting. The concrete semantics $\llbracket \cdot \rrbracket : \text{Reg} \rightarrow C \rightarrow C$ of regular commands is inductively defined as follows:

$$\begin{aligned} \llbracket e \rrbracket c &\triangleq \langle e \rangle c \\ \llbracket r_1; r_2 \rrbracket c &\triangleq \llbracket r_2 \rrbracket (\llbracket r_1 \rrbracket c) \\ \llbracket r_1 \oplus r_2 \rrbracket c &\triangleq \llbracket r_1 \rrbracket c \vee \llbracket r_2 \rrbracket c \\ \llbracket r^* \rrbracket c &\triangleq \bigvee \{ \llbracket r \rrbracket^n c \mid n \in \mathbb{N} \}. \end{aligned} \quad (2)$$

2.2.2 Abstract Semantics. The abstract semantics $\llbracket \cdot \rrbracket_A^\# : \text{Reg} \rightarrow A \rightarrow A$ of regular commands on an abstract domain $A_{\alpha, \gamma} \in \text{Abs}(C)$ is inductively defined as follows:

$$\begin{aligned} \llbracket e \rrbracket_A^\# a &\triangleq \llbracket e \rrbracket^A a \\ \llbracket r_1; r_2 \rrbracket_A^\# a &\triangleq \llbracket r_2 \rrbracket_A^\# (\llbracket r_1 \rrbracket_A^\# a) \\ \llbracket r_1 \oplus r_2 \rrbracket_A^\# a &\triangleq \llbracket r_1 \rrbracket_A^\# a \vee_A \llbracket r_2 \rrbracket_A^\# a \\ \llbracket r^* \rrbracket_A^\# a &\triangleq \bigvee_A \{ (\llbracket r \rrbracket_A^\#)^n a \mid n \in \mathbb{N} \}, \end{aligned} \tag{3}$$

where it is worth emphasizing that as abstract semantics $\llbracket e \rrbracket_A^\#$ of basic expressions we consider their bcas on A , namely $\llbracket e \rrbracket^A \triangleq \alpha \circ \llbracket e \rrbracket \circ \gamma$, i.e., we assume that no additional loss of precision is due to their interpretation. It is easy to check by structural induction that the abstract semantics in Equation (3) is monotonic (provided that $\llbracket e \rrbracket$ are monotone functions) and correct, i.e.,

$$\alpha \llbracket r \rrbracket \leq \llbracket r \rrbracket_A^\# \alpha. \tag{4}$$

Let us remark that Equation (3) is the standard definition by *structural induction* of abstract semantics used in abstract interpretation, adapted to the language of regular commands. Therefore, it turns out that the abstract semantics of the choice command preserves bcas, namely

$$\llbracket r_1 \oplus r_2 \rrbracket^A a = \llbracket r_1 \rrbracket^A a \vee_A \llbracket r_2 \rrbracket^A a. \tag{5}$$

As observed above in Section 1, this property of preserving bcas, in general, does not hold for sequential composition and Kleene iteration: For example, $\llbracket r_2 \rrbracket^A \circ \llbracket r_1 \rrbracket^A$ is not guaranteed to be the bca $\llbracket r_1; r_2 \rrbracket^A$. However, it can be easily seen, by structural induction, that all the definitions in Equation (3) preserve the property of being complete, meaning that if $\llbracket r_1 \rrbracket_A^\#, \llbracket r_2 \rrbracket_A^\#, \llbracket r \rrbracket_A^\#$ are complete, then $\llbracket r_1; r_2 \rrbracket_A^\#, \llbracket r_1 \oplus r_2 \rrbracket_A^\#$ and $\llbracket r^* \rrbracket_A^\#$ are complete as well. In the following, as a shorthand, we write $\mathbb{C}^A(r)$ instead of $\mathbb{C}^A(\llbracket r \rrbracket)$ to denote that A is complete for $\llbracket r \rrbracket$.

In our proofs, we will exploit some standard properties of abstract and concrete semantics, as summarized by the following lemma.

LEMMA 2.1. *Let $A_{\alpha, \gamma} \in \text{Abs}(C)$. For all $r \in \text{Reg}$ and $c, d \in C$:*

- (a) $\llbracket r \rrbracket^A \alpha(c) \leq_A \llbracket r \rrbracket_A^\# \alpha(c)$;
- (b) $\llbracket r \rrbracket_A^\# \alpha(c) = \alpha(\llbracket r \rrbracket c) \Rightarrow \alpha(\llbracket r \rrbracket c) = \alpha(\llbracket r \rrbracket \gamma \alpha(c))$;
- (c) $\llbracket r \rrbracket_A^\# \alpha(c) \leq_A \alpha(c) \Rightarrow \llbracket r^* \rrbracket_A^\# \alpha(c) = \alpha(c)$;
- (d) $\llbracket r \rrbracket^A \alpha(c) \leq_A \alpha(c) \Rightarrow \llbracket r^* \rrbracket^A \alpha(c) = \alpha(c)$;
- (e) $d \leq \llbracket r \rrbracket c \Rightarrow \llbracket r^* \rrbracket (c \vee d) = \llbracket r^* \rrbracket c$.

PROOF. Let us recall that $A = \gamma \alpha$.

- (a) By correctness (4), and by definition of bca $\llbracket r \rrbracket^A$.
- (b) By (a), $\alpha(\llbracket r \rrbracket c) \leq_A \alpha(\llbracket r \rrbracket \gamma \alpha(c)) = \llbracket r \rrbracket_A^\# \alpha(c) \leq_A \llbracket r \rrbracket_A^\# \alpha(c) = \alpha(\llbracket r \rrbracket c)$.
- (c) Let us assume that $\llbracket r \rrbracket_A^\# \alpha(c) \leq \alpha(c)$. The following property can be proved by an easy induction on $n \geq 1$:

$$\forall n \geq 1. \llbracket r \rrbracket_A^\# n \alpha(c) \leq_A \alpha(c). \tag{†}$$

Thus,

$$\begin{aligned} \llbracket r^* \rrbracket_A^\# \alpha(c) &= \text{[by definition]} \\ \bigvee_A \{ \llbracket r \rrbracket_A^\# n \alpha(c) \mid n \in \mathbb{N} \} &= \text{[by (†) and } \llbracket r \rrbracket_A^\# 0 \alpha(c) = \alpha(c)] \\ &\alpha(c). \end{aligned}$$

- (d) The proof is analogous to that of (c) and therefore omitted.

(e) Let us assume that $d \leq \llbracket r \rrbracket c$. Then

$$\begin{aligned}
 \llbracket r^* \rrbracket c &\leq \quad [\text{by monotonicity of } \llbracket r^* \rrbracket] \\
 \llbracket r^* \rrbracket (c \vee d) &\leq \quad [\text{by hyp. } d \leq \llbracket r \rrbracket c] \\
 \llbracket r^* \rrbracket (c \vee \llbracket r \rrbracket c) &\leq \quad [\text{as } \llbracket r \rrbracket c \leq \llbracket r^* \rrbracket c] \\
 \llbracket r^* \rrbracket (c \vee \llbracket r^* \rrbracket c) &= \quad [\text{as } c \leq \llbracket r^* \rrbracket c] \\
 \llbracket r^* \rrbracket (\llbracket r^* \rrbracket c) &= \quad [\text{by idempotency of } \llbracket r^* \rrbracket] \\
 \llbracket r^* \rrbracket c. &
 \end{aligned}$$

□

2.2.3 Programs. We consider standard basic expressions used in deterministic while programs: no-op instruction, assignments, and Boolean guards, as defined below:

$$\text{Exp} \ni e ::= \mathbf{skip} \mid x := a \mid b?$$

where a ranges over arithmetic expressions on integer values in $\mathbb{Z}$ and variables $x \in \text{Var}$, and b ranges over Boolean expressions in BExp including negation. Hence, a standard deterministic imperative language Imp (cf. Winskel [1993]) can be defined using guarded branching and loop commands as syntactic sugar as follows (cf. Kozen [1997, Section 2.2]):

$$\begin{aligned}
 \text{if } b \text{ then } c_1 \text{ else } c_2 &\triangleq (b?; c_1) \oplus (\neg b?; c_2) \\
 \text{while } b \text{ do } c &\triangleq (b?; c)^*; \neg b?
 \end{aligned} \tag{6}$$

The syntax of Imp commands is defined by the following grammar:

$$\text{Imp} \ni c ::= \mathbf{skip} \mid x := a \mid c; c \mid \text{if } b \text{ then } c \text{ else } c \mid \text{while } b \text{ do } c$$

To improve readability, in our running examples we will often use this syntactic sugar.

A program store $\sigma : V \rightarrow \mathbb{Z}$ is a total function from a finite set of variables of interest $V \subseteq \text{Var}$ to values and $\Sigma \triangleq V \rightarrow \mathbb{Z}$ denotes the set of stores on the variables ranging in a set V that, for simplicity, is left implicit. This definition of stores allows us to compose logical proofs for regular commands by choosing a finite set of variables V that is large enough to include all the variables occurring in a finite set of commands. The concrete domain is $\mathbb{S} \triangleq \wp(\Sigma)$, ordered by inclusion.

The semantics $\llbracket \cdot \rrbracket : \text{Exp} \rightarrow \mathbb{S} \rightarrow \mathbb{S}$ of basic expressions is defined as follows:

$$\begin{aligned}
 \llbracket \mathbf{skip} \rrbracket p &\triangleq p \\
 \llbracket x := a \rrbracket p &\triangleq \{\sigma[x \mapsto \llbracket a \rrbracket \sigma] \mid \sigma \in p\} \\
 \llbracket b? \rrbracket p &\triangleq \{\sigma \in p \mid \llbracket b \rrbracket \sigma = \mathbf{tt}\},
 \end{aligned}$$

where store update $\sigma[x \mapsto v]$ and the semantics of arithmetic expressions $\llbracket a \rrbracket : \Sigma \rightarrow \mathbb{Z}$ and Boolean expressions $\llbracket b \rrbracket : \Sigma \rightarrow \{\mathbf{tt}, \mathbf{ff}\}$ are defined as expected.

For brevity, we overload b to denote the set $\llbracket b? \rrbracket \Sigma$ of all and only stores that satisfy b , so that $\llbracket b? \rrbracket p = p \cap b$ filters the concrete stores in p making b true. The usual strongest post-condition for r for a pre-condition $p \in \mathbb{S}$ is therefore $\text{post}[r]p \triangleq \llbracket r \rrbracket p$. Analogously, we define $\text{post}_A[r]\alpha(p) \triangleq \llbracket r \rrbracket_A^\# \alpha(p)$.

In the following, we will present some simple running examples involving programs with just one variable, so that $V = \{x\}$. In these cases, to simplify the notation, $\wp(\mathbb{Z})$ will be used to represent sets of stores in $\mathbb{S}$, i.e., $p \in \wp(\mathbb{Z})$ represents the set $\{\sigma \in \Sigma \mid \sigma(x) \in p\} \in \mathbb{S}$. Accordingly, $\text{Abs}(\wp(\mathbb{Z}))$ will represent $\text{Abs}(\mathbb{S})$. For example, $\{-2, 2\}$ will be used to represent a verbose expression such as $x = -2 \vee x = 2$.

The term *global* in the section title refers to the universal quantification over any possible set S of stores in Equation (7), which we prove to be a major limitation. The following results provide a sufficient and necessary condition on the abstract domain A for guaranteeing both $\mathbb{C}^A(b?)$ and $\mathbb{C}^A(\neg b?)$. We first observe that when the functions $\llbracket b? \rrbracket$ and $\llbracket \neg b? \rrbracket$ are complete in a strict abstract domain A , then b and $\neg b$ are expressible in A , that is, $A(b) = b$ and $A(\neg b) = \neg b$ hold.

LEMMA 3.2. *Let $A \in \text{Abs}(\mathbb{S})$ be strict. If $\mathbb{C}^A(b?)$ and $\mathbb{C}^A(\neg b?)$ hold, then b and $\neg b$ are expressible in A .*

PROOF. Let us show the contrapositive, so we assume that either $A(b) \neq b$ or $A(\neg b) \neq \neg b$. Since, for all $p \in \mathbb{S}$, $p \subseteq A(p)$ holds, we have that $b \subsetneq A(b)$ or $\neg b \subsetneq A(\neg b)$, so that either $A(b) \cap \neg b \neq \emptyset$ or $A(\neg b) \cap b \neq \emptyset$. Assume that $A(b) \cap \neg b \neq \emptyset$ holds. Then $A(b \cap \neg b) = A(\emptyset) = \emptyset$ while $\emptyset \subsetneq A(b) \cap \neg b \subseteq A(A(b) \cap \neg b)$, so that $\mathbb{C}^A(\neg b)$ does not hold. The case $A(\neg b) \cap b \neq \emptyset$ is symmetric. $\square$

Furthermore, when b and $\neg b$ are both expressible in A , it turns out that the completeness of $\llbracket b? \rrbracket$ and $\llbracket \neg b? \rrbracket$ boils down to a co-additivity condition for the abstraction map α , or, equivalently, an additivity condition for the concretization map γ . This is clearly a way too strong requirement in abstract interpretation as co-additive abstraction maps imply that the abstract domain is (isomorphic to) a complete sublattice of the concrete domain, namely it is a disjunctive abstraction [Giacobazzi and Ranzato 1996, 1998].

LEMMA 3.3. *Let b and $\neg b$ be expressible in $A_{\alpha, \gamma} \in \text{Abs}(\mathbb{S})$. Then, $\mathbb{C}^A(b?)$ and $\mathbb{C}^A(\neg b?)$ hold iff*

$$\forall p \in \mathbb{S}. \quad \alpha(p \cap b) = \alpha(p) \wedge_A \alpha(b) \quad \wedge \quad \alpha(p \cap \neg b) = \alpha(p) \wedge_A \alpha(\neg b). \quad (8)$$

PROOF. We show that Equation (7) is equivalent to Equation (8). Observe that for all $p \in \mathbb{S}$,

$$\begin{aligned} \alpha(\gamma\alpha(p) \cap b) &= [\text{because } b \text{ is expressible in } A] \\ \alpha(\gamma\alpha(p) \cap \gamma\alpha(b)) &= [\text{by GI, } \gamma \text{ is co-additive}] \\ \alpha\gamma(\alpha(p) \wedge_A \alpha(b)) &= [\text{by GI, } \alpha\gamma = id] \\ \alpha(p) \wedge_A \alpha(b). \end{aligned}$$

Thus, when b and $\neg b$ are expressible, it turns out that $\alpha(\gamma\alpha(p) \cap b) = \alpha(p \cap b)$ iff $\alpha(p) \wedge_A \alpha(b) = \alpha(p \cap b)$. Symmetrically, $\alpha(\gamma\alpha(p) \cap \neg b) = \alpha(p \cap \neg b)$ iff $\alpha(p) \wedge_A \alpha(\neg b) = \alpha(p \cap \neg b)$. We have therefore proved (7) $\Leftrightarrow$ (8). $\square$

The next characterization result provides an effective way to check whether an abstract domain A is complete w.r.t. a Boolean guard b . It amounts to check that b and $\neg b$ are both expressible in A and that the union of the concretizations of any two abstract points in A below, respectively, $\alpha(b)$ and $\alpha(\neg b)$, is also expressible in A (see Example 3.5).

THEOREM 3.4 (COMPLETE GUARDS). *Let b and $\neg b$ be expressible in $A_{\alpha, \gamma} \in \text{Abs}(\mathbb{S})$. Then, $\mathbb{C}^A(b?)$ and $\mathbb{C}^A(\neg b?)$ hold iff*

$$\forall a_1, a_2 \in A. \quad (a_1 \leq_A \alpha(b) \wedge a_2 \leq_A \alpha(\neg b) \Rightarrow \gamma(a_1 \vee_A a_2) = \gamma(a_1) \cup \gamma(a_2)). \quad (9)$$

PROOF. Let us first show that $\mathbb{C}^A(b) \wedge \mathbb{C}^A(\neg b)$ is equivalent to Condition (9). We know that $\gamma\alpha(b) = b$ and $\gamma\alpha(\neg b) = \neg b$. By co-additivity of γ , for any $p \in \mathbb{S}$, we have that

$$\gamma(\alpha(p) \wedge_A \alpha(b)) = \gamma\alpha(p) \cap b \quad \text{and} \quad \gamma(\alpha(p) \wedge_A \alpha(\neg b)) = \gamma\alpha(p) \cap \neg b. \quad (§)$$

By Lemma 3.3, to prove our statement we show that (8) $\Leftrightarrow$ (9) holds.

($\Rightarrow$) By contraposition we prove that if Equation (9) is not satisfied, then Equation (8) does not hold. Let us assume that there exist $a_1 \leq_A \alpha(b)$ and $a_2 \leq_A \alpha(\neg b)$ such that $\gamma(a_1 \vee_A a_2) \supsetneq \gamma(a_1) \cup \gamma(a_2)$, so

that there exists $\sigma \in \gamma(a_1 \vee_A a_2)$ such that $\sigma \notin \gamma(a_1) \cup \gamma(a_2)$. Clearly, either $\sigma \in b$ or $\sigma \in \neg b$. Without loss of generality we assume that $\sigma \in b$ (the case $\sigma \in \neg b$ is symmetric). Let $p \triangleq \gamma(a_1) \cup \gamma(a_2)$ so that:

$$\begin{aligned} \gamma\alpha(p \cap b) &= \text{[by definition of } p\text{]} \\ \gamma\alpha((\gamma(a_1) \cup \gamma(a_2)) \cap b) &= \text{[by distributivity]} \\ \gamma\alpha((\gamma(a_1) \cap b) \cup (\gamma(a_2) \cap b)) &= \text{[by } \gamma(a_1) \subseteq \gamma\alpha(b) = b \text{ and } \gamma(a_2) \subseteq \gamma\alpha(\neg b) = \neg b\text{]} \\ \gamma\alpha\gamma(a_1) &= \text{[by GI, } \alpha\gamma = id\text{]} \\ \gamma(a_1). \end{aligned}$$

Thus, $\sigma \notin \gamma\alpha(p \cap b) = \gamma(a_1)$, because $\sigma \notin \gamma(a_1) \cup \gamma(a_2)$. Moreover,

$$\begin{aligned} \gamma(\alpha(p) \wedge_A \alpha(b)) &= \text{[by (§)]} \\ \gamma\alpha(p) \cap b &= \text{[by definition of } p\text{]} \\ \gamma(\alpha(\gamma(a_1) \cup \gamma(a_2))) \cap b &= \text{[by GI, } \alpha \text{ is additive]} \\ \gamma(\alpha\gamma(a_1) \vee_A \alpha\gamma(a_2)) \cap b &= \text{[by GI, } \alpha\gamma = id\text{]} \\ \gamma(a_1 \vee_A a_2) \cap b, \end{aligned}$$

so that $\sigma \in \gamma(\alpha(p) \wedge_A \alpha(b))$ because we have assumed $\sigma \in \gamma(a_1 \vee_A a_2)$ and $\sigma \in b$. Thus, $\gamma\alpha(p \cap b) \neq \gamma(\alpha(p) \wedge_A \alpha(b))$. Since γ is injective, $\alpha(p \cap b) \neq \alpha(p) \wedge_A \alpha(b)$, i.e., Equation (8) does not hold.

($\Leftarrow$) By contraposition, we prove that if Equation (8) fails, then Equation (9) does not hold. If Equation (8) does not hold, then there must exist some $p \in \mathbb{S}$ such that

$$\alpha(p \cap b) <_A \alpha(p) \wedge_A \alpha(b) \text{ or } \alpha(p \cap \neg b) <_A \alpha(p) \wedge_A \alpha(\neg b).$$

Thus, by applying γ , which, by GI, is injective, $\gamma\alpha(p \cap b) \subsetneq \gamma(\alpha(p) \wedge_A \alpha(b))$ or $\gamma\alpha(p \cap \neg b) \subsetneq \gamma(\alpha(p) \wedge_A \alpha(\neg b))$. In turn, since γ is co-additive and $b, \neg b$ are expressible in A , $\gamma\alpha(p \cap b) \subsetneq \gamma(\alpha(p)) \cap b$ or $\gamma\alpha(p \cap \neg b) \subsetneq \gamma(\alpha(p)) \cap \neg b$. Hence, we obtain that

$$\gamma\alpha(p \cap b) \cup \gamma\alpha(p \cap \neg b) \subsetneq \gamma(\alpha(p)). \quad (\dagger)$$

Let us now consider $a_1 \triangleq \alpha(p \cap b)$ and $a_2 \triangleq \alpha(p \cap \neg b)$. Then,

$$\begin{aligned} \gamma(a_1 \vee_A a_2) &= \text{[by definition]} \\ \gamma(\alpha(p \cap b) \vee_A \alpha(p \cap \neg b)) &= \text{[by GI, } \alpha \text{ is additive]} \\ \gamma(\alpha((p \cap b) \cup (p \cap \neg b))) &= \text{[by distributivity]} \\ \gamma(\alpha(p)) &\supsetneq \text{[by (\dagger)]} \\ \gamma\alpha(p \cap b) \cup \gamma\alpha(p \cap \neg b) &= \text{[by definition]} \\ \gamma(a_1) \cup \gamma(a_2), \end{aligned}$$

thus proving that Equation (9) does not hold. $\square$

Let us illustrate through some examples how Theorem 3.4 can be applied to check the completeness of some guards.

Example 3.5. Let $\text{Sign}, \text{Sign}_1, \text{Sign}_2 \in \text{Abs}(\wp(\mathbb{Z}))$ be the abstract domains depicted in Figure 3 and consider the programs AbsVal and ReLU defined in Section 1.

In Sign , all expressible Boolean guards are complete and the completeness of both the programs AbsVal and ReLU can be proved inductively.

In Sign_1 , no expressible Boolean guard is complete (except the trivial ones **tt** and **ff**). Indeed, only the elements $\emptyset$ and $\mathbb{Z}$ satisfy Condition (9). The negation of $\mathbb{Z}_{=0}$ is $\mathbb{Z}_{\neq 0}$, which does not belong to

Sign_1 . For the guard $\mathbb{Z}_{<0}$, its negation $\mathbb{Z}_{\geq 0}$ is in Sign_1 , but the join of $\mathbb{Z}_{<0}$ with $\mathbb{Z}_{>0} \leq_{\text{Sign}_1} \mathbb{Z}_{\geq 0}$ is again $\mathbb{Z}_{\neq 0} \notin \text{Sign}_1$. Dually for $\mathbb{Z}_{>0}$.

In Sign_2 , the guards $\mathbb{Z}_{\geq 0}$ and $\mathbb{Z}_{<0}$ are complete, while $\mathbb{Z}_{\leq 0}$ and $\mathbb{Z}_{=0}$ are not. For example, for $\mathbb{Z}_{=0} \leq_{\text{Sign}_2} \mathbb{Z}_{\geq 0}$ and $\mathbb{Z}_{<0}$ we have $\gamma(\mathbb{Z}_{=0} \vee_{\text{Sign}_2} \mathbb{Z}_{<0}) = \gamma(\mathbb{Z}_{\leq 0}) = \gamma(\mathbb{Z}_{=0}) \cup \gamma(\mathbb{Z}_{<0})$. It follows that, even if both ReLU and AbsVal are complete in Sign_2 , only ReLU can be inductively proved to be complete, because all its basic expressions are complete in Sign_2 . In the case of AbsVal instead, the assignment $x := -x$ is not complete, e.g., for $p \triangleq x > 0$ (the fact that such input will never be provided to that branch of code is irrelevant).

It is worth remarking that Condition (9) of Theorem 3.4 suggests a strategy to make a given abstract domain A complete for b and $\neg b$: We can either add to A the concrete values $\gamma(a_1) \cup \gamma(a_2)$ that are missing in A or remove either a_1 or a_2 from A when $\gamma(a_1) \cup \gamma(a_2)$ is not in A .

Example 3.6. Consider the domain $\text{Int}_{\mathbb{N}} \in \text{Abs}(\wp(\mathbb{N}))$ of (possibly infinite) intervals on natural numbers $\mathbb{N}$ [Cousot and Cousot 1977] and assume we are interested in testing equality to 0. The Boolean guards $x = 0?$ and $x \neq 0?$ are expressible in $\text{Int}_{\mathbb{N}}$ as the intervals $[0, 0]$ and $[1, \infty]$, respectively. However, $\emptyset$ is the only (trivial) interval below $[0, 0]$ and, besides $\emptyset$, any element of the form $[a, b]$, with $1 < a \leq b$, is below $[1, \infty]$. Theorem 3.4 requires that any element in the set $\{[0, 0] \cup [a, b] \mid 1 < a \leq b\}$ is also exactly represented. Therefore, an abstract domain that is complete for the guards $x = 0?$ and $x \neq 0?$ can be obtained by adding to $\text{Int}_{\mathbb{N}}$ all the elements in $\{[0, 0] \cup [a, b] \mid 1 < a \leq b\}$. Observe that $\text{Int}_{\mathbb{N}} \cup \{[0, 0] \cup [a, b] \mid 1 < a \leq b\}$ is Moore closed (see Section 2.1.1) and therefore it is an abstract domain of $\wp(\mathbb{N})$ (cf. Section 2.1.1). Note that only the guards $x = 0?$ and $x \neq 0?$ are complete on this refined abstract domain.

Because all interesting programs include Boolean guards, complete abstract domains refining a given domain may indeed become very close to the concrete domain, therefore limiting the effectiveness of this notion of completeness in program analysis. The following example, similar to Example 3.6, shows that this phenomenon can be a major drawback of refining an abstract domain to achieve completeness for the Boolean guards occurring in a given program.

Example 3.7. Consider the abstract domain $\text{Int} \in \text{Abs}(\wp(\mathbb{Z}))$ of integer intervals [Cousot and Cousot 1977]. The only Boolean guards b such that both b and $\neg b$ are expressible in Int are the infinite intervals $[-\infty, k]$ and $[k, \infty]$, for some $k \in \mathbb{Z}$, together with the trivial intervals $\mathbb{Z}$ and $\emptyset$. In fact, in Int , the complement of any finite interval $[a, b] \in \text{Int}$, with $a \leq b$, must be necessarily approximated. However, if we consider $b = [-\infty, k]$ and, correspondingly, $\neg b = [k + 1, \infty]$, then Condition (9) of Theorem 3.4 is not satisfied. As an example, let us fix $k = -1$, i.e., $b = [-\infty, -1]$ and, correspondingly, $\neg b = [0, \infty]$. Condition (9) of Theorem 3.4 would require the presence of all the concrete joins $[n_1, n_2] \cup [m_1, m_2]$ with $n_1 \leq n_2 < 0 \leq m_1 \leq m_2$, because $[n_1, n_2] \leq_{\text{Int}} [-\infty, -1]$ and $[m_1, m_2] \leq_{\text{Int}} [0, \infty]$, but these joins are not intervals, unless $n_2 = 0$ and $m_1 = 1$. If we add all such joins $[n_1, n_2] \cup [m_1, m_2]$ to Int , then we obtain a Moore closed subset of $\wp(\mathbb{Z})$ and therefore we achieve a complete abstraction for the guards $x < 0?$ and $x \geq 0?$.

Even a basic guard such as $b \triangleq \mathbb{Z}_{=0} = [0, 0]$ would need its complement $\neg b = \mathbb{Z}_{\neq 0} = [-\infty, -1] \cup [1, \infty]$ as well as the concrete joins $[n_1, n_2] \cup [0, 0]$ for any $n_1 \leq n_2 < -1$ or $1 < n_1 \leq n_2$, because any such interval $[n_1, n_2]$ is below $[-\infty, -1] \cup [1, \infty]$. Moreover, notice that the intersection of $\neg b$ with any interval $[n, m]$ with $n < 0 < m$ is $[n, -1] \cup [1, m]$, and since abstract domains are Moore closed, all these sets must be included as well.

4 LOCAL COMPLETENESS

Section 3 shows that the standard notion of (global) completeness (1) for Boolean guards is a too strong requirement for abstract domains, often met in practice just by trivial guards or domains.

While completeness can be hard/impossible to achieve *globally*, i.e., for *all* possible sets of stores, it could well happen that completeness holds *locally*, i.e., just for *some* store properties. We therefore put forward a notion of *local completeness*, which in program analysis corresponds to considering completeness only along certain program executions.

Definition 4.1 (Local Completeness). An abstract domain $A \in \text{Abs}(C)$ is *locally complete* for $f : C \rightarrow C$ at a concrete value $c \in C$, written $\mathbb{C}_c^A(f)$, if the following condition holds:

$$\mathbb{C}_c^A(f) \stackrel{\Delta}{\Leftrightarrow} Af(c) = AfA(c).$$

Accordingly, ordinary completeness as defined by Equation (1) is also referred to as *global completeness*. Note that global completeness amounts to the universal quantification of local completeness, in the sense that $\mathbb{C}^A(f) \Leftrightarrow \forall c \in C. \mathbb{C}_c^A(f)$. Let us also observe that A is trivially locally complete for any f on any abstract value $A(c)$ (i.e., $\mathbb{C}_{A(c)}^A(f)$ always holds). As discussed in Section 1, in program analysis it may well happen that $\llbracket r \rrbracket$ is not globally complete w.r.t. the abstract domain A but it is locally complete for a particular class of inputs p (that is a value in the concrete domain of powerset of stores). In the following, we write $\mathbb{C}_p^A(r)$ instead of the more verbose $\mathbb{C}_p^A(\llbracket r \rrbracket)$.

Example 4.2. Consider the following Imp program

$$c \triangleq \text{if } (0 < x) \text{ then } x := x - 2 \text{ else } x := -x$$

and the interval abstraction Int . While the transfer function $\llbracket 0 < x? \rrbracket$ is not globally complete (see Example 3.7), it is locally complete for any set $p \in \wp(\mathbb{Z})$ satisfying one of the following conditions:

$$(1) p \subseteq \mathbb{Z}_{>0}, \text{ or } (2) p \subseteq \mathbb{Z}_{\leq 0}, \text{ or } (3) \{0, 1\} \subseteq p.$$

Since the transfer functions for constant addition and multiplication are globally complete, the program c is locally complete for any p satisfying one of the above conditions (1)–(3), meaning that its abstract interpretation in Int will not lose precision. For example, if $p = \{0, 1, 4\}$, then condition (3) holds and we have that $\text{Int}(\llbracket c \rrbracket p) = \text{Int}(\{-1, 0, 2\}) = [-1, 2]$ and $\text{Int}(\llbracket c \rrbracket \text{Int}(p)) = \text{Int}(\llbracket c \rrbracket [0, 4]) = \text{Int}(\{-1, 0, 1, 2\}) = [-1, 2]$. As an example of local incompleteness, if $p = \{0, 4\}$, then we have that $\text{Int}(\llbracket c \rrbracket p) = \text{Int}(\{0, 2\}) = [0, 2]$, but $\text{Int}(\llbracket c \rrbracket \text{Int}(p)) = [-1, 2]$.

Let us remark that, with respect to compositional reasoning, there is a significant key difference between global and local completeness: While the composition (via generic regular commands operators, and consequently via conditionals and loops) of globally complete transfer functions is always globally complete, the same does not necessarily hold for local completeness that depends on a given input property. Equivalently, local completeness of a composite program may well depend on the partial store properties met during the computation, as shown by the following example.

Example 4.3. Consider a composition $c; c$, where c is defined in Example 4.2. Int is locally complete for c on the input property $p = \{2, 6\}$, because condition (1) of Example 4.2 holds. However, Int is not locally complete for $c; c$ on p , because $\text{Int}(\llbracket c; c \rrbracket \{2, 6\}) = \text{Int}(\llbracket c \rrbracket \{0, 4\}) = [0, 2]$ while $\text{Int}(\llbracket c; c \rrbracket \text{Int}(\{2, 6\})) = \text{Int}(\llbracket c; c \rrbracket [2, 6]) = \text{Int}(\llbracket c \rrbracket [0, 4]) = \text{Int}(\{-1, 0, 1, 2\}) = [-1, 2]$.

4.1 Locally Complete Boolean Guards

By focussing on Boolean guards, we can characterize the local completeness of both positive $b?$ and negative $\neg b?$ branches of a conditional statement $b \in \text{BExp}$ for a given input store property.

THEOREM 4.4 (LOCALLY COMPLETE GUARDS). Let $b \in \text{BExp}$, $A \in \text{Abs}(\mathbb{S})$ and $p \in \mathbb{S}$. Then, A is locally complete for both $\llbracket b? \rrbracket$ and $\llbracket \neg b? \rrbracket$ on p iff $(A(p \cap b) \cap b) \cup (A(p \cap \neg b) \cap \neg b) \in A$.

PROOF. ($\Rightarrow$): We assume that $\mathbb{C}_p^A(b)$ and $\mathbb{C}_p^A(\neg b)$ and prove that $(A(p \cap b) \cap b) \cup (A(p \cap \neg b) \cap \neg b)$ is expressible in A . We have that

$$\begin{aligned} A((A(p \cap b) \cap b) \cup (A(p \cap \neg b) \cap \neg b)) &\subseteq \text{ [by monotonicity]} \\ A(A(p \cap b) \cup A(p \cap \neg b)) &= \text{ [by GI, } \alpha \text{ is additive and } A(A(q) \cup A(w)) = A(q \cup w) \text{ for any } q, w\text{]} \\ A((p \cap b) \cup (p \cap \neg b)) &= \text{ [by set theoretic properties]} \\ &A(p), \end{aligned}$$

so that, by exploiting the hypothesis of local completeness of $\llbracket b? \rrbracket$ on p ,

$$A((A(p \cap b) \cap b) \cup (A(p \cap \neg b) \cap \neg b)) \cap b \subseteq A(p) \cap b \subseteq A(A(p) \cap b) \cap b = A(p \cap b) \cap b.$$

Likewise,

$$A((A(p \cap b) \cap b) \cup (A(p \cap \neg b) \cap \neg b)) \cap \neg b \subseteq A(p \cap \neg b) \cap \neg b.$$

Thus,

$$\begin{aligned} A((A(p \cap b) \cap b) \cup (A(p \cap \neg b) \cap \neg b)) &= \text{ [by set properties]} \\ (A((A(p \cap b) \cap b) \cup (A(p \cap \neg b) \cap \neg b)) \cap b) \cup (A((A(p \cap b) \cap b) \cup (A(p \cap \neg b) \cap \neg b)) \cap \neg b) &\subseteq \text{ [by above inclusions]} \\ &(A(p \cap b) \cap b) \cup (A(p \cap \neg b) \cap \neg b), \end{aligned}$$

meaning that $(A(p \cap b) \cap b) \cup (A(p \cap \neg b) \cap \neg b) \in A$.

($\Leftarrow$): For the converse implication, we assume that $(A(p \cap b) \cap b) \cup (A(p \cap \neg b) \cap \neg b) \in A$ and prove that both $\mathbb{C}_p^A(b)$ and $\mathbb{C}_p^A(\neg b)$ hold. It turns out that $p = (p \cap b) \cup (p \cap \neg b) \subseteq (A(p \cap b) \cap b) \cup (A(p \cap \neg b) \cap \neg b)$.

Therefore, by exploiting the hypothesis, we obtain that

$$A(p) \subseteq A((A(p \cap b) \cap b) \cup (A(p \cap \neg b) \cap \neg b)) = (A(p \cap b) \cap b) \cup (A(p \cap \neg b) \cap \neg b).$$

Thus, $A(p) \cap b \subseteq A(p \cap b) \cap b \subseteq A(p \cap b)$, in turn entailing that local completeness $A(A(p) \cap b) = A(p \cap b)$ holds. Analogously, we obtain that $A(A(p) \cap \neg b) = A(p \cap \neg b)$. $\square$

Example 4.5. Let us apply Theorem 4.4 to show that for the interval abstraction Int , the Boolean guard $0 < x$ is locally complete for any $p \in \wp(\mathbb{Z})$ such that (1) $p \subseteq \mathbb{Z}_{>0}$, or (2) $p \subseteq \mathbb{Z}_{\leq 0}$, or (3) $\{0, 1\} \subseteq p$. In fact, it turns out that positive and negative branches $\llbracket x > 0? \rrbracket$ and $\llbracket x \leq 0? \rrbracket$ are both locally complete on p iff

$$(\text{Int}(p \cap x > 0) \cap x > 0) \cup (\text{Int}(p \cap x \leq 0) \cap x \leq 0) \in \text{Int}. \quad (*)$$

Since $x > 0$ and $x \leq 0$ both belong to Int , condition $(*)$ boils down to

$$\text{Int}(p \cap x > 0) \cup \text{Int}(p \cap x \leq 0) \in \text{Int}. \quad (\star)$$

Now, it is just a matter of noticing that $(\star)$ holds iff either $\text{Int}(p \cap x \leq 0)$ is empty (condition (1)), or $\text{Int}(p \cap x > 0)$ is empty (condition (2)) or $\text{Int}(p \cap x \leq 0)$ and $\text{Int}(p \cap x > 0)$ are contiguous nonempty intervals, and this happens iff p includes both cut points 0 and 1 of the guard $0 < x$.

Likewise Theorem 3.4 can be used to construct domains that are complete for some Boolean guards, Theorem 4.4 gives a way to construct domains that are locally complete for some Boolean guards. The next example shows that, as expected, making an abstract domain locally complete for some guards is less demanding than making it (globally) complete for the same guards.

Example 4.6. In Example 3.7 we have shown that to make the domain Int globally complete for the guards $b \triangleq x < 0 = x \in [-\infty, -1]$ and $\neg b \triangleq x \geq 0 = [0, \infty]$ would require the addition of all the concrete joins $[n_1, n_2] \cup [m_1, m_2]$ with $n_1 \leq n_2 < 0 \leq m_1 \leq m_2$.

Given any input p , Theorem 4.4 shows that Int is made locally complete for the guards b and $\neg b$ by adding the unique element $(\text{Int}(p \cap b) \cap b) \cup (\text{Int}(p \cap \neg b) \cap \neg b)$ and taking the Moore closure. For

example, when $p \triangleq \{n \mid n \text{ odd}\}$ then $(\text{Int}(p \cap b) \cap b) \cup (\text{Int}(p \cap \neg b) \cap \neg b) = [-\infty, -1] \cup [1, \infty] = \mathbb{Z}_{\neq 0}$ is the unique element to add (and, by Moore closure, any concrete join of the form $[n, -1] \cup [1, m]$ with $n \leq -1$ and $m \geq 1$). Thus, we get the domain that contains all the intervals, possibly with a hole in 0.

5 LOCAL COMPLETENESS LOGIC

We define a proof system for program analysis of regular commands, parameterized by an abstraction A , whose provable triples $\vdash_A [p] \text{ r } [q]$ guarantee that

- (i) q is an *under-approximation* of $\llbracket r \rrbracket p$ (i.e., $q \subseteq \llbracket r \rrbracket p$);
- (ii) $\llbracket r \rrbracket$ is *locally complete* for input p and abstraction A (i.e., $\mathbb{C}_p^A(r)$ holds);
- (iii) q and $\llbracket r \rrbracket p$ have the same *over-approximation* in A (i.e., $A(q) = A(\llbracket r \rrbracket p)$).

Given a correctness specification $spec$, we recall that abstract interpretation raises an alarm when $\gamma(\llbracket r \rrbracket_A^\# \alpha(p)) \not\subseteq spec$: Such alarm is false if $\llbracket r \rrbracket p \subseteq spec$ and true otherwise. It turns out that the above three properties of any provable triple $\vdash_A [p] \text{ r } [q]$ allow us to distinguish between true and false alarms, as described below:

Case 1: If the over-approximation $\llbracket r \rrbracket_A^\# \alpha(p)$ does not raise alarms, i.e., $\gamma(\llbracket r \rrbracket_A^\# \alpha(p)) \subseteq spec$ holds, then the program r does not exhibit unwanted behaviours. It should be remarked that this already holds for any sound and possibly incomplete over-approximating abstract interpretation.

Case 2: If $spec$ is expressible in A and the abstract interpretation $\llbracket r \rrbracket_A^\# \alpha(p)$ raises some alarms because $\gamma(\llbracket r \rrbracket_A^\# \alpha(p)) \not\subseteq spec$, then, by local completeness, any provable triple $\vdash_A [p] \text{ r } [q]$ is such that $q \setminus spec \neq \emptyset$ and all the stores in $q \setminus spec$ are true alarms. As discussed in the Introduction, let us recall that by relying on generic, thus possibly incomplete, abstract interpretation we could not distinguish which alarms in $\gamma(\llbracket r \rrbracket_A^\# \alpha(p)) \setminus spec$ are true ones and which are false.

Case 3: If $spec$ is expressible in A , some alarm is raised because $\gamma(\llbracket r \rrbracket_A^\# \alpha(p)) \not\subseteq spec$ but any attempt to derive a triple $\vdash_A [p] \text{ r } [q]$ for some under-approximation q fails because some proof obligations of local completeness $\mathbb{C}_c^A(f)$ are not met, then the abstraction A is not precise enough to distinguish between true and false alarms in a compositional way (for r on p). In this case, one could refine the abstraction A to enhance its precision and repeat the analysis, possibly guided by the failed proof obligations (see Example 5.9 and also Section 8).

The logical proof system $\vdash_A$ is defined in Figure 4 and called LCL_A . Our objective in designing this deductive system has been to track the assumptions of local completeness needed for having a compositional proof. The distinctive rules are (transfer) and (relax) whose premises depend directly on the underlying abstraction A . For the readers' convenience, we copy rule (relax) as follows:

$$\frac{p' \leq p \leq A(p') \quad \vdash_A [p'] \text{ r } [q'] \quad q \leq q' \leq A(q)}{\vdash_A [p] \text{ r } [q]} \text{ (relax)}.$$

The combined consequence rule (relax) is the key principle that allows us to adapt and generalize partial proofs to broader contexts. The novelty of (relax) lies in combining an over- and under-approximating reasoning: (relax) allows us to infer a post-condition q that is an under-approximation of the exact behaviour but whose abstraction $A(q)$ is a sound over-approximation of it, i.e., such that $q \subseteq \llbracket r \rrbracket p \subseteq A(\llbracket r \rrbracket p) = A(q)$ holds. Likewise in the consequence rules of de Vries and Koutavas [2011], O'Hearn [2020], and Raad et al. [2020], the logical ordering between pre-conditions $p' \leq p$ and post-conditions $q \leq q'$ in the premises of (relax) is reversed w.r.t. the canonical consequence rule of Hoare logic and this is needed because our post-conditions q are always under-approximations. Let us also remark that the premises of (relax) imply that $A(p) = A(p')$ and

$\frac{\mathbb{C}_p^A(e)}{\vdash_A [p] e \llbracket [e] p \rrbracket} \text{ (transfer)}$	$\frac{p' \leq p \leq A(p') \quad \vdash_A [p'] r [q'] \quad q \leq q' \leq A(q)}{\vdash_A [p] r [q]} \text{ (relax)}$
$\frac{\vdash_A [p] r_1 [w] \quad \vdash_A [w] r_2 [q]}{\vdash_A [p] r_1; r_2 [q]} \text{ (seq)}$	$\frac{\vdash_A [p] r_1 [q_1] \quad \vdash_A [p] r_2 [q_2]}{\vdash_A [p] r_1 \oplus r_2 [q_1 \vee q_2]} \text{ (join)}$
$\frac{\vdash_A [p] r [w] \quad \vdash_A [p \vee w] r^* [q]}{\vdash_A [p] r^* [q]} \text{ (rec)}$	$\frac{\vdash_A [p] r [q] \quad q \leq A(p)}{\vdash_A [p] r^* [p \vee q]} \text{ (iterate)}$

Fig. 4. The proof system LCL_A .

$A(q) = A(q')$. Example 5.10 will show that a dual version of (relax) with p strengthening p' and q weakening q' as in the classical consequence rule of Hoare logic would not be sound w.r.t. local completeness.

The crux of (relax) is to constrain this under-approximating post-condition q to have *the same abstraction* as the exact behaviour to preserve the precision of the deduction. This opens up an interesting perspective about the generality of our proof system that will be tackled in Section 6 for showing how to recover O'Hearn [2020]'s IL as an instance of our proof system. Moreover, in Section 5.3 we also show how an easy dualization of LCL_A allows us to accomodate backward abstract reasoning as used in backward program analysis.

Technically, the validity of the rule (relax) relies on observing that local completeness is a kind of “abstract convex property,” meaning that if A is locally complete for some $c \in C$, then A is locally complete for all $d \in C$ such that $c \leq d \leq A(c)$ holds.

LEMMA 5.1. *If $\mathbb{C}_c^A(f)$ and $d \in [c, A(c)]$, then $\mathbb{C}_d^A(f)$.*

PROOF. Since $c \leq d \leq A(c)$, by monotonicity of f and A , we obtain that $Af(c) \leq Af(d) \leq AfA(c)$ and $AfA(c) \leq AfAA(c) = AfA(c)$. Since $AfA(c) = Af(c)$, we have that $Af(c) = Af(d)$. Finally, $Af(d) \leq AfA(d) \leq Af(c) = Af(d)$. $\square$

The rule (transfer) checks that the basic expressions e are locally complete on p and, in that case, provides the output of the corresponding transfer function $\llbracket [e] p \rrbracket$ on p as post-condition,

$$\frac{\mathbb{C}_p^A(e)}{\vdash_A [p] e \llbracket [e] p \rrbracket} \text{ (transfer)}.$$

Of course, for no-ops, Boolean guards and assignments, the rule (transfer) can be equivalently stated in symbolic form as follows:

$$\begin{array}{c} \frac{}{\vdash_A [p] \text{skip} [p]} \text{ (skip)} \quad \frac{\mathbb{C}_p^A(b?)}{\vdash_A [p] b? [p \wedge b]} \text{ (assume)} \\ \frac{\mathbb{C}_p^A(x := a)}{\vdash_A [p] x := a [\exists v. (p[v/x] \wedge x = a[v/x])]} \text{ (assign)}, \end{array}$$

where $[v/x]$ denotes the substitution for replacing x by v .

The rule (seq) for sequential composition and the rule (join) for choice are standard. The rule (rec) allows us to unfold one step of Kleene iteration, until the rule (iterate) can be applied.

The rule (iterate) is a distinguishing rule of LCL_A and is as much fundamental as rule (relax) for several reasons: Both rules have premises depending on the abstraction A ; under-approximated post-conditions are only introduced by these two rules (all the other rules are otherwise “exact”); while the concrete semantics of r^* can be infinitary (e.g., consider $(x := x + 1)^*$), using (iterate)

$$\begin{array}{c}
\frac{\mathbb{C}_{p_1}^{\text{Int}}(b_1?)}{\vdash_{\text{Int}} [p_1] b_1? [\{1, 999, 1000\}]} \quad (\text{tr.}) \quad \frac{\mathbb{C}_{\{1,999,1000\}}^{\text{Int}}(e_1)}{\vdash_{\text{Int}} [\{1, 999, 1000\}] e_1? [\{0, 998, 999\}]} \quad (\text{tr.}) \quad \frac{\mathbb{C}_{p_1}^{\text{Int}}(b_2?)}{\vdash_{\text{Int}} [p_1] b_2? [\{0, 1, 999\}]} \quad (\text{tr.}) \quad \frac{\mathbb{C}_{\{0,1,999\}}^{\text{Int}}(e_2)}{\vdash_{\text{Int}} [\{0, 1, 999\}] e_2 [\{1, 2, 1000\}]} \quad (\text{tr.}) \\
\vdash_{\text{Int}} [p_1] r_1 [\{0, 998, 999\}] \quad (\text{seq}) \quad \vdash_{\text{Int}} [p_1] r_2 [\{1, 2, 1000\}] \quad (\text{seq}) \\
\vdash_{\text{Int}} [p_1] r_1 \oplus r_2 [\{0, 1, 2, 998, 999, 1000\}] \quad (\text{join}) \\
\vdash_{\text{Int}} [p] r_1 \oplus r_2 [\{0, 1, 2, 998, 999, 1000\}] \quad (\text{iterate}) \\
(\star) \\
\frac{\mathbb{C}_p^{\text{Int}}(b_1?)}{\vdash_{\text{Int}} [p] b_1? [p]} \quad (\text{tr.}) \quad \frac{\mathbb{C}_p^{\text{Int}}(e_1)}{\vdash_{\text{Int}} [p] e_1 [\{0, 998\}]} \quad (\text{tr.}) \quad \frac{\mathbb{C}_p^{\text{Int}}(b_2?)}{\vdash_{\text{Int}} [p] b_2? [p]} \quad (\text{tr.}) \quad \frac{\mathbb{C}_p^{\text{Int}}(e_2)}{\vdash_{\text{Int}} [p] e_2 [\{2, 1000\}]} \quad (\text{tr.}) \\
\vdash_{\text{Int}} [p] r_1 [\{0, 998\}] \quad (\text{seq}) \quad \vdash_{\text{Int}} [p] r_2 [\{2, 1000\}] \quad (\text{seq}) \\
\vdash_{\text{Int}} [p] r_1 \oplus r_2 [\{0, 2, 998, 1000\}] \quad (\text{join}) \\
\vdash_{\text{Int}} [p] r [\{0, 2, 1000\}] \quad (\text{iterate}) \\
(\star) \\
\vdash_{\text{Int}} [\{0, 1, 999, 1000\}] r [\{0, 1, 2, 998, 999, 1000\}] \quad (\text{relax}) \\
\vdash_{\text{Int}} [\{0, 1, 2, 998, 999, 1000\}] r [\{0, 2, 1000\}] \quad (\text{rec}) \\
\vdash_{\text{Int}} [p] r [\{0, 2, 1000\}]
\end{array}$$

Legenda: $b_1 \triangleq 0 < x$ $e_1 \triangleq x := x - 1$ $b_2 \triangleq x < 1000$ $e_2 \triangleq x := x + 1$ $p_1 \triangleq \{0, 1, 999, 1000\}$

Fig. 5. Derivation of $\vdash_{\text{Int}} [p = \{1, 999\}] r [\{0, 2, 1000\}]$ for Example 5.2, where the label (tr.) stands for (transfer).

we can exploit the abstraction A to stop the proof when the abstraction of a finitary input p is already an infinitary abstract invariant³ (cf. Lemma 5.4), returning a finite under-approximation of the concrete invariant; the combination of under- and over-approximations in the rule (iterate) is therefore more expressive than the sum of its parts, as it allows us to speed up both program analysis and alarm detection.

The next two examples illustrate the key features of LCL_A: The first one exploits all the rules, and the second one is applied to a classical while-loop. They will be revisited in Section 5.1 to show how LCL_A can help in program analysis.

Example 5.2. Let us consider the interval domain Int , the pre-condition $p \triangleq \{1, 999\}$ and the command $r \triangleq (r_1 \oplus r_2)^*$, where

$$r_1 \triangleq (0 < x?; x := x - 1), \quad r_2 \triangleq (x < 1000?; x := x + 1).$$

The triple $\vdash_{\text{Int}} [p] r [\{0, 2, 1000\}]$ can be derived as shown in Figure 5, where for brevity we let

$$\begin{array}{lll}
b_1 \triangleq 0 < x & e_1 \triangleq x := x - 1 & p_1 \triangleq \{0, 1, 999, 1000\} \\
b_2 \triangleq x < 1000 & e_2 \triangleq x := x + 1 &
\end{array}$$

Notably, each instance of rule (transfer) used in the derivation exposes a proof obligation (such as $\mathbb{C}_p^{\text{Int}}(e_2)$, $\mathbb{C}_{p_1}^{\text{Int}}(b_1?)$, etc.) concerning the local completeness of a basic command. This proof needs just one application of (rec) to compute an under-approximation of $\text{post}[r]p = \llbracket r \rrbracket p$, because the rule (iterate) can stop the unfolding of the Kleene iterate operator as soon as an abstract invariant is detected, before the actual concrete invariant is fully computed (in this case the abstract invariant is detected by $\{0, 1, 2, 998, 999, 1000\} \subseteq [0, 1000]$). Moreover, (relax) is exploited to reduce the number of values to be taken into account (along the pre-conditions by navigating the derivation tree bottom-up and along the post-conditions when the tree is explored top-down).

Finally, a similar result is soon obtained on any input $p_k \triangleq \{k, 999\}$ for some $k \in \mathbb{N}$ by applying the rule (rec) for k times: Then the rule (iterate) can be used.

Example 5.3. Let us consider the domain Sign , the pre-condition $p \triangleq \{-10, -1, 100\}$, and the following Imp program,

$$\begin{aligned}
c &\triangleq \text{while } (x \leq 0) \text{ do } x := x * 10 \\
&= (x \leq 0?; x := x * 10)^*; 0 < x?
\end{aligned}$$

³A maybe non-obvious consequence of the condition $q \leq A(p)$.

Let us verify that c does not satisfy the correctness specification $spec \triangleq x < 10$, even if the loop c diverges on inputs $\{-10, -1\}$. The derivation in Figure 6 proves the triple $\vdash_{\text{Sign}} [p] c [\{100\}]$. As the post-condition $\{100\}$ is an under-approximation of $\llbracket r \rrbracket p$ (cf. Theorem 5.5 (1)), we conclude that $100 \notin spec$ is a true alarm. Observe that all proof obligations about local completeness due to rule (transfer) are satisfied, as, e.g., letting $b \triangleq x \leq 0$, for $\mathbb{C}_p^{\text{Sign}}(b?)$, we have that

$$\text{Sign}(\llbracket b? \rrbracket \text{Sign}(p)) = \text{Sign}(\llbracket b? \rrbracket \mathbb{Z}_{\neq 0}) = \text{Sign}(\mathbb{Z}_{<0}) = \mathbb{Z}_{<0} = \text{Sign}(\{-10, -1\}) = \text{Sign}(\llbracket b? \rrbracket p).$$

Of course, let us point out that some additional valid rules could be added to our proof system, for example the following two rules can be easily proved to be valid:

$$\frac{\vdash_A [p] r [q] \quad q \leq p}{\vdash_A [p] r^* [p]} \text{ (invariant)} \quad \frac{\vdash_A [p] r [q] \quad A(p) = A(q)}{\vdash_A [p] r^* [q]} \text{ (abs-fix)}.$$

LEMMA 5.4. *The rules (invariant) and (abs-fix) can be derived in LCL_A .*

PROOF. We first show that (invariant) is a derived rule. If $q \leq p$, then we have $A(p \vee q) = A(p)$. Since $\vdash_A [p] r [q]$, by (iterate) it follows $\vdash_A [p] r^* [p \vee q]$, i.e., $\vdash_A [p] r^* [p]$.

For (abs-fix), since γ is 1-1, if $A(q) = A(p)$, then $\alpha(p) = \alpha(q)$. Therefore, $A(p \vee q) = A(p)$, because α is additive and thus $A(p \vee q) = \gamma(\alpha(p \vee q)) = \gamma(\alpha(p) \vee_A \alpha(q)) = \gamma\alpha(q) = A(q)$. Since $\vdash_A [p] r [q]$ and $q \leq A(q) = A(p)$, by (iterate) it follows $\vdash_A [p] r^* [p \vee q]$. Since $q \leq (p \vee q) \leq A(p \vee q) = A(q)$, by (relax), we conclude $\vdash_A [p] r^* [q]$. $\square$

The rule (invariant) is the analogous of the loop invariant rule in Hoare logic, while (abs-fix) allows us to accelerate the convergence of Kleene iteration to an abstract fixpoint.

It is worth remarking that indeed (invariant) and (iterate) are equivalent rules within LCL_A . In fact, the proof of Lemma 5.4 shows that the rule (invariant) can be derived from the rule (iterate), and, vice versa, (iterate) can be derived using the rules (invariant), (rec) and (relax), as shown by the following inferences:

$$\frac{\frac{\frac{p \leq (p \vee q) \leq A(p) \quad \vdash_A [p] r [q]}{\vdash_A [p \vee q] r [q]} \text{ (relax)} \quad q \leq p \vee q}{\vdash_A [p \vee q] r^* [p \vee q]} \text{ (invariant)},}{\vdash_A [p] r^* [p \vee q]} \text{ (rec)}$$

where the proof obligation $q \leq p \vee q$ trivially holds, while $p \leq (p \vee q) \leq A(p)$ holds because the assumption $q \leq A(p)$ of the rule (iterate) implies that $p \leq (p \vee q) \leq (p \vee A(p)) = A(p)$.

5.1 Soundness

Our program logic LCL_A turns out to be sound for the target properties (i)–(iii) stated at the beginning of Section 5, as formalized by the following soundness result, where Conditions (1) and (2) embody, respectively, items (i) and (ii)+(iii).

THEOREM 5.5 (SOUNDNESS OF LCL_A). *Let $A_{\alpha, \gamma} \in \text{Abs}(C)$. For all $r \in \text{Reg}$, $p, q \in C$, if $\vdash_A [p] r [q]$, then*

- (1) $q \leq \llbracket r \rrbracket p$, and
- (2) $\llbracket r \rrbracket_A^\# \alpha(p) = \alpha(q) = \alpha(\llbracket r \rrbracket p)$.

PROOF. For the sake of simplicity, let us refer to the equality $\llbracket r \rrbracket_A^\# \alpha(p) = \alpha(q)$ as (2a) and use (2b) for the equality $\llbracket r \rrbracket_A^\# \alpha(p) = \alpha(\llbracket r \rrbracket p)$. Then we note that (2b) follows immediately from (1) and (2a) because, by monotonicity of α and correctness (4), we have that

$$\alpha(q) \leq_A \alpha(\llbracket r \rrbracket p) \leq_A \llbracket r \rrbracket_A^\# \alpha(p) = \alpha(q).$$

$$\begin{array}{c}
\frac{\mathbb{C}_p^{\text{Sign}}(x \leq 0?)}{\vdash_{\text{Sign}} [p] \ x \leq 0? [\{-10, -1\}]} \quad (\text{transfer}) \quad \frac{\mathbb{C}_{\{-10, -1\}}^{\text{Sign}}(x := x * 10)}{\vdash_{\text{Sign}} [\{-10, -1\}] \ x := x * 10 [\{-100, -10\}]} \quad (\text{transfer}) \\
\frac{\vdash_{\text{Sign}} [p] \ x \leq 0?; x := x * 10 [\{-100, -10\}] \quad \{-100, -10\} \subseteq \text{Sign}(p) = \mathbb{Z}_{\neq 0}}{\vdash_{\text{Sign}} [p] \ (x \leq 0?; x := x * 10) * [\{-100, -10, -1, 100\}] \quad \{-100, 100\} \subseteq \{-100, -10, -1, 100\} \subseteq \text{Sign}(\{-100, 100\}) = \mathbb{Z}_{\neq 0}} \quad (\text{seq}) \\
\frac{\vdash_{\text{Sign}} [p] \ (x \leq 0?; x := x * 10) * [\{-100, -10, -1, 100\}] \quad \{-100, 100\} \subseteq \{-100, -10, -1, 100\} \subseteq \text{Sign}(\{-100, 100\}) = \mathbb{Z}_{\neq 0}}{\vdash_{\text{Sign}} [p] \ (x \leq 0?; x := x * 10) * [\{-100, 100\}]} \quad (\text{iterate}) \\
\frac{\vdash_{\text{Sign}} [p] \ (x \leq 0?; x := x * 10) * [\{-100, 100\}]}{\vdash_{\text{Sign}} [p] \ c [\{100\}]} \quad (\text{relax}) \quad \frac{\mathbb{C}_{\{-100, 100\}}^{\text{Sign}}(0 < x?)}{\vdash_{\text{Sign}} [\{-100, 100\}] \ 0 < x? [\{100\}]} \quad (\text{transfer}) \\
\vdash_{\text{Sign}} [p] \ c [\{100\}] \quad (\text{seq})
\end{array}$$

Fig. 6. Derivation of $\vdash_{\text{Sign}} [p = \{-10, -1, 100\}] \ c [\{100\}]$ for Example 5.3.

Therefore, in some cases we just prove (1) and (2a) and leave the proof of (2b) implicit. We proceed by induction on the derivation tree of $\vdash_A [p] \ r [q]$, by distinguishing the various cases on the basis of the last rule that is applied.

(transfer) : If rule (transfer) is applied as last rule, then it must be $r \equiv e \in \text{Exp}$ such that $\mathbb{C}_p^A(e)$ and $q = \llbracket e \rrbracket p$. We have that (1) $\llbracket e \rrbracket p \leq \llbracket e \rrbracket p$ trivially holds, while $\llbracket e \rrbracket_A^\# \alpha(p) = \llbracket e \rrbracket^A \alpha(p) = \alpha(\llbracket e \rrbracket \gamma \alpha(p)) = \alpha(\llbracket e \rrbracket p)$ holds by $\mathbb{C}_p^A(e)$, so that we also have (2).

(relax) : If the last rule applied is (relax), then we assume by induction that $\vdash_A [p'] \ r [q']$ can be derived and let p, q such that $p' \leq p \leq A(p')$ and $q \leq q' \leq A(q)$.

(1) By inductive hypothesis (1) and monotonicity of $\llbracket r \rrbracket$ we have the following:

$$q \leq q' \leq \llbracket r \rrbracket p' \leq \llbracket r \rrbracket p.$$

(2) Let us observe that from $p \in [p', A(p')]$ and $q' \in [q, A(q)]$, we derive $\alpha(p') = \alpha(p)$ and $\alpha(q') = \alpha(q)$. Therefore:

$$\begin{aligned}
\alpha(q) &\leq_A \quad [\text{by monotonicity of } \alpha \text{ and (1)}] \\
\alpha(\llbracket r \rrbracket p) &\leq_A \quad [\text{by correctness (4)}] \\
\llbracket r \rrbracket_A^\# \alpha(p) &= \quad [\text{by hyp.}] \\
\llbracket r \rrbracket_A^\# \alpha(p') &= \quad [\text{by ind. hyp. (2a)}] \\
\alpha(q') &= \quad [\text{by hyp.}] \\
\alpha(q), &
\end{aligned}$$

so that, $\llbracket r \rrbracket_A^\# \alpha(p) = \alpha(q) = \alpha(\llbracket r \rrbracket p)$ follows.

(seq) : If the last rule applied is (seq), then it must be $r \equiv r_1; r_2$ and we can assume by induction that $\vdash_A [p] \ r_1 [w]$ and $\vdash_A [w] \ r_2 [q]$, can be derived for some w . We need to prove the thesis for the conclusion $\vdash_A [p] \ r_1; r_2 [q]$.

(1) By inductive hypotheses (1) and monotonicity of $\llbracket r_2 \rrbracket$:

$$q \leq \llbracket r_2 \rrbracket w \leq \llbracket r_2 \rrbracket (\llbracket r_1 \rrbracket p) = \llbracket r_1; r_2 \rrbracket p.$$

(2) We have that

$$\begin{aligned}
\alpha(q) &\leq_A \quad [\text{by (1) and monotonicity of } \alpha] \\
\alpha(\llbracket r_1; r_2 \rrbracket p) &\leq_A \quad [\text{by correctness (4)}] \\
\llbracket r_1; r_2 \rrbracket_A^\# \alpha(p) &= \quad [\text{by definition}] \\
\llbracket r_2 \rrbracket_A^\# (\llbracket r_1 \rrbracket_A^\# \alpha(p)) &= \quad [\text{by ind.hyp. (2a), } \llbracket r_1 \rrbracket_A^\# \alpha(p) = \alpha(w)] \\
\llbracket r_2 \rrbracket_A^\# \alpha(w) &= \quad [\text{by ind.hyp. (2a)}] \\
\alpha(q), &
\end{aligned}$$

so that $\llbracket r_1; r_2 \rrbracket_A^\# \alpha(p) = \alpha(q) = \alpha(\llbracket r_1; r_2 \rrbracket p)$ follows.

(join) : If the last rule applied is (join), then it must be $r \equiv r_1 \oplus r_2$ and we assume by induction that $\vdash_A [p] r_1 [q_1]$ and $\vdash_A [p] r_2 [q_2]$ can be derived for some suitable q_1 and q_2 such that $q = q_1 \vee q_2$. We need to prove the thesis for the conclusion $\vdash_A [p] r_1 \oplus r_2 [q_1 \vee q_2]$.

(1) By inductive hypothesis, $q_i \leq \llbracket r_i \rrbracket p$, therefore

$$q_1 \vee q_2 \leq \llbracket r_1 \rrbracket p \vee \llbracket r_2 \rrbracket p = \llbracket r_1 \oplus r_2 \rrbracket p.$$

(2a) We have that

$$\begin{aligned} \llbracket r_1 \oplus r_2 \rrbracket_A^\# \alpha(p) &= [\text{by definition}] \\ \llbracket r_1 \rrbracket_A^\# \alpha(p) \vee_A \llbracket r_2 \rrbracket_A^\# \alpha(p) &= [\text{by ind. hyp. (2a)}] \\ \alpha(q_1) \vee_A \alpha(q_2) &= [\text{by additivity of } \alpha] \\ \alpha(q_1 \vee q_2), \end{aligned}$$

so that $\llbracket r_1 \oplus r_2 \rrbracket_A^\# \alpha(p) = \alpha(q_1 \vee q_2) = \alpha(\llbracket r_1 \oplus r_2 \rrbracket p)$ follows by (1) and (2a).

(iterate) : If the last rule applied is (iterate), then it must be $r \equiv r_1^*$, and we assume by induction that $\vdash_A [p] r_1 [q_1]$ for some q_1 such that $q_1 \leq A(p)$ and $q = p \vee q_1$. Note that $\alpha(q_1) \leq_A \alpha(A(p)) = \alpha(p)$. We need to prove the thesis for the conclusion $\vdash_A [p] r_1^* [p \vee q_1]$.

(1) By inductive hypothesis (1), we have that

$$p \vee q_1 \leq p \vee \llbracket r_1 \rrbracket p = \llbracket r_1 \rrbracket^0 p \vee \llbracket r_1 \rrbracket p \leq \bigvee_{n \in \mathbb{N}} \llbracket r_1 \rrbracket^n p = \llbracket r_1^* \rrbracket p.$$

(2a) By inductive hypothesis (2) and hypothesis $q_1 \leq A(p)$, we have $\llbracket r_1 \rrbracket_A^\# \alpha(p) = \alpha(q_1) \leq_A \alpha(p)$, so that Lemma 2.1 (c) is applicable. Therefore,

$$\begin{aligned} \llbracket r_1^* \rrbracket_A^\# \alpha(p) &= [\text{by Lemma 2.1 (c)}] \\ \alpha(p) &= [\text{by hyp. } q_1 \leq A(p)] \\ \alpha(p) \vee_A \alpha(q_1) &= [\text{by additivity of } \alpha] \\ \alpha(p \vee q_1), \end{aligned}$$

so that, $\llbracket r_1^* \rrbracket_A^\# \alpha(p) = \alpha(p \vee q_1) = \alpha(\llbracket r_1^* \rrbracket p)$ follows by (1) and (2a).

(rec) : If the last rule applied is (rec), then it must be $r \equiv r_1^*$ and we assume by induction that $\vdash_A [p] r_1 [w]$ and $\vdash_A [p \vee w] r_1^* [q]$ can be derived for some w . We need to prove the thesis for the conclusion $\vdash_A [p] r_1^* [q]$.

(1) By inductive hypothesis (1) and Lemma 2.1 (e) we have

$$q \leq \llbracket r_1^* \rrbracket (p \vee w) = \llbracket r_1^* \rrbracket p.$$

(2) We have that

$$\begin{aligned} \alpha(q) &\leq_A [\text{by monotonicity of } \alpha \text{ and (1)}] \\ \alpha(\llbracket r_1^* \rrbracket p) &\leq_A [\text{by correctness (4)}] \\ \llbracket r_1^* \rrbracket_A^\# \alpha(p) &\leq_A [\text{by monotonicity}] \\ \llbracket r_1^* \rrbracket_A^\# \alpha(p \vee w) &= [\text{by ind.hyp. (2a)}] \\ \alpha(q), \end{aligned}$$

so that $\llbracket r_1^* \rrbracket_A^\# \alpha(p) = \alpha(q) = \alpha(\llbracket r_1^* \rrbracket p)$ follows. $\square$

As a consequence, if a correctness specification $spec$ is expressible in A , i.e., if $spec = \gamma(a)$ for some abstract value $a \in A$, then any provable triple $\vdash_A [p] r [q]$ allows us to use q to witness *either the correctness or the incorrectness* of r for the pre-condition p , as stated by the following result.

COROLLARY 5.6 (PRECISION). For all $A_{\alpha,\gamma} \in \text{Abs}(C)$, $r \in \text{Reg}$, $p, q \in C$, if $\vdash_A [p] \text{ r } [q]$, then:

$$\forall a \in A. \llbracket r \rrbracket p \leq \gamma(a) \text{ iff } q \leq \gamma(a).$$

Example 5.7. In Example 5.3, we already noticed that, by Theorem 5.5 (1), $\llbracket c \rrbracket p$ does not satisfy *spec*. Note that for $p' \triangleq \{-10, -1, 5, 100\}$ we could also prove, e.g., $\vdash_{\text{Sign}} [p'] \text{ c } [\{5\}]$, where the post-condition $\{5\}$ has no alarm, even if *spec* is not satisfied: Since *spec* is not expressible in Sign, then Corollary 5.6 is not applicable.

Example 5.8. Let us consider again Example 5.2 and the triple $\vdash_{\text{Int}} [p] \text{ r } [\{0, 2, 1000\}]$ (see Figure 5.2) to discuss the cases of three different correctness specifications: $\text{spec}_1 \triangleq x \neq 2$, $\text{spec}_2 \triangleq x \leq 1000$ and $\text{spec}_3 \triangleq 100 \leq x$.

Despite that spec_1 is not expressible in Int, the triple $\vdash_{\text{Int}} [p] \text{ r } [\{0, 2, 1000\}]$ exposes the true alarm $2 \notin \text{spec}_1$, since $2 \in \{0, 2, 1000\} \subseteq \llbracket r \rrbracket p$ (cf. Theorem 5.5 (1)).

By Theorem 5.5 (2), we know that $\llbracket r \rrbracket p$ satisfies spec_2 , because $\text{Int}(\llbracket r \rrbracket p) = \text{Int}(\{0, 2, 1000\}) = [0, 1000] \subseteq \text{spec}_2$. Since spec_2 is expressible in Int, by Corollary 5.6, any other provable triple $\vdash_{\text{Int}} [p] \text{ r } [q]$ will guarantee that spec_2 holds.

Finally, the triple $\vdash_{\text{Int}} [p] \text{ r } [\{0, 2, 1000\}]$ exposes two true alarms $0, 2 \notin \text{spec}_3$. Likewise spec_2 , by Corollary 5.6, the post-condition q of any provable triple $\vdash_{\text{Int}} [p] \text{ r } [q]$ will contain a true alarm for spec_3 . In particular, any such triple $\vdash_{\text{Int}} [p] \text{ r } [q]$, is such that $0 \in q$, therefore contradicting the assertion spec_3 , because it must be $\text{Int}(q) = [0, 1000] = \text{Int}(\llbracket r \rrbracket p)$.

Example 5.9. Let us consider the program $r \triangleq r_1^*; r_2$, where

$$\begin{aligned} r_1 &\triangleq (x < 1000?; x := x + 10), \\ r_2 &\triangleq (x = 30?; x := -1) \oplus (x \neq 30?). \end{aligned}$$

Consider the correctness specification $\text{spec} \triangleq x \neq -1$, and two pre-conditions $p_1 \triangleq \{10\}$ and $p_2 \triangleq \{11\}$. In the case of p_1 , the value 30 for x is actually computed by the Kleene iteration r_1^* , causing a violation of *spec*. In the case p_2 , any alarm raised by a static analysis would be a false positive since r_1^* will never store the value 30 in x . Let us use the abstract domain Sign for the analysis of r .

First, consider p_1 and observe that by applying (rec) and (iterate) we can derive $\vdash_{\text{Sign}} [\{10\}] \text{ r}_1^* [\{10, 20, 30\}]$. Then, by (relax), we prove $\vdash_{\text{Sign}} [\{10\}] \text{ r}_1^* [\{10, 30\}]$. Note that $\mathbb{C}_{\{10, 30\}}^{\text{Sign}}(x = 30?)$ and $\mathbb{C}_{\{10, 30\}}^{\text{Sign}}(x \neq 30?)$, so that $\vdash_{\text{Sign}} [\{10, 30\}] \text{ r}_2 [\{-1, 10\}]$ can be proved. Finally, by applying (seq), we derive the triple $\vdash_{\text{Sign}} [\{10\}] \text{ r}_1^*; r_2 [\{-1, 10\}]$, whose post-condition includes the true alarm -1 violating *spec*. The whole derivation is in Figure 7.

Consider now the pre-condition p_2 . Here, we apply (rec), (iterate) and (relax) so as to derive $\vdash_{\text{Sign}} [\{11\}] \text{ r}_1^* [\{11, 31\}]$. In this case, since local completeness $\mathbb{C}_{\{11, 31\}}^{\text{Sign}}(x = 30?)$ does not hold (indeed note that any nonempty subset of $\{11, 31\}$ is not locally complete for $x = 30?$ on Sign), the proof cannot be completed in Sign. It is worth observing that to prove that r satisfies *spec* for input p_2 we need to resort to a more precise abstract domain where the guard $x = 30?$ is locally complete for $\{11, 31\}$. In this sense, we remark that the failed proof obligation $\mathbb{C}_{\{11, 31\}}^{\text{Sign}}(x = 30?)$ could be exploited to find a refinement of the current abstraction Sign where the derivation can be completed. For example, a possible choice is to extend the abstract domain Sign by taking Sign_{30} as the Moore closure of $\text{Sign} \cup \{x \neq 30\}$: Then, the triple $\vdash_{\text{Sign}_{30}} [p_2] \text{ r } [\{11\}]$ could be derived to witness that r satisfies *spec* (by Corollary 5.6).

The next example shows that the classical consequence rule of Hoare logic for strengthening pre-conditions and weakening post-conditions is in general not compatible with the local completeness property (2) of Theorem 5.5.

$$\begin{array}{c}
\frac{\mathbb{C}_{\{10,20\}}^{\text{Sign}}(b_1?)}{\vdash_{\text{Sign}} [10,20] b_1? [\{10,20\}]} \quad (\text{transfer}) \quad \frac{\mathbb{C}_{\{10,20\}}^{\text{Sign}}(e_1)}{\vdash_{\text{Sign}} [\{10,20\}] e_1 [\{20,30\}]} \quad (\text{transfer}) \\
\vdash_{\text{Sign}} [10,20] r_1 [\{20,30\}] \quad (\text{seq}) \quad \frac{\{20,30\} \subseteq \text{Sign}(\{10,20\}) = \mathbb{Z}_{>0}}{\vdash_{\text{Sign}} [\{10,20\}] r_1^* [\{10,20,30\}]} \quad (\text{iterate}) \\
\vdash_{\text{Sign}} [\{10,20\}] r_1^* [\{10,20,30\}] \quad (*) \quad (\text{rec}) \\
\vdash_{\text{Sign}} [p_1] b_1? [p_1] \quad (\text{tr.}) \quad \frac{\mathbb{C}_{p_1}^{\text{Sign}}(e_1)}{\vdash_{\text{Sign}} [p_1] e_1 [\{20\}]} \quad (\text{tr.}) \\
\vdash_{\text{Sign}} [p_1] r_1 [\{20\}] \quad (\text{seq}) \quad (*) \quad (\text{rec}) \quad \frac{\mathbb{C}_{\{10,30\}}^{\text{Sign}}(b_2?)}{\vdash_{\text{Sign}} [\{10,30\}] b_2? [\{30\}]} \quad (\text{tr.}) \quad \frac{\mathbb{C}_{\{30\}}^{\text{Sign}}(e_2)}{\vdash_{\text{Sign}} [\{30\}] e_2 [\{-1\}]} \quad (\text{tr.}) \\
\vdash_{\text{Sign}} [p_1] r_1^* [\{10,20,30\}] \quad \{10,30\} \subseteq \{10,20,30\} \subseteq \mathbb{Z}_{>0} \quad (\text{relax}) \quad \vdash_{\text{Sign}} [10,30] r_3 [\{-1\}] \quad (\text{seq}) \quad \frac{\mathbb{C}_{\{10,30\}}^{\text{Sign}}(\neg b_2)}{\vdash_{\text{Sign}} [\{10,30\}] \neg b_2? [\{10\}]} \quad (\text{tr.}) \\
\vdash_{\text{Sign}} [p_1] r_1^* [\{10,30\}] \quad (\text{join}) \\
\vdash_{\text{Sign}} [p_1] r [\{-1,10\}] \quad (\text{seq})
\end{array}$$

Legenda: $r \triangleq r_1^*; r_2$ $r_1 \triangleq b_1?; e_1$ $r_2 \triangleq (b_2?; e_2) \oplus (\neg b_2?)$ $r_3 \triangleq b_2?; e_2$
 $b_1 \triangleq x < 1000$ $e_1 \triangleq x := x + 10$ $b_2 \triangleq x = 30$ $e_2 \triangleq x := -1$

Fig. 7. Derivation of $\vdash_{\text{Sign}} [p_1 = \{10\}] r [\{-1, 10\}]$ for Example 5.9.

Example 5.10. Consider the abstract domain Int , the absolute value program from the Introduction

$$\text{AbsVal}(x) \triangleq \text{if } (x < 0) \text{ then } x := -x \text{ else skip}$$

and the pre-conditions $p \triangleq \{-5, 5\}$ and $p' \triangleq \{-5, -1, 0, 5\}$. By applying (transfer) (to $x < 0?$, $x \geq 0?$, $x := -x$ and **skip**), (seq) twice and (join), one can easily derive $\vdash_{\text{Int}} [p'] \text{AbsVal}(x) [q']$ with $q' = \{0, 1, 5\}$. In fact, Int is locally complete for $\llbracket \text{AbsVal}(x) \rrbracket$ on p' , because $\text{Int}(\llbracket \text{AbsVal}(x) \rrbracket \text{Int}(p')) = [0, 5] = \text{Int}(q')$.

Imagine now to replace the rule (relax) by its dual (convex) version, inspired by the consequence rule of Hoare logic,

$$\frac{p \leq p' \leq A(p) \quad \vdash_A [p'] r [q'] \quad q' \leq q \leq A(q')}{\vdash_A [p] r [q]}$$

If we let $q = \text{Int}(q')$, since $p \subseteq p' \subseteq \text{Int}(p) = [-5, 5]$, and $q' \subseteq q = \text{Int}(q')$, then we could derive, e.g., $\vdash_{\text{Int}} [p] \text{AbsVal}(x) [q]$. However, Int is not locally complete for $\llbracket \text{AbsVal}(x) \rrbracket$ on p , because $\text{Int}(\llbracket \text{AbsVal}(x) \rrbracket p) = [5, 5] \neq [0, 5] = \text{Int}(\llbracket \text{AbsVal}(x) \rrbracket \text{Int}(p))$.

5.2 On the Logical Completeness of LCL_A

We now study the completeness of LCL_A as a proof system. In this context, completeness is intended in the logical sense, that is the ability of the proof system to derive any valid triple. To avoid the clash of terminology with completeness in abstract interpretation, we call it *logical* (in)completeness. Our logic LCL_A is not logically complete in general, i.e., the converse of Theorem 5.5 does not hold, as shown by the following example.

Example 5.11 (Logical Incompleteness). Let $e_1 \triangleq x := x - 1$, $e_2 \triangleq x := x + 1$, and $p \triangleq \mathbb{Z}_{\geq 2}$. The abstract domain $A = \text{Sign}$ is locally complete for $\llbracket e_1; e_2 \rrbracket$ on p :

$$\begin{aligned}
\alpha(\llbracket e_1; e_2 \rrbracket p) &= \alpha(\llbracket e_2 \rrbracket (\llbracket e_1 \rrbracket p)) = \alpha(p) = \mathbb{Z}_{>0} \\
\llbracket e_1; e_2 \rrbracket_A^\# \alpha(p) &= \llbracket e_2 \rrbracket_A^\# (\llbracket e_1 \rrbracket_A^\# \mathbb{Z}_{>0}) = \llbracket e_2 \rrbracket_A^\# \mathbb{Z}_{\geq 0} = \mathbb{Z}_{>0},
\end{aligned}$$

but the triple $\vdash_{\text{Sign}} [p] e_1; e_2 [p]$ cannot be derived. This is because, in the attempt to derive the triple $\vdash_{\text{Sign}} [p] e_1 [\llbracket e_1 \rrbracket p]$ by rule (transfer), the proof obligation $\mathbb{C}_p^{\text{Sign}}(e_1)$ fails: $\alpha(\llbracket e_1 \rrbracket p) = \alpha(\mathbb{Z}_{\geq 1}) = \mathbb{Z}_{>0} \neq \mathbb{Z}_{\geq 0} = \alpha(\mathbb{Z}_{\geq 0}) = \alpha(\llbracket e_1 \rrbracket \text{Sign}(p))$. Note that the rule (relax) cannot help. In fact, let us assume that there exists some p' such that $p' \subseteq p \subseteq \text{Sign}(p')$ and the triple $\vdash_{\text{Sign}} [p'] e_1 [\llbracket e_1 \rrbracket p']$ is provable by (transfer). Then, $\text{Sign}(p') = \text{Sign}(p) = \mathbb{Z}_{>0}$ and $p' \subseteq p = \mathbb{Z}_{\geq 2}$ imply that $\llbracket e_1 \rrbracket p' \subseteq \mathbb{Z}_{\geq 1}$.

Hence, we would have that $\alpha(\llbracket e_1 \rrbracket p') \leq \alpha(\mathbb{Z}_{\geq 1}) = \mathbb{Z}_{>0}$ while $\alpha(\llbracket e_1 \rrbracket \text{Sign}(p')) = \alpha(\llbracket e_1 \rrbracket \mathbb{Z}_{>0}) = \mathbb{Z}_{\geq 0}$. Thus, $\mathbb{C}_{p'}^{\text{Sign}}(e_1)$ does not hold, contradicting the hypothesis that $\vdash_{\text{Sign}} [p'] e_1 \llbracket e_1 \rrbracket p'$ is provable.

For a result of logical completeness for LCL_A , we need the following:

(1) to add the infinitary rule (limit) for Kleene star, whose soundness is shown by Lemma 5.12:

$$\frac{\forall n \in \mathbb{N}. \vdash_A [p_n] r [p_{n+1}]}{\vdash_A [p_0] r^* [\bigvee_{i \in \mathbb{N}} p_i]} \text{ (limit)}$$

(2) to assume that all the basic expressions occurring in a command $r \in \text{Reg}$ are globally complete on A .

LEMMA 5.12. *The proof system extended with rule (limit) is sound.*

PROOF. We extend the inductive proof of Theorem 5.5 by considering the case where the last rule applied to derive $\vdash_A [p] r [q]$ is (limit). As in the proof of Theorem 5.5, we refer to the equality $\llbracket r \rrbracket_A^\# \alpha(p) = \alpha(q)$ as (2a) and use (2b) for the equality $\llbracket r \rrbracket_A^\# \alpha(p) = \alpha(\llbracket r \rrbracket p)$. Then we recall that (2b) follows immediately by (1) and (2a).

(limit) : If the last rule applied is (limit), then it must be $r \equiv r_1^*$, and we can assume by induction that $\forall n \in \mathbb{N}. \vdash_A [p_n] r_1 [p_{n+1}]$. We need to prove the thesis for the conclusion $\vdash_A [p_0] r_1^* [\bigvee_{i \in \mathbb{N}} p_i]$.

(1) We have that for every $n \in \mathbb{N}$, $p_n \leq \llbracket r_1 \rrbracket^n p_0$: obvious for $n = 0$; then, by inductive hypothesis, $p_{n+1} \leq \llbracket r_1 \rrbracket p_n \leq \llbracket r_1 \rrbracket (\llbracket r_1 \rrbracket^n p_0) = \llbracket r_1 \rrbracket^{n+1} p_0$. Thus, $\bigvee_{n \in \mathbb{N}} p_n \leq \bigvee_{n \in \mathbb{N}} \llbracket r_1 \rrbracket^n p_0 = \llbracket r_1^* \rrbracket p_0$.

(2a) We have that for every $n \in \mathbb{N}$, $\llbracket r_1 \rrbracket_A^\# \alpha(p_0) = \alpha(p_n)$. In fact, by induction on n , this is obvious for the base case $n = 0$. Then, for the inductive case:

$$\begin{aligned} \llbracket r_1 \rrbracket_A^\# \alpha(p_0) &= \text{[by definition]} \\ \llbracket r_1 \rrbracket_A^\# (\llbracket r_1 \rrbracket_A^\# \alpha(p_0)) &= \text{[by ind. hyp. on } n\text{]} \\ \llbracket r_1 \rrbracket_A^\# \alpha(p_n) &= \text{[by ind. hyp. (2a)]} \\ &= \alpha(p_{n+1}). \end{aligned}$$

Consequently,

$$\begin{aligned} \llbracket r_1^* \rrbracket_A^\# \alpha(p_0) &= \text{[by definition]} \\ \bigvee_A \{ \llbracket r_1 \rrbracket_A^\# \alpha(p_0) \mid n \in \mathbb{N} \} &= \text{[by property above]} \\ \bigvee_A \{ \alpha(p_n) \mid n \in \mathbb{N} \} &= \text{[by additivity of } \alpha\text{]} \\ &= \alpha(\bigvee_{n \in \mathbb{N}} p_n). \end{aligned}$$

□

We are therefore able to obtain a result of logical completeness for LCL_A when this includes the powerful infinitary rule (limit) and under an assumption of global completeness for the basic expressions occurring in the program. Let us remark that incorrectness logic [O'Hearn 2020] also includes this infinitary rule (limit), there called *backwards variant* rule. Likewise (limit), the backwards variant rule of IL allows us to derive other finitary rules (e.g., *Iterate zero*) and plays a crucial role for proving the logical completeness of IL [O'Hearn 2020, Theorem 6].

LEMMA 5.13. *Let $A_{\alpha, \gamma} \in \text{Abs}(C)$ and $r \in \text{Reg}$. Then:*

$$(\forall e \in \text{Exp}(r). \mathbb{C}^A(e)) \Rightarrow \llbracket r \rrbracket_A^\# \alpha = \alpha \llbracket r \rrbracket.$$

PROOF. We proceed by structural induction on $r \in \text{Reg}$.

($r \equiv e$):

$$\begin{aligned}\alpha \circ \llbracket e \rrbracket &= \text{[by } \mathbb{C}^A(e)\text{]} \\ \alpha \circ \llbracket e \rrbracket \circ \gamma \circ \alpha &= \text{[by definition]} \\ \llbracket e \rrbracket_A^\# \circ \alpha &\end{aligned}$$

($r \equiv r_1; r_2$):

$$\begin{aligned}\alpha \circ \llbracket r_1; r_2 \rrbracket &= \text{[by definition]} \\ \alpha \circ \llbracket r_2 \rrbracket \circ \llbracket r_1 \rrbracket &= \text{[by ind.hyp.]} \\ \llbracket r_2 \rrbracket_A^\# \circ \alpha \circ \llbracket r_1 \rrbracket &= \text{[by ind.hyp.]} \\ \llbracket r_2 \rrbracket_A^\# \circ \llbracket r_1 \rrbracket_A^\# \circ \alpha &= \text{[by definition]} \\ \llbracket r_1; r_2 \rrbracket_A^\# \circ \alpha &\end{aligned}$$

($r \equiv r_1 \oplus r_2$): Let us take an arbitrary $p \in C$.

$$\begin{aligned}\alpha(\llbracket r_1 \oplus r_2 \rrbracket p) &= \text{[by definition]} \\ \alpha(\llbracket r_1 \rrbracket p \vee \llbracket r_2 \rrbracket p) &= \text{[by additivity of } \alpha\text{]} \\ \alpha(\llbracket r_1 \rrbracket p) \vee_A \alpha(\llbracket r_2 \rrbracket p) &= \text{[by ind.hyp.]} \\ \llbracket r_1 \rrbracket_A^\# \alpha(p) \vee_A \llbracket r_2 \rrbracket_A^\# \alpha(p) &= \text{[by definition]} \\ \llbracket r_1 \oplus r_2 \rrbracket_A^\# \alpha(p) &\end{aligned}$$

($r \equiv r_1^*$): Let us first prove the following property:

$$\forall n \in \mathbb{N}. \alpha \llbracket r_1 \rrbracket^n = \llbracket r_1 \rrbracket_A^\#{}^n \alpha. \quad (*)$$

For $n = 0$: $\alpha \circ \llbracket r_1 \rrbracket^0 = \alpha = \llbracket r_1 \rrbracket_A^\#{}^0 \circ \alpha$.

For $n + 1$,

$$\begin{aligned}\alpha \circ \llbracket r_1 \rrbracket^{n+1} &= \text{[by definition]} \\ \alpha \circ \llbracket r_1 \rrbracket \circ \llbracket r_1 \rrbracket^n &= \text{[by ind.hyp.]} \\ \llbracket r_1 \rrbracket_A^\# \circ \alpha \circ \llbracket r_1 \rrbracket^n &= \text{[by ind.hyp.]} \\ \llbracket r_1 \rrbracket_A^\# \circ \llbracket r_1 \rrbracket_A^\#{}^n \circ \alpha &= \text{[by definition]} \\ \llbracket r_1 \rrbracket_A^\#{}^{n+1} \circ \alpha &\end{aligned}$$

Thus, for any $p \in C$,

$$\begin{aligned}\alpha(\llbracket r_1^* \rrbracket p) &= \text{[by definition]} \\ \alpha(\bigvee \{ \llbracket r_1 \rrbracket^n p \mid n \in \mathbb{N} \}) &= \text{[by additivity of } \alpha\text{]} \\ \bigvee_A \{ \alpha(\llbracket r_1 \rrbracket^n p) \mid n \in \mathbb{N} \} &= \text{[by } (*)\text{]} \\ \bigvee_A \{ \llbracket r_1 \rrbracket_A^\#{}^n \alpha(p) \mid n \in \mathbb{N} \} &= \text{[by definition]} \\ \llbracket r_1^* \rrbracket_A^\# \alpha(p) &\end{aligned}$$

This closes the proof. □

THEOREM 5.14 (LOGICAL COMPLETENESS). *Let $A_{\alpha, \gamma} \in \text{Abs}(C)$ and $r \in \text{Reg}$ such that any $e \in \text{Exp}(r)$ is globally complete on A . For any $p, q \in C$, if $q \leq \llbracket r \rrbracket p$ and $\llbracket r \rrbracket_A^\# \alpha(p) = \alpha(q)$, then the triple $\vdash_A [p] \ r \ [q]$ can be derived in the proof system extended with the (limit) rule.*

PROOF. We proceed by structural induction on r .

($r \equiv e$): By hypothesis, $\llbracket e \rrbracket_A^\# \alpha(p) = \alpha(\llbracket e \rrbracket p)$, thus, by Lemma 2.1 (b), $\alpha(\llbracket e \rrbracket p) = \alpha(\llbracket e \rrbracket \gamma \alpha(p))$. Hence, $\mathbb{C}_p^A(\llbracket e \rrbracket)$ holds, so that by (transfer), $\vdash_A [p] e \llbracket e \rrbracket p$.

($r \equiv r_1; r_2$): Assume that $q \leq \llbracket r_1; r_2 \rrbracket p$ and $\llbracket r_1; r_2 \rrbracket_A^\# \alpha(p) = \alpha(q) = \alpha(\llbracket r_1; r_2 \rrbracket p)$. Let $w \triangleq \llbracket r_1 \rrbracket p$. Then, we have that:

- (1) $w \leq \llbracket r_1 \rrbracket p$ and, by Lemma 5.13, $\llbracket r_1 \rrbracket_A^\# \alpha(p) = \alpha(\llbracket r_1 \rrbracket p) = \alpha(w)$;
- (2) $q \leq \llbracket r_2 \rrbracket w$ and, by Lemma 5.13, $\llbracket r_2 \rrbracket_A^\# \alpha(w) = \alpha(\llbracket r_2 \rrbracket w) = \alpha(\llbracket r_2 \rrbracket \llbracket r_1 \rrbracket p) = \alpha(\llbracket r_1; r_2 \rrbracket p) = \alpha(q)$.

Thus, by induction, we derive $\vdash_A [p] r_1 [w]$ and $\vdash_A [w] r_2 [q]$, so that by (seq), $\vdash_A [p] r_1; r_2 [q]$ follows.

($r \equiv r_1 \oplus r_2$): Assume that $q \leq \llbracket r_1 \oplus r_2 \rrbracket p$ and $\llbracket r_1 \oplus r_2 \rrbracket_A^\# \alpha(p) = \alpha(q) = \alpha(\llbracket r_1 \oplus r_2 \rrbracket p)$. Let $q_i \triangleq \llbracket r_i \rrbracket p$. Then, we have that $q_i \leq \llbracket r_i \rrbracket p$ and, by Lemma 5.13, $\llbracket r_i \rrbracket_A^\# \alpha(p) = \alpha(\llbracket r_i \rrbracket p) = \alpha(q_i)$. Thus, by induction, $\vdash_A [p] r_i [q_i]$ so that by (join), $\vdash_A [p] r_1 \oplus r_2 [q_1 \vee q_2]$. Then, because $q \leq \llbracket r_1 \oplus r_2 \rrbracket p = \llbracket r_1 \rrbracket p \vee \llbracket r_2 \rrbracket p = (q_1 \vee q_2) \leq \gamma \alpha(\llbracket r_1 \oplus r_2 \rrbracket p) = \gamma \alpha(q)$, by (relax), $\vdash_A [p] r_1 \oplus r_2 [q]$ follows.

($r \equiv r_1^*$): Assume that $q \leq \llbracket r_1^* \rrbracket p$ and $\llbracket r_1^* \rrbracket_A^\# \alpha(p) = \alpha(q) = \alpha(\llbracket r_1^* \rrbracket p)$. Let us consider the $\mathbb{N}$ -indexed sequence $\{p_n\}_{n \in \mathbb{N}}$ of values in C , where $p_n \triangleq \llbracket r_1 \rrbracket^n p$. We have that for all $n \in \mathbb{N}$, $p_{n+1} \leq \llbracket r_1 \rrbracket p_n$ and, by Lemma 5.13, $\llbracket r_1 \rrbracket_A^\# \alpha(p_n) = \alpha(\llbracket r_1 \rrbracket p_n) = \alpha(p_{n+1})$. Thus, by induction, for all $n \in \mathbb{N}$, $\vdash_A [p_n] r_1 [p_{n+1}]$ is derivable. Hence, by (limit), we derive $\vdash_A [p] r_1^* [\bigvee_{n \in \mathbb{N}} p_n]$. Notice that $\bigvee_{n \in \mathbb{N}} p_n = \bigvee \{\llbracket r_1 \rrbracket^n p \mid n \in \mathbb{N}\} = \llbracket r_1^* \rrbracket p$. Therefore, $q \leq \llbracket r_1^* \rrbracket p \leq \gamma \alpha(\llbracket r_1^* \rrbracket p) = \gamma \alpha(q)$, and by (relax) it follows $\vdash_A [p] r_1^* [q]$. $\square$

It is worth noting that if any $e \in \text{Exp}(r)$ is globally complete on A , then, as proved by [Giacobazzi et al. 2015, Theorem 5.1], $\llbracket r \rrbracket_A^\# \alpha = \alpha \llbracket r \rrbracket$ also holds. Thus, the hypotheses of Theorem 5.14 imply that properties (1–2) of the soundness Theorem 5.5 all hold, i.e., Theorem 5.14 provides a result of (limited) logical completeness for LCL_A . Vice versa, whenever the language is Turing complete, $C = \mathbb{S}$ and the abstraction A is not trivial, LCL_A turns out to be *intrinsically incomplete*, meaning that it is always possible to find a valid triple (w.r.t. properties (1–2) of Theorem 5.5) but LCL_A is unable to prove it.

THEOREM 5.15 (INTRINSIC INCOMPLETENESS). *Let Reg be a Turing complete language. Let $A_{\alpha, \gamma} \in \text{Abs}(\mathbb{S})$. If $\gamma \alpha \neq \text{id}$ and $\gamma \alpha \neq \lambda x. \Sigma$, then there exist $p, q \in \mathbb{S}$ and $r \in \text{Reg}$ such that $q \leq \llbracket r \rrbracket p$ and $\llbracket r \rrbracket_A^\# \alpha(p) = \alpha(\llbracket r \rrbracket p) = \alpha(q)$, but $\not\vdash A[p] r [q]$.*

PROOF. The assumption of Turing completeness of Reg implies the possibility of defining regular commands $r_{=c?}, r_{\neq c?} \in \text{Reg}$, for any store $c \in \Sigma$, such that, for any $p \in \mathbb{S}$, $\llbracket r_{=c?} \rrbracket p = p \cap \{c\}$ and $\llbracket r_{\neq c?} \rrbracket p = p \cap \neg\{c\}$. Likewise, Turing completeness implies the possibility of defining regular commands $r_c \in \text{Reg}$, for any $c \in \Sigma$, such that, for any $\sigma \in \Sigma$ we have $\llbracket r_c \rrbracket \{\sigma\} = \{c\}$. Finally, Turing completeness implies that there exists $w \in \text{Reg}$ such that for any $p \in \mathbb{S}$, $\llbracket w \rrbracket p = \emptyset$ holds. For the sake of clarity, by considering Imp programs over just one variable x so that $c \in \mathbb{Z}$, these programs would be as follows:

$$\begin{aligned} w &\triangleq \text{while } \texttt{tt} \text{ do skip} \\ r_{=c?} &\triangleq \text{if } (x = c) \text{ then skip else } w \\ r_{\neq c?} &\triangleq \text{if } \neg(x = c) \text{ then skip else } w \\ r_c &\triangleq x := c. \end{aligned}$$

Consider now $A_{\alpha, \gamma} \in \text{Abs}(\mathbb{S})$ such that $\gamma \alpha \neq \text{id}$ and $\gamma \alpha \neq \lambda x. \Sigma$. Then there exists $p \in \wp(\Sigma)$ such that $p \subsetneq A(p)$ (because $\gamma \alpha \neq \text{id}$) and $w \in \wp(\Sigma)$ such that $A(w) \subsetneq \Sigma$ (because $\gamma \alpha \neq \lambda x. \Sigma$). Let r_b ,

$r_{=a?}$, and $r_{\neq a?}$ be regular commands defined as above for some $a \in A(p) \setminus p$ and $b \in \Sigma \setminus A(w)$. Now, take the program $r \triangleq r_1; w$, where

$$r_1 \triangleq (r_{=a?}; r_b) \oplus (r_{\neq a?}; w).$$

We have that $\llbracket r \rrbracket p = \llbracket w \rrbracket (\llbracket r_1 \rrbracket p) = \emptyset$ and $\llbracket r \rrbracket A(p) = \llbracket w \rrbracket (\llbracket r_1 \rrbracket A(p)) = \emptyset$, so that local completeness $A(\llbracket r \rrbracket p) = A(\llbracket r \rrbracket A(p))$ holds. However, the triple $\vdash_A [p] r [\emptyset]$ cannot be derived, because we cannot derive any triple $\vdash_A [p] r_1 [q]$ (to be used by the rule (seq) for $r_1; w$). This is a consequence of the fact that $\llbracket r_1 \rrbracket$ is not locally complete on p in A . In fact, $\llbracket r_1 \rrbracket p = \emptyset$ while $\llbracket r_1 \rrbracket A(p) = \llbracket (r_{=a?}; r_b) \oplus (r_{\neq a?}; w) \rrbracket A(p) = \llbracket r_{=a?}; r_b \rrbracket A(p) \cup \llbracket r_{\neq a?}; w \rrbracket A(p) = \{b\} \cup \emptyset = \{b\}$. Hence, $\alpha(\llbracket r_1 \rrbracket p) = \alpha(\emptyset)$ while $\llbracket r_1 \rrbracket_A^\# \alpha(p) \supseteq \alpha(\llbracket r_1 \rrbracket A(p)) = \alpha(\{b\})$ (by soundness of $\llbracket r_1 \rrbracket_A^\#$ and monotonicity of α). We observe that $\alpha(\emptyset) \neq \alpha(\{b\})$; otherwise, by monotonicity of γ and extensivity of $\gamma\alpha$, we would have $b \in \gamma\alpha(\{b\}) = \gamma\alpha(\emptyset) \subseteq \gamma\alpha(w) = A(w)$, which would be a contradiction. Now suppose that $\vdash_A [p] r_1 [q]$ for some q . By Theorem 5.5 (1), it must be $q \subseteq \llbracket r_1 \rrbracket p = \emptyset$, thus $q = \emptyset$ and by Theorem 5.5 (2), we would obtain $\alpha(\emptyset) = \alpha(q) = \llbracket r_1 \rrbracket_A^\# \alpha(p) \supseteq \alpha(\{b\})$, which is a contradiction. The rule (relax) cannot help here neither, because any strengthening of p , say p' , such that $p' \subseteq p \subseteq A(p')$ makes $A(p) = A(p')$ therefore making $\llbracket r_1 \rrbracket$ incomplete also for p' in A . $\square$

Turing completeness is crucial here because the proof relies upon the possibility of (1) specifying in Reg an arbitrary store in Σ , (2) effectively checking store inequality, and (3) specifying in Reg an undefined transfer function. The proof of Theorem 5.15 generalizes to LCL_A and to Turing complete regular commands the proofs in Bruni et al. [2020] and Giacobazzi et al. [2015]: The class of all programs for which an abstract interpretation on A is globally complete is the set of all programs if and only if A is trivial. As a consequence of Theorems 5.14 and 5.15, LCL_A cannot be a logically complete proof system for a Turing complete language unless the abstraction A is trivial. Of course, the identical abstraction is unfeasible here as both rules (transfer) and (relax) would become vacuous. Hence, the only meaningful straightforward abstraction for LCL_A is $A = \lambda x. \Sigma$. In light of this, we may therefore observe that logical completeness of IL [O'Hearn 2020, Theorem 6] follows from the choice made in its consequence rule of letting pre-conditions and post-conditions be, respectively, arbitrarily weakened and strengthened, i.e., the choice of the abstraction $A = \lambda x. \Sigma$ in rule (relax). Section 6 provides additional details on the relationship with incorrectness logic.

5.3 A Backward Proof System

As pioneered by Cousot and Cousot [1977, Section 5.2], it is known that abstract interpretation-based program analysis can be defined either backward or forward (see Bourdoncle [1993]; Miné [2017] for examples of backward abstract interpretations). Instead of propagating forward an abstract store in a program control flow graph (CFG) from its entry point, a backward analysis can start from any program point q of the CFG (possibly, but not necessarily, an exit point) and from any input abstract value a it abstractly computes backward in the CFG to derive *necessary* abstract conditions for the executions reaching q and satisfying the store property a . Backward analysis is typically used after a preliminary forward analysis to refine its results, as acknowledged by Bourdoncle [1993] for abstract program debugging. An under-approximating backward program analysis by abstract interpretation has been put forward by Miné [2014]. In the following, we show how LCL_A can be easily dualized to accomodate backward analyses.

In backward analysis, the basic expressions $e \in \text{Exp}$ have a co-additive backward concrete semantics $\langle e \rangle_- : C \rightarrow C$. Notably, $\langle e \rangle_- Y \triangleq \{\sigma \mid (\sigma, \sigma') \in R_e \Rightarrow \sigma' \in Y\}$, where $R_e \triangleq \{(\sigma, \langle e \rangle \sigma) \mid \sigma \in \Sigma\}$ is the transition relation for e . For example, for the basic commands of Imp (Section 2.2.3)

we have that

$$\begin{aligned} \llbracket \text{skip} \rrbracket_{\leftarrow} Y &\triangleq Y \\ \llbracket x := a \rrbracket_{\leftarrow} Y &\triangleq \{\sigma \in \Sigma \mid \sigma[x \mapsto \llbracket a \rrbracket \sigma] \in Y\} \\ \llbracket b? \rrbracket_{\leftarrow} Y &\triangleq \{\sigma \in \Sigma \mid \llbracket b \rrbracket \sigma = \mathbf{tt} \Rightarrow \sigma \in Y\} = Y \cup \neg b. \end{aligned}$$

Furthermore, in backward concrete and abstract semantics: (1) the control flows backwards and (2) meets replace joins. Hence, the backward semantics of regular commands is given as the following dual definition of (2):

$$\begin{aligned} \llbracket e \rrbracket_{\leftarrow} c &\triangleq \langle e \rangle_{\leftarrow} c \\ \llbracket r_1; r_2 \rrbracket_{\leftarrow} c &\triangleq \llbracket r_1 \rrbracket_{\leftarrow} (\llbracket r_2 \rrbracket_{\leftarrow} c) \\ \llbracket r_1 \oplus r_2 \rrbracket_{\leftarrow} c &\triangleq \llbracket r_1 \rrbracket_{\leftarrow} c \wedge \llbracket r_2 \rrbracket_{\leftarrow} c \\ \llbracket r^* \rrbracket_{\leftarrow} c &\triangleq \bigwedge \{ \llbracket r \rrbracket_{\leftarrow}^n c \mid n \in \mathbb{N} \}. \end{aligned}$$

This defines a standard backward semantics, e.g., as given in Miné [2014, Section 2] for imperative programs.

By duality, the backward abstract semantics uses an *under-approximating* abstraction $U_{\alpha, \gamma} \in \text{Abs}^{\leftarrow}(C)$, meaning that U is defined by a Galois insertion w.r.t. the concrete domain $\langle C, \geq \rangle$ where the partial order is the inverse relation $\geq \triangleq \leq^{-1}$. This means that for all $c \in C$, an under-approximation relation $\gamma\alpha(c) \leq c$ replaces an over-approximating relation $c \leq \gamma\alpha(c)$.

Example 5.16. The interval domain Int can be viewed as an under-approximating abstraction by dualizing its abstraction and concretization maps $\alpha^{\leftarrow} : \wp(\mathbb{Z}) \rightarrow \text{Int}$ and $\gamma^{\leftarrow} : \text{Int} \rightarrow \wp(\mathbb{Z})$ as follows:

$$\begin{aligned} \gamma^{\leftarrow}([l, u]) &\triangleq \neg\gamma([l, u]) = \{z \in \mathbb{Z} \mid z < l \vee u > z\} \\ \alpha^{\leftarrow}(X) &\triangleq \alpha(\neg X) = [\min(\neg X), \max(\neg X)]. \end{aligned}$$

For example, $\alpha^{\leftarrow}(\{x \in \mathbb{Z} \mid x < -3 \vee x > 5\} \cup \{0, 1, 2\}) = [-3, 5]$ and $\alpha^{\leftarrow}(\{x \in \mathbb{Z} \mid x < -3\} \cup \{0, 1, 2\}) = [-3, +\infty]$.

Accordingly, the abstract semantics $\llbracket r \rrbracket_U^{\#}$ for an under-approximating abstraction $U \in \text{Abs}^{\leftarrow}(C)$ is defined from (3) by duality as follows:

$$\begin{aligned} \llbracket e \rrbracket_U^{\#} a &\triangleq \alpha(\langle e \rangle_{\leftarrow} \gamma(a)) \\ \llbracket r_1; r_2 \rrbracket_U^{\#} a &\triangleq \llbracket r_1 \rrbracket_U^{\#} (\llbracket r_2 \rrbracket_A^{\#} a) \\ \llbracket r_1 \oplus r_2 \rrbracket_U^{\#} a &\triangleq \llbracket r_1 \rrbracket_U^{\#} a \wedge_U \llbracket r_2 \rrbracket_U^{\#} a \\ \llbracket r^* \rrbracket_U^{\#} a &\triangleq \bigwedge_U \{ (\llbracket r \rrbracket_U^{\#})^n a \mid n \in \mathbb{N} \} \end{aligned}$$

Correspondingly, when our proof system $\vdash_U$ is instantiated to an under-approximating abstraction $U \in \text{Abs}^{\leftarrow}(C)$, we need to replace $\leq$, which models logical implication, with $\geq$ and logical disjunction $\vee$ with conjunction $\wedge$. Hence, the fundamental (relax) rule becomes

$$\frac{U(p') \leq p \leq p' \quad \vdash_U [p'] \text{ r } [q'] \quad U(q) \leq q' \leq q}{\vdash_U [p] \text{ r } [q]}.$$

Thus, here the condition p is logically stronger than p' and weaker than the under-approximation $U(p')$, and dually for q . By duality, as a direct consequence of Theorems 5.5 and 5.14, a result of soundness and limited logical completeness for the dual logic $\vdash_U$ can be derived. Hence, a provable triple $\vdash_U [p] \text{ r } [q]$ for an under-approximation U implies that: $\llbracket r \rrbracket_{\leftarrow} q \leq p$, i.e., p is an over-approximation of the backward semantics, and $\llbracket r \rrbracket_U^{\#} \alpha(q) = \alpha(p) = \alpha(\llbracket r \rrbracket_{\leftarrow} q)$, i.e., local completeness holds.

6 RELATIONSHIP WITH INCORRECTNESS LOGIC

The idea to reverse the direction of implication in Hoare's consequence rule was investigated by de Vries and Koutavas [2011] reverse Hoare logic to study reachability specifications for randomized algorithms. They first put forward the following consequence rule for under-approximation triples:

$$\frac{p' \leq p \quad [p'] \text{ r } [q'] \quad q \leq q'}{[p] \text{ r } [q]} \text{ (cons)}.$$

O'Hearn's incorrectness logic extends the approach of reverse Hoare logic with an explicit handling of error detection and propagation. As pointed out by O'Hearn [2020], "Program correctness and incorrectness are two sides of the same coin [...] Incorrectness logic is so basic that it could have been defined and studied immediately after or alongside the fundamental works of Floyd and Hoare on correctness in the 1960s." Because IL is tailored to under-approximations, it can be used to prove the presence of bugs but not their absence.

In O'Hearn [2020], programs are regular commands that include primitives such as: the error() statement, to halt execution and raise an error signal **er**; assume(b) statements, analogous to our Boolean guards b?; and nondeterministic assignments $x := \text{nond}()$ also present in the setting of reverse Hoare logic. Thus, we set

$$\text{Exp} \ni e ::= \text{skip} \mid x := a \mid \text{error}() \mid \text{assume}(b) \mid x := \text{nond}().$$

Incorrectness logic triples take the form $\vdash_{\text{IL}} [p] \text{ c } [\epsilon : q]$, as their post-conditions are extended with labels $\epsilon \in \{\text{ok}, \text{er}\}$ (following O'Hearn [2020], we use colors for a visual differentiation) to distinguish the case of error-free termination **ok**: q leading to an under-approximating post-condition q , from interrupted computations **er**: q , meaning that some error occurred under the circumstances reported by q . We write $\vdash_{\text{IL}} [p] \text{ c } [\text{ok} : q][\text{er} : w]$ when $\vdash_{\text{IL}} [p] \text{ c } [\text{ok} : q]$ and $\vdash_{\text{IL}} [p] \text{ c } [\text{er} : w]$ are derivable for the same pre-condition p . Notably, the proof system of IL is proved to be sound and complete (in the logical sense): An error can arise iff it can be exposed by some provable triple.

Next, we sketch how IL can be seen as an instance of LCL_A . Due to error handling, the domain $\mathbb{S}$ is not informative enough, but this is not a problem given the generality of LCL_A . Therefore, we exploit the following four ingredients:

- (i) A suitable concrete domain $C \triangleq \wp(\{\text{ok}, \text{er}\} \times \Sigma)$ that can distinguish between normal and erroneous termination. For $q \in \wp(\Sigma)$ and $\epsilon \in \{\text{ok}, \text{er}\}$ we write $\epsilon : q$ as a shorthand for $\{\epsilon : \sigma \mid \sigma \in q\} = \{\epsilon\} \times q$. Clearly, a generic $S \in C$ takes the form **ok**: $q \cup \text{er}$: w for suitable $q, w \in \wp(\Sigma)$, so we denote elements in C more concisely as **ok**: q, er : w .
- (ii) Transfer functions $f : C \rightarrow C$ are additive functions such that

$$f(\text{ok} : q, \text{er} : w) = \text{er} : w \cup \bigcup_{\sigma \in q} f(\text{ok} : \sigma)$$

meaning that all the errors w in the argument are preserved. possibly further errors are generated by the application of f to some $\sigma \in q$.

- (iii) The semantics of basic transfer functions is defined in Figure 8, where we write $q[x \mapsto v]$ as a shorthand for $\{\sigma[x \mapsto v] \mid \sigma \in q\}$. For any $r \in \text{Reg}$ and any $q, w \in \wp(\Sigma)$, by structural induction on r , it follows that $\llbracket r \rrbracket(\text{er} : w) = \text{er} : w$ and $\llbracket r \rrbracket(\text{ok} : q, \text{er} : w) = \llbracket r \rrbracket(\text{ok} : q) \cup \text{er} : w$.
- (iv) The abstract domain A_{tr} is the trivial abstraction such that $\gamma\alpha = \lambda X. \top = \lambda X. (\{\text{ok}, \text{er}\} \times \Sigma)$. Note that A_{tr} is (globally) complete for every transfer function, therefore all proof obligations $\mathbb{C}_c^A(e)$ are trivially satisfied, and (transfer) becomes an axiom. Moreover, by $A_{tr}(p') = \top = A_{tr}(q)$, the rule (relax) boils down to the rule (cons).

At the level of concrete semantics, we can establish a tight connection between the transfer function $\llbracket r \rrbracket : C \rightarrow C$ associated with $r \in \text{Reg}$ and its relational denotational semantics that

$$\begin{aligned}
\llbracket \text{skip} \rrbracket(\text{ok} : q, \text{er} : w) &\triangleq \text{ok} : q, \text{er} : w \\
\llbracket x := a \rrbracket(\text{ok} : q, \text{er} : w) &\triangleq \text{ok} : \bigcup_{\sigma \in q} \{\sigma[x \mapsto \llbracket a \rrbracket \sigma]\}, \text{er} : w \\
\llbracket \text{error}() \rrbracket(\text{ok} : q, \text{er} : w) &\triangleq \text{er} : (q \cup w) \\
\llbracket \text{assume}(b) \rrbracket(\text{ok} : q, \text{er} : w) &\triangleq \text{ok} : (q \cap b), \text{er} : w \\
\llbracket x := \text{nond}() \rrbracket(\text{ok} : q, \text{er} : w) &\triangleq \text{ok} : \bigcup_{v \in \mathbb{Z}} q[x \mapsto v], \text{er} : w
\end{aligned}$$

Fig. 8. Basic transfer functions for additional statements.

was taken as a reference model for IL. Let us denote by $\llbracket r \rrbracket \epsilon \subseteq \Sigma \times \Sigma$ the relational semantics defined in O'Hearn [2020, Figure 4]. To formalize the correspondence, we find it convenient to let $\llbracket r \rrbracket \epsilon : \mathbb{S} \rightarrow \mathbb{S}$ denote the functional version of $\llbracket r \rrbracket \epsilon$ defined by:

$$\llbracket r \rrbracket \epsilon p \triangleq \{\sigma' \in \Sigma \mid \sigma \in p, (\sigma, \sigma') \in \llbracket r \rrbracket \epsilon\}.$$

LEMMA 6.1. *For any $r \in \text{Reg}$ and $p \in \mathbb{S}$, we have:*

$$\llbracket r \rrbracket(\text{ok} : p) = \text{ok} : \llbracket r \rrbracket_{\text{ok}} p, \text{er} : \llbracket r \rrbracket_{\text{er}} p.$$

PROOF. The proof is by structural induction on $r \in \text{Reg}$.

($r \equiv \text{skip}$): By definition,

$$\llbracket \text{skip} \rrbracket(\text{ok} : p) = \text{ok} : p, \text{er} : \emptyset = \text{ok} : \llbracket \text{skip} \rrbracket_{\text{ok}} p, \text{er} : \llbracket \text{skip} \rrbracket_{\text{er}} p.$$

($r \equiv x := a$): By definition,

$$\llbracket x := a \rrbracket(\text{ok} : p) = \text{ok} : \bigcup_{\sigma \in p} \{\sigma[x \mapsto \llbracket a \rrbracket \sigma]\}, \text{er} : \emptyset = \text{ok} : \llbracket x := a \rrbracket_{\text{ok}} p, \text{er} : \llbracket x := a \rrbracket_{\text{er}} p.$$

($r \equiv \text{error}()$): By definition,

$$\llbracket \text{error}() \rrbracket(\text{ok} : p) = \text{ok} : \emptyset, \text{er} : p = \text{ok} : \llbracket \text{error}() \rrbracket_{\text{ok}} p, \text{er} : \llbracket \text{error}() \rrbracket_{\text{er}} p.$$

($r \equiv \text{assume}(b)$): By definition,

$$\llbracket \text{assume}(b) \rrbracket(\text{ok} : p) = \text{ok} : (p \cap b), \text{er} : \emptyset = \text{ok} : \llbracket \text{assume}(b) \rrbracket_{\text{ok}} p, \text{er} : \llbracket \text{assume}(b) \rrbracket_{\text{er}} p.$$

($r \equiv x := \text{nond}()$): By definition,

$$\llbracket x := \text{nond}() \rrbracket(\text{ok} : p) = \text{ok} : \bigcup_{v \in \mathbb{Z}} p[x \mapsto v], \text{er} : \emptyset = \text{ok} : \llbracket x := \text{nond}() \rrbracket_{\text{ok}} p, \text{er} : \llbracket x := \text{nond}() \rrbracket_{\text{er}} p.$$

($r \equiv r_1; r_2$): Let us assume the inductive hypotheses

$$\begin{aligned}
\forall p \in \mathbb{S}. \llbracket r_1 \rrbracket(\text{ok} : p) &= \text{ok} : \llbracket r_1 \rrbracket_{\text{ok}} p, \text{er} : \llbracket r_1 \rrbracket_{\text{er}} p \\
\forall p' \in \mathbb{S}. \llbracket r_2 \rrbracket(\text{ok} : p') &= \text{ok} : \llbracket r_2 \rrbracket_{\text{ok}} p', \text{er} : \llbracket r_2 \rrbracket_{\text{er}} p'.
\end{aligned}$$

Then, we have

$$\begin{aligned}
\llbracket r_1; r_2 \rrbracket(\text{ok} : p) &= \llbracket r_2 \rrbracket(\llbracket r_1 \rrbracket(\text{ok} : p)) = \\
&\llbracket r_2 \rrbracket(\text{ok} : \llbracket r_1 \rrbracket_{\text{ok}} p, \text{er} : \llbracket r_1 \rrbracket_{\text{er}} p) = \\
&\text{er} : \llbracket r_1 \rrbracket_{\text{er}} p \cup \llbracket r_2 \rrbracket(\text{ok} : \llbracket r_1 \rrbracket_{\text{ok}} p) = \\
&\text{er} : \llbracket r_1 \rrbracket_{\text{er}} p \cup \text{ok} : \llbracket r_2 \rrbracket_{\text{ok}}(\llbracket r_1 \rrbracket_{\text{ok}} p), \text{er} : \llbracket r_2 \rrbracket_{\text{er}}(\llbracket r_1 \rrbracket_{\text{ok}} p) = \\
&\text{er} : \llbracket r_1 \rrbracket_{\text{er}} p \cup \text{ok} : (\llbracket r_1 \rrbracket_{\text{ok}} \cdot \llbracket r_2 \rrbracket_{\text{ok}}) p, \text{er} : (\llbracket r_1 \rrbracket_{\text{ok}} \cdot \llbracket r_2 \rrbracket_{\text{er}}) p = \\
&\text{ok} : (\llbracket r_1; r_2 \rrbracket_{\text{ok}} p), \text{er} : (\llbracket r_1 \rrbracket_{\text{er}} p \cup (\llbracket r_1 \rrbracket_{\text{ok}} \cdot \llbracket r_2 \rrbracket_{\text{er}}) p) = \\
&\text{ok} : \llbracket r_1; r_2 \rrbracket_{\text{ok}} p, \text{er} : \llbracket r_1; r_2 \rrbracket_{\text{er}} p.
\end{aligned}$$

$(r \equiv r_1 \oplus r_2)$: Let us assume the inductive hypotheses

$$\begin{aligned} \forall p \in \mathbb{S}. \llbracket r_1 \rrbracket(\text{ok} : p) &= \text{ok} : \llbracket r_1 \rrbracket_{\text{ok}} p, \text{er} : \llbracket r_1 \rrbracket_{\text{er}} p, \\ \forall p \in \mathbb{S}. \llbracket r_2 \rrbracket(\text{ok} : p) &= \text{ok} : \llbracket r_2 \rrbracket_{\text{ok}} p, \text{er} : \llbracket r_2 \rrbracket_{\text{er}} p. \end{aligned}$$

Then, we have

$$\begin{aligned} \llbracket r_1 \oplus r_2 \rrbracket(\text{ok} : p) &= \llbracket r_1 \rrbracket(\text{ok} : p) \cup \llbracket r_2 \rrbracket(\text{ok} : p) = \\ &(\text{ok} : \llbracket r_1 \rrbracket_{\text{ok}} p, \text{er} : \llbracket r_1 \rrbracket_{\text{er}} p) \cup (\text{ok} : \llbracket r_2 \rrbracket_{\text{ok}} p, \text{er} : \llbracket r_2 \rrbracket_{\text{er}} p) = \\ &\text{ok} : (\llbracket r_1 \rrbracket_{\text{ok}} p \cup \llbracket r_2 \rrbracket_{\text{ok}} p), \text{er} : (\llbracket r_1 \rrbracket_{\text{er}} p \cup \llbracket r_2 \rrbracket_{\text{er}} p) = \\ &\text{ok} : \llbracket r_1 \oplus r_2 \rrbracket_{\text{ok}} p, \text{er} : \llbracket r_1 \oplus r_2 \rrbracket_{\text{er}} p. \end{aligned}$$

$(r \equiv r_1^*)$: Let us assume the inductive hypotheses

$$\forall p \in \mathbb{S}. \llbracket r_1 \rrbracket(\text{ok} : p) = \text{ok} : \llbracket r_1 \rrbracket_{\text{ok}} p, \text{er} : \llbracket r_1 \rrbracket_{\text{er}} p.$$

Then, we prove by induction on $n \in \mathbb{N}$ that

$$\forall n \in \mathbb{N}. \llbracket r_1 \rrbracket^n(\text{ok} : p) = \text{ok} : \llbracket r_1^n \rrbracket_{\text{ok}} p, \text{er} : \llbracket r_1^n \rrbracket_{\text{er}} p.$$

Finally, we have that

$$\begin{aligned} \llbracket r_1^* \rrbracket(\text{ok} : p) &= \\ \bigcup \{ \llbracket r_1 \rrbracket^n(\text{ok} : p) \mid n \in \mathbb{N} \} &= \\ \bigcup \{ (\text{ok} : \llbracket r_1^n \rrbracket_{\text{ok}} p, \text{er} : \llbracket r_1^n \rrbracket_{\text{er}} p) \mid n \in \mathbb{N} \} &= \\ \text{ok} : \bigcup \{ \llbracket r_1^n \rrbracket_{\text{ok}} p \mid n \in \mathbb{N} \}, \text{er} : \bigcup \{ \llbracket r_1^n \rrbracket_{\text{er}} p \mid n \in \mathbb{N} \} &= \\ \text{ok} : \llbracket r_1^* \rrbracket_{\text{ok}} p, \text{er} : \llbracket r_1^* \rrbracket_{\text{er}} p. \end{aligned}$$

This therefore concludes the proof. $\square$

Lemma 6.1 is instrumental for proving the equivalence between IL and the particular instance $\text{LCL}_{A_{tr}}$ of our logic as determined by (i)–(iv) above. This correspondence necessarily follows as a consequence of the (logical) completeness of IL [O’Hearn 2020, Theorem 6] and our Theorem 5.14 of limited completeness, which guarantee that any under-approximation of the strongest post-condition $\text{post}[r]p$ is provable in both logics.

COROLLARY 6.2. *For any $p, q, w \in \mathbb{S}$ and $r \in \text{Reg}$:*

$$\vdash_{\text{IL}} [p] \text{ } r \text{ } [\text{ok} : q][\text{er} : w] \quad \text{iff} \quad \vdash_{A_{tr}} [\text{ok} : p] \text{ } r \text{ } [\text{ok} : q, \text{er} : w].$$

It is interesting to observe that any instance LCL_A of our logic using an abstraction $A \neq A_{tr}$ would only be able to derive some triples of IL *but not all of them* (while, of course, any triple derived in $\vdash_A$ would also be derivable in $\vdash_{\text{IL}}$). This is a consequence of the intrinsic incompleteness Theorem 5.15 that extends the impossibility results of Giacobazzi et al. [2015] and Bruni et al. [2020] to regular commands. Therefore, whenever $A \neq A_{tr}$ there will always exist some program r and some triple $\vdash_{\text{IL}} [p] \text{ } r \text{ } [\text{ok} : q][\text{er} : w]$ such that it will not be possible to derive $\vdash_A [\text{ok} : p] \text{ } r \text{ } [\text{ok} : q, \text{er} : w]$ because some proof obligation $\mathbb{C}_c^A(e)$ of local completeness will fail for some basic expression e appearing in r .

The correspondence provided by Corollary 6.2 is interesting, because although the rules of IL and ours share some similarities, they also display significant differences:

- (a) The most visible difference is that the pre-conditions of the triples in $\vdash_{\text{IL}}$ are elements of $\mathbb{S}$ while pre-conditions of triples in $\vdash_{A_{tr}}$ are elements of C , meaning that the rules in $\vdash_{\text{IL}}$ are concerned only with normal inputs, while the rules in $\vdash_{A_{tr}}$ must deal also with (the propagation of) erroneous inputs.

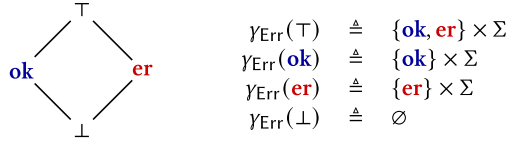


Fig. 9. Abstract domain Err.

- (b) Building on (a), the rules of IL are tailored to error propagation, while our rules are designed for any concrete domain. Both logical frameworks can be extended to deal with different kinds of error and error recovery mechanisms. Because our rule (transfer) is parametric on basic expressions, it should not be necessary to change any rule of our proof system to implement such extensions. For example, IL exploits two rules for sequential composition while LCL_A just needs a single composition rule but it delegates error propagation to the underlying concrete domain.
- (c) Some rules of IL, such as *Disjunction*, *Choice*, and *Iterate zero* in O’Hearn [2020, Figure 2], are designed to incrementally build the under-approximation bottom-up by composing smaller under-approximations into larger ones. On the contrary, LCL_A is constrained to work in the opposite direction by the requirement of preserving the over-approximation of the strongest post-condition in the abstraction A .
- (d) Finally, incorrectness logic includes specific rules for dealing with the introduction of fresh local variables, while in the context of abstract interpretation it is typically assumed that the program variables are statically known. Of course, it would be possible to deal with dynamic allocation in LCL_A although its formalization would be technically more involved.

The use of an abstract over-approximation in LCL_A that constrains any under-approximation has some advantages but also carries drawbacks. The main advantages are as follows: (1) by exploiting the over-approximation, LCL_A can also prove the absence of errors and (2) under the hypothesis that the correctness specification spec is expressible in the abstraction A , *any provable triple will suffice to establish either spec is satisfied or violated*. However, as already mentioned, whenever A is not trivial, not all the possible under-approximations can be obtained by our system. The next example shows this phenomenon by combining the interval abstraction Int with the simple error domain Err depicted in Figure 9.

Example 6.3. Let us revisit Example 5.8 (in turn using the regular command r of Example 5.2). To study $\text{spec}_2 \triangleq x \leq 1000$, in the context of $\vdash_{\text{IL}}$ we focus on the command $\widehat{r}_2 \triangleq r; r_{s2}$ where

$$r_{s2} \triangleq \text{if } x \leq 1000 \text{ then skip else error() } = (x \leq 1000?; \text{skip}) \oplus (1000 < x?; \text{error}()).$$

Using $\vdash_{\text{Int}}$, it was shown in Example 5.8 that $\vdash_{\text{Int}}$ is expressive enough to prove that r satisfies spec_2 , so that $\widehat{r}_2$ will never issue the error signal. Since the absence of errors cannot be proved by under-approximations, incorrectness logic cannot derive any useful information in this case. Let us consider instead the abstract domain $\text{Int}^+ \triangleq \text{Err} \sqcap \text{Int}$ defined as reduced product [Cousot and Cousot 1979, Section 10.1] of Err and Int, that is, whose concretization map is defined as $\gamma_{\text{Int}^+}(\langle a_1, a_2 \rangle) \triangleq \gamma_{\text{Err}}(a_1) \cap \gamma_{\text{Int}}(a_2)$. By mimicking the derivation of $\vdash_{\text{Int}} [p] r \{[0, 2, 1000]\}$, it is not hard to check that $\vdash_{\text{Int}^+} [\text{ok}:p] \widehat{r}_2 [\text{ok}:[0, 2, 1000]]$ is derivable. Since $\text{Int}^+(\text{ok}:[0, 2, 1000]) = \text{ok}:[0, 1000]$ over-approximates the strongest post-condition, this is enough for proving that no error will be issued by $\widehat{r}_2$ with pre-condition p . Actually, by Corollary 5.6, since spec_2 is expressible in Int, we are guaranteed that any provable triple $\vdash_{\text{Int}} [p] r [q]$ in our system will be able to prove that spec_2 holds.

$\frac{\vdash_A [p \wedge b] c [w] \quad \vdash_A [p \vee w] \text{ while } b \text{ do } c [q]}{\vdash_A [p] \text{ while } b \text{ do } c [q]} \text{ (unroll)}$	$\frac{\mathbb{C}_p^A(b) \quad \mathbb{C}_p^A(\neg b) \quad \vdash_A [p \wedge b] c [q] \quad q \leq A(p)}{\vdash_A [p] \text{ while } b \text{ do } c [(p \vee q) \wedge \neg b]} \text{ (loopinv)}$
---	---

Fig. 10. The rules for while loops.

To study $\text{spec}_3 \triangleq 100 \leq x$, we focus on $\widehat{r}_3 \triangleq r; r_{s3}$ where

$$r_{s3} \triangleq \text{if } 100 \leq x \text{ then skip else error()} = (100 \leq x?; \text{skip}) \oplus (x < 100?; \text{error}())$$

We know from Example 5.8, that $\llbracket r \rrbracket p \not\subseteq \text{spec}_3$, so that $\widehat{r}_3$ can issue some errors. Within IL we can derive triples that exhibit some erroneous situation, like $\vdash_{\text{IL}} [p] \widehat{r}_3 [\text{er}; \{0\}]$ as well as others that do not, e.g., $\vdash_{\text{IL}} [p] \widehat{r}_3 [\text{ok}; \{1000\}]$.

Let us consider once again the abstract domain Int^+ . By mimicking the derivation of $\vdash_{\text{Int}} [p] r [\{0, 2, 1000\}]$, but applying (rec) 100 times to include the values 100 and 101 that are necessary to satisfy the local completeness requirements for the tests $100 \leq x?$ and $x < 100?$, we can then derive $\vdash_{\text{Int}^+} [\text{ok}; p] \widehat{r}_3 [\text{ok}; \{1000\}, \text{er}; \{0, 2\}]$ using the abstract domain Int^+ . Note that, since $\text{Int}^+(\text{ok}; \{1000\}, \text{er}; \{0, 2\}) = \top; [0, 1000]$, the rule (relax) cannot be used to discard all the errors because this would change the abstract over-approximation of the post-condition. Since $\text{ok}; [100, +\infty]$ is expressible in Int^+ , by Corollary 5.6, the label $\top$ in the over-approximation provides good evidence about the occurrence of one or more errors. Moreover, because the over-approximation induced by Int is always preserved, any provable triple $\vdash_{\text{Int}^+} [p] \widehat{r}_3 [q]$ is such that q will contain the true alarm $\text{er}; \{0\}$ (as well as $\text{ok}; \{1000\}$).

To develop some tool or extend existing ones to exploit LCL_A the idea is to start the analysis with some well-known abstract domain and then collect the local completeness proof obligations arising from the analysis: When they are not satisfied, then a new abstract domain must be synthesized to carry out the proof. As discussed in the conclusions, a successful strategy outlined in Bruni et al. [2022] would be to refine the current abstract domain in a minimal way to recover local completeness of the analysis.

7 Imp PROGRAMS

The encoding (6) of while loops as regular commands is such that, due to the recursive rule (rec), proof obligations of local completeness for the Boolean guard of the loop are required at each iteration. This turns out to be advantageous to keep the rules of the logic as simple and straightforward as possible. However, this is not strictly necessary for achieving local completeness of the loop invariant. We therefore introduce in Figure 10 two additional rules for while loops that relax the local completeness requirements needed during their recursive applications. The key new rule is (unroll): it is analogous to the rule (rec) when applied to $r = b?; c$, but the premise $\vdash_A [p \wedge b] c [w]$ of (unroll) requires local completeness exclusively on the body c of the loop and not on the Boolean guard $b?$. Moreover, the rule (loopinv) is analogous to (iterate) and can be used to stop a proof attempt as soon as an abstract fixpoint $A(p)$ is reached. By combining these rules (unroll) and (loopinv), the local completeness requirements for the Boolean guard $b?$ of the loop, namely $\mathbb{C}_p^A(b?)$ and $\mathbb{C}_p^A(\neg b?)$, are therefore needed only at the last iteration and not at each iteration.

LEMMA 7.1. *The proof system extended with rules (unroll) and (loopinv) is sound.*

PROOF. We extend the proof by induction of Theorem 5.5 by considering the case where the last rule applied to derive $\vdash_A [p] r [q]$ is either (unroll) or (loopinv). As in the proof of Theorem 5.5,

we refer to the equality $\llbracket r \rrbracket_A^\# \alpha(p) = \alpha(q)$ as (2a) and use (2b) for the equality $\llbracket r \rrbracket_A^\# \alpha(p) = \alpha(\llbracket r \rrbracket p)$, and we recall that (2b) follows immediately by (1) and (2a).

(unroll): If the last rule applied is (unroll), then it must be $r \equiv \mathbf{while\ b\ do\ c}$, and we can assume by induction that $\vdash_A [p \wedge b] c [w]$ and $\vdash_A [p \vee w] \mathbf{while\ b\ do\ c} [q]$ can be derived for some w . We need to prove the thesis for the conclusion $\vdash_A [p] \mathbf{while\ b\ do\ c} [q]$.

(1) By inductive hypothesis (1), we have that $w \leq \llbracket c \rrbracket (p \wedge b) = \llbracket b?; c \rrbracket p$, so that, by Lemma 2.1 (e), it follows $\llbracket (b?; c)^* \rrbracket p = \llbracket (b?; c)^* \rrbracket (p \vee w)$. Then, by inductive hypothesis (1), we derive the following:

$$q \leq \llbracket \mathbf{while\ b\ do\ c} \rrbracket (p \vee w) = \llbracket \neg b? \rrbracket (\llbracket (b?; c)^* \rrbracket (p \vee w)) = \llbracket \neg b? \rrbracket (\llbracket (b?; c)^* \rrbracket p) = \llbracket \mathbf{while\ b\ do\ c} \rrbracket p.$$

(2) The proof is analogous to the case of rule (rec) in the proof of Theorem 5.5:

$$\begin{aligned} \alpha(q) &\leq_A && \text{[by monotonicity of } \alpha \text{ and (1)]} \\ \alpha(\llbracket \mathbf{while\ b\ do\ c} \rrbracket p) &\leq_A && \text{[by correctness (4)]} \\ \llbracket \mathbf{while\ b\ do\ c} \rrbracket_A^\# \alpha(p) &\leq_A && \text{[by monotonicity]} \\ \llbracket \mathbf{while\ b\ do\ c} \rrbracket_A^\# \alpha(p \vee w) &= && \text{[by ind.hyp. (2a)]} \\ &\alpha(q), \end{aligned}$$

so that $\llbracket \mathbf{while\ b\ do\ c} \rrbracket_A^\# \alpha(p) = \alpha(q) = \alpha(\llbracket \mathbf{while\ b\ do\ c} \rrbracket p)$ follows.

(loopinv): If the last rule applied is (loopinv), then it must be $r \equiv \mathbf{while\ b\ do\ c}$, and we can assume by induction that $\vdash_A [p \wedge b] c [q_1]$ can be derived for some q_1 such that $q_1 \leq A(p)$ and $q = (p \vee q_1) \wedge \neg b$, with $\mathbb{C}_p^A(b)$ and $\mathbb{C}_p^A(\neg b)$. We need to prove the thesis for the conclusion $\vdash_A [p] \mathbf{while\ b\ do\ c} [(p \vee q_1) \wedge \neg b]$.

(1) We first observe that, by inductive hypothesis (1), $q_1 \leq \llbracket c \rrbracket (p \wedge b) = \llbracket b?; c \rrbracket p$ and that $\llbracket b?; c \rrbracket^0 p = p$. Then

$$p \vee q_1 \leq \llbracket b?; c \rrbracket^0 p \vee \llbracket b?; c \rrbracket^1 p \leq \bigvee_{n \in \mathbb{N}} \llbracket b?; c \rrbracket^n p = \llbracket (b?; c)^* \rrbracket p.$$

From this, it immediately follows by monotonicity of $\llbracket \neg b \rrbracket$:

$$(p \vee q_1) \wedge \neg b = \llbracket \neg b \rrbracket (p \vee q_1) \leq \llbracket \neg b \rrbracket (\llbracket (b?; c)^* \rrbracket p) = \llbracket \mathbf{while\ b\ do\ c} \rrbracket p.$$

(2a) By $\mathbb{C}_p^A(b)$, we get $\alpha(p \wedge b) = \alpha(\llbracket b \rrbracket p) = \llbracket b \rrbracket_A^\# \alpha(p)$. Therefore, $\alpha(q_1) = \llbracket c \rrbracket_A^\# \alpha(p \wedge b) = \llbracket c \rrbracket_A^\# \llbracket b \rrbracket_A^\# \alpha(p) = \llbracket b?; c \rrbracket_A^\# \alpha(p)$. By monotonicity of α , we have that $\alpha(q_1) \leq_A \alpha(p)$, so that Lemma 2.1 (c) is applicable to derive $\llbracket (b?; c)^* \rrbracket_A^\# \alpha(p) = \alpha(p)$. Since $\mathbb{C}_p^A(\neg b)$ and $p \leq (p \vee q_1) \leq A(p)$ hold, by convexity, we get that $\mathbb{C}_{(p \vee q_1)}^A(\neg b)$ holds. Therefore,

$$\begin{aligned} \alpha((p \vee q_1) \wedge \neg b) &= && \text{[by definition]} \\ \alpha(\llbracket \neg b \rrbracket (p \vee q_1)) &= && \text{[by convexity } \mathbb{C}_{(p \vee q_1)}^A(\neg b)] \\ \llbracket \neg b \rrbracket_A^\# \alpha(p \vee q_1) &= && \text{[by additivity of } \alpha] \\ \llbracket \neg b \rrbracket_A^\# (\alpha(p) \vee_A \alpha(q_1)) &= && \text{[by hyp. } q_1 \leq A(p)] \\ \llbracket \neg b \rrbracket_A^\# \alpha(p) &= && \text{[by Lemma 2.1 (c), see above]} \\ \llbracket \neg b \rrbracket_A^\# (\llbracket (b?; c)^* \rrbracket_A^\# \alpha(p)) &= && \text{[by definition]} \\ \llbracket \mathbf{while\ b\ do\ c} \rrbracket_A^\# \alpha(p) &= && \end{aligned}$$

□

We refer to the next section for an example (cf. Example 8.2) showing the advantage of applying the rules (unroll) and (loopinv) because the local completeness requirements are not met at every iteration of the proof for a while loop but just at the last one.

$$\boxed{\frac{A' \leq A \quad A(p) = A'(p) \quad \vdash_{A'}^{\leq} [p] \text{ r } [q] \quad A'(q) = A(q)}{\vdash_A^{\leq} [p] \text{ r } [q]}} \text{ (refine)}$$

Fig. 11. The refinement rule of $\text{LCL}_A^{\leq}$.

8 A LOGIC FOR LOCALLY COMPLETE BEST CORRECT APPROXIMATIONS

When a provable triple $\vdash_A [p] \text{ r } [q]$ is used for program verification, we exploit the following property:

$$q \leq \llbracket r \rrbracket p \leq \alpha(\llbracket r \rrbracket p) = \alpha(q), \quad (§)$$

so that, by under-approximation $q \leq \llbracket r \rrbracket p$, any alarm in q is a true alarm for p , and, by over-approximation $\llbracket r \rrbracket p \leq \alpha(q)$, the lack of alarms in $\alpha(q)$ entails the correctness of p . In LCL_A , local completeness can be viewed as a technical assumption to infer (§), meaning that it is a necessary condition for deriving $\vdash_A [p] \text{ r } [q]$ within our proof system, since we are able to prove only triples that satisfy local completeness $\llbracket r \rrbracket_A^\# \alpha(p) = \alpha(\llbracket r \rrbracket p)$ (cf. Theorem 5.5). In this section, we show that it is possible to relax our program logic so that local completeness is not required for the intensional and inductively defined abstract interpreter $\llbracket r \rrbracket_A^\#$ but merely for the best correct approximation $\llbracket r \rrbracket^A \triangleq \alpha \circ \llbracket r \rrbracket \circ \gamma$ of the extensional concrete semantics $\llbracket r \rrbracket$. This is achieved by allowing different abstract domains in different sub-derivations to increase the precision of the analysis whenever necessary. Without the extension proposed in this section, whenever it would be convenient to use different abstract domains for different parts of the proof, the only possibility would be to check if it is possible to complete the derivation in the abstract domain obtained as the reduced product of all domains involved in every sub-derivation: For example, if we are able to derive $\vdash_{A_1} [p] \text{ r }_1 [w]$ and $\vdash_{A_2} [w] \text{ r }_2 [q]$, then we could try to derive $\vdash_{A_1 \sqcap A_2} [p] \text{ r }_1; \text{ r }_2 [q]$ leveraging the reduced product $A_1 \sqcap A_2$. One remarkable advantage of using different abstractions within the same derivation will be that it is not necessary to consider their join at every step.

This extension of LCL_A is obtained by adding the rule (refine) in Figure 11, where we recall that $\leq$ denotes the refinement relation between abstract domains and write $\vdash_A^{\leq} [p] \text{ r } [q]$ for a triple that can be derived in this extended proof system $\text{LCL}_A^{\leq} \triangleq \text{LCL}_A \cup \{(\text{refine})\}$. When A is not locally complete for r on p , (refine) allows us to exploit an abstraction refinement A' of A , which is locally complete provided that the over-approximations in A and A' of both p and q coincide. The soundness result for $\text{LCL}_A^{\leq}$ shows that any triple $\vdash_A^{\leq} [p] \text{ r } [q]$ still ensures that $q \leq \llbracket r \rrbracket p \leq \alpha(q)$ holds. Let us remark that the only difference in soundness of LCL_A and $\text{LCL}_A^{\leq}$, as stated by Theorems 5.5 and 8.1, is that the intensional abstract semantics $\llbracket r \rrbracket_A^\#$ of Theorem 5.5 is replaced by the bca $\llbracket r \rrbracket^A$ of Theorem 8.1.

THEOREM 8.1 (SOUNDNESS OF $\text{LCL}_A^{\leq}$). *Let $A_{\alpha, \gamma} \in \text{Abs}(C)$. For all $r \in \text{Reg}$, $p, q \in C$, if $\vdash_A^{\leq} [p] \text{ r } [q]$ then:*

- (1) $q \leq \llbracket r \rrbracket p$, and
- (2) $\llbracket r \rrbracket^A \alpha(p) = \alpha(q) = \alpha(\llbracket r \rrbracket p)$.

PROOF. As in the proof of Theorem 5.5, we refer to the equality $\llbracket r \rrbracket^A \alpha(p) = \alpha(q)$ as (2a) and use (2b) for the equality $\llbracket r \rrbracket^A \alpha(p) = \alpha(\llbracket r \rrbracket p)$. We then recall that (2b) follows immediately by (1) and (2a).

The proof is by induction on the derivation tree of $\vdash_A^{\leq} [p] \text{ r } [q]$. For the cases where the last used rule is in LCL_A (in Figure 4), the proof follows the same pattern of the proof of Theorem 5.5: for (1) there is nothing to change, while for (2) we just need to replace $\llbracket \cdot \rrbracket_A^\#$ with $\llbracket \cdot \rrbracket^A$.

everywhere and slightly change the proof of (2a) for the sequence with the additional inequality $\llbracket r_1; r_2 \rrbracket^A \alpha(p) \leq_A \llbracket r_2 \rrbracket^A (\llbracket r_1 \rrbracket^A \alpha(p))$ as follows:

$$\begin{aligned}
 \alpha(q) &\leq_A && \text{[by (1) and monotonicity of } \alpha \text{]} \\
 \alpha(\llbracket r_1; r_2 \rrbracket p) &\leq_A && \text{[by correctness (4)]} \\
 \llbracket r_1; r_2 \rrbracket^A \alpha(p) &\leq_A && \text{[by definition]} \\
 \llbracket r_2 \rrbracket^A (\llbracket r_1 \rrbracket^A \alpha(p)) &= && \text{[by ind.hyp. (2a), } \llbracket r_1 \rrbracket^A \alpha(p) = \alpha(w) \text{]} \\
 \llbracket r_2 \rrbracket^A \alpha(w) &= && \text{[by ind.hyp. (2a)]} \\
 \alpha(q). & & &
 \end{aligned}$$

Analogously, we slightly change the proof of (2a) for (iterate) as follows:

$$\begin{aligned}
 \llbracket r_1^* \rrbracket^A \alpha(p) &= && \text{[by Lemma 2.1 (d)]} \\
 \alpha(p) &= && \text{[by hyp. } q \leq A(p) \text{]} \\
 \alpha(p) \vee_A \alpha(q) &= && \text{[by additivity of } \alpha \text{]} \\
 \alpha(p \vee q). & & &
 \end{aligned}$$

Instead, no changes are necessary for the join, because the abstract semantics of the choice command preserves bcas (cf. Equation (5)).

Therefore, to conclude, we just need to consider the case for the new rule (refine):

(refine) : If the last rule applied is (refine), then we can assume by induction that $\vdash_{A'}^{\leq} [p] \text{ r } [q]$ for some abstraction refinement $A' \leq A$ such that $A'(p) = A(p)$ and $A'(q) = A(q)$. We need to prove the thesis for the conclusion $\vdash_A^{\leq} [p] \text{ r } [q]$.

(1) By inductive hypothesis (1) we already have that $q \leq \llbracket r \rrbracket p$ holds.

(2) Let us recall that $\llbracket r \rrbracket^A \alpha(p) = \alpha(\llbracket r \rrbracket A(p))$. We first note that, by inductive hypothesis (2a), $\alpha'(\llbracket r \rrbracket A'(p)) = \alpha'(q)$, so that $\llbracket r \rrbracket A'(p) \leq A'(q)$ holds. Therefore:

$$\begin{aligned}
 \alpha(q) &\leq_A && \text{[by monotonicity of } \alpha \text{ and (1)]} \\
 \alpha(\llbracket r \rrbracket p) &\leq_A && \text{[by monotonicity of } \alpha \circ \llbracket r \rrbracket \text{]} \\
 \alpha(\llbracket r \rrbracket A(p)) &= && \text{[by hyp.]} \\
 \alpha(\llbracket r \rrbracket A'(p)) &\leq_A && \text{[by monotonicity of } \alpha \text{ and above hyp.]} \\
 \alpha(A'(q)) &= && \text{[by hyp.]} \\
 \alpha(A(q)) &= && \text{[by GI]} \\
 \alpha(q) & & &
 \end{aligned}$$

so that, $\llbracket r \rrbracket^A \alpha(p) = \alpha(q) = \alpha(\llbracket r \rrbracket p)$ follows. $\square$

The following example shows how the rule (refine) allows us to infer a triple $\vdash_A^{\leq} [p] \text{ r } [q]$ when $\llbracket r \rrbracket_A^\#$ is not locally complete for A on p , i.e., soundness as stated by Theorem 5.5 does not hold, while the best correct approximation $\llbracket r \rrbracket^A$ turns out to be locally complete. Interestingly, the rule (refine) allows us to combine sub-derivations that are computed in different abstractions without the need to resort to a common more precise abstract domain such as their reduced product.

Example 8.2 (Benefits of (refine)). Let us consider the `Imp` program $c \triangleq c_1; c_2$, where

$$c_1 \triangleq y := (2 * y) + 1; \text{AbsVal}(y), \quad c_2 \triangleq x := y; c_3 \quad c_3 \triangleq \text{while } x > 1 \text{ do } \{x := x - 1; y := \max(0, y - 1)\},$$

using the syntactic sugar

$$y := \max(a_1, a_2) \triangleq \text{if } a_2 < a_1 \text{ then } y := a_1 \text{ else } y := a_2.$$

Consider the pre-condition $p \triangleq y \in [-101, 100]$. It turns out that the interval domain $A \triangleq \text{Int}$ is not locally complete, because

$$\begin{aligned} \llbracket c \rrbracket_A^\# \alpha(p) &= \llbracket c_2 \rrbracket_A^\# (\llbracket c_1 \rrbracket_A^\# (y \in [-101, 100])) \\ &= \llbracket c_2 \rrbracket_A^\# (\llbracket \text{AbsVal}(y) \rrbracket_A^\# (\llbracket y := (2 * y) + 1 \rrbracket_A^\# (y \in [-101, 100]))) \\ &= \llbracket c_2 \rrbracket_A^\# (\llbracket \text{AbsVal}(y) \rrbracket_A^\# (y \in [-201, 201])) \\ &= \llbracket c_3 \rrbracket_A^\# (\llbracket x := y \rrbracket_A^\# (y \in [0, 201])) \\ &= \llbracket c_3 \rrbracket_A^\# (x \in [0, 201] \wedge y \in [0, 201]) \\ &= x \in [0, 1] \wedge y \in [0, 201], \end{aligned}$$

while

$$\begin{aligned} \llbracket c \rrbracket \alpha(p) &= \llbracket c_2 \rrbracket (\llbracket c_1 \rrbracket (y \in [-101, 100])) \\ &= \llbracket c_2 \rrbracket (\llbracket \text{AbsVal}(y) \rrbracket (\llbracket y := (2 * y) + 1 \rrbracket (y \in [-101, 100]))) \\ &= \llbracket c_2 \rrbracket (\llbracket \text{AbsVal}(y) \rrbracket (y \in [-201, 201] \wedge y \text{ odd})) \\ &= \llbracket c_3 \rrbracket (\llbracket x := y \rrbracket (y \in [1, 201] \wedge y \text{ odd})) \\ &= \llbracket c_3 \rrbracket (x = y \wedge y \in [1, 201] \wedge y \text{ odd}) \\ &= \llbracket x \leq 1? \rrbracket (\llbracket (x > 1?; x := x - 1; y := \max(0, y - 1)) \rrbracket^* (x = y \wedge y \in [1, 201] \wedge y \text{ odd})) \\ &= \llbracket x \leq 1? \rrbracket (x = y \wedge y \in [1, 201]) \\ &= x \in [1, 1] \wedge y \in [1, 1], \end{aligned}$$

and thus $\llbracket c \rrbracket^A \alpha(p) = \alpha(\llbracket c \rrbracket A(p)) = \alpha(\llbracket c \rrbracket p) = x \in [1, 1] \wedge y \in [1, 1]$. Therefore, by Theorem 5.5 (2), it is not possible to derive in LCL_A a triple $\vdash_A [p] c [q]$, for any q . However, we show that, in this case, we can successfully resort to the rule (refine) to infer the triple $\vdash_A^{\leq} [p] c [q]$ with $q \triangleq x = 1 \wedge y = 1$. To achieve this, we identify two suitable refinements of Int that are used in different subtrees of the logical derivation of $\vdash_A^{\leq} [p] c [q]$. The lack of local completeness in Int is due to two reasons:

- (i) the presence of the value 0 introduced in the over-approximation of c_1 ;
- (ii) the linear relationship $x = y$ between the variables x and y after the assignment $x := y$ cannot be expressed in a nonrelational abstraction such as Int .

To address (i), we enrich the interval abstraction by adding a new abstract value $\mathbb{Z}_{\neq 0}$ expressing $x \neq 0$. Let A_1 denote the Moore closure of Int with this additional point $\mathbb{Z}_{\neq 0}$, so that A_1 contains all the intervals possibly having a “hole” in 0 (see Example 4.6). Using $A_1 \leq A$, we can infer the triple $\vdash_{A_1} [p] c_1 [w]$ in LCL_{A_1} , where the post-condition is $w \triangleq y \in \{1, 3, 201\}$, as shown by the proof in Figure 12, where the application of the rules is quite straightforward. In particular, note the use of rule (relax) to reduce the number of concrete points in the under-approximation and the inclusion of the value 3 in w , which will be useful for accelerating the convergence in the successive part of the proof carried out in a second abstraction refinement A_2 .

To address (ii), we consider the well-known weakly relational abstract domain $A_2 \triangleq \text{Oct}$ of octagons [Miné 2006]. Oct consists of octagonal constraints between two variables x, y of type $\pm x \pm y \leq k$ and interval constraints of type $\pm x \leq k$, where $k \in \mathbb{Z} \cup \{-\infty, +\infty\}$, while the representation of the abstract values in Oct relies on difference bound matrices. $(\text{Oct}, \subseteq)$ is a complete lattice and is defined by a GI such that, for all $X \in \wp(\mathbb{Q}^n)$, $\text{Oct}(X)$ is the least octagon containing X . Using A_2 it is possible to derive the triple $\vdash_{A_2} [p] c_2 [q]$ in LCL_{A_2} , as shown in Figure 13, where

$$\begin{array}{c}
\frac{\frac{C_{w_1}^{A_1} (y < 0?)}{\vdash_{A_1} [w_1] (y < 0?) [y \in \{-201, -3, -1\}]} \quad (\text{transfer}) \quad \frac{C_{w_1}^{A_1} (y \in \{-201, -3, -1\}) (y := -y)}{\vdash_{A_1} [y \in \{-201, -3, -1\}] y := -y [w]} \quad (\text{transfer})}{\vdash_{A_1} [w_1] (y < 0?) [y \in \{-201, -3, -1\}] y := -y [w]} \quad (\text{seq}) \\
(*) \\
\frac{\frac{C_p^{A_1} (y := 2 * y + 1)}{\vdash_{A_1} [p] y := 2 * y + 1 [y \in [-201, 201] \wedge y \text{ odd}]} \quad (\text{transfer}) \quad \frac{(*)}{\vdash_{A_1} [w_1] (y < 0?) [y := -y [w]]} \quad (\text{seq}) \quad \frac{C_{w_1}^{A_1} (y \geq 0?)}{\vdash_{A_1} [w_1] (y \geq 0?) [w]} \quad (\text{transfer}) \quad \frac{C_w^{A_1} (\text{skip})}{\vdash_{A_1} [w] \text{skip} [w]} \quad (\text{transfer})}{\vdash_{A_1} [p] y := 2 * y + 1 [w]} \quad (\text{relax}) \quad \frac{(*)}{\vdash_{A_1} [w_1] (y < 0?) [y := -y [w]]} \quad (\text{seq}) \quad \frac{(*)}{\vdash_{A_1} [w_1] (y \geq 0?) [w]} \quad (\text{seq}) \quad \frac{(*)}{\vdash_{A_1} [w] \text{skip} [w]} \quad (\text{seq})}{\vdash_{A_1} [p] y := 2 * y + 1 [w]} \quad (\text{join}) \\
\vdash_{A_1} [p] c_1 [w] \quad (\text{seq})
\end{array}$$

Fig. 12. Derivation of $\vdash_{A_1} [p] c_1 [w]$ for Example 8.2, with $w_1 \triangleq y \in \{-201, -3, -1, 1, 3, 201\}$.

$$\begin{array}{c}
\frac{C_{A_2}^{A_2} (x = y \wedge y \in \{3, 201\}) (x := x - 1)}{\vdash_{A_2} [x = y \wedge y \in \{3, 201\}] x := x - 1 [x = y - 1 \wedge y \in \{3, 201\}]} \quad (\text{transfer}) \quad \frac{C_{A_2}^{A_2} (x = y - 1 \wedge y \in \{3, 201\}) (y := \max(0, y - 1))}{\vdash_{A_2} [x = y - 1 \wedge y \in \{3, 201\}] y := \max(0, y - 1) [x = y \wedge y \in \{2, 200\}]} \quad (\text{transfer})}{\vdash_{A_2} [x = y \wedge y \in \{3, 201\}] c_4 [x = y \wedge y \in \{2, 200\}]} \quad (\text{seq}) \\
(*) \quad (\text{unroll}) \\
\frac{C_{A_2}^{A_2} (x = y \wedge w_3) (x := x - 1)}{\vdash_{A_2} [x = y \wedge w_3] x := x - 1 [x = y - 1 \wedge w_3]} \quad (\text{transfer}) \quad \frac{C_{A_2}^{A_2} (x = y - 1 \wedge w_3) (y := \max(0, y - 1))}{\vdash_{A_2} [x = y - 1 \wedge w_3] y := \max(0, y - 1) [x = y \wedge y \in \{1, 2, 199, 200\}]} \quad (\text{transfer})}{\vdash_{A_2} [x = y \wedge w_3] c_4 [x = y \wedge y \in \{1, 2, 199, 200\}]} \quad (\text{seq}) \\
(**) \quad (\text{loopinv}) \\
\frac{C_w^{A_2} (x := y)}{\vdash_{A_2} [w] x := y [x = y \wedge w]} \quad (\text{transfer}) \quad \frac{(*)}{\vdash_{A_2} [x = y \wedge w_2] (x > 1?) \quad C_{A_2}^{A_2} (x = y \wedge w_2) (x \leq 1?) \quad (**)} \quad \frac{(*)}{\vdash_{A_2} [x = y \wedge y \in \{1, 2, 199, 200\}] \leq (x = y \wedge y \in \{1, 201\})} \quad (\text{loopinv})}{\vdash_{A_2} [w] x := y [x = y \wedge w]} \quad (\text{seq}) \\
(*) \quad (\text{unroll}) \\
\vdash_{A_2} [w] c_2 [q]
\end{array}$$

Fig. 13. Derivation of $\vdash_{A_2} [w] x := y; c_2 [q]$ for Example 8.2, with $w_2 \triangleq y \in \{1, 2, 3, 200, 201\}$ and $w_3 \triangleq y \in \{2, 3, 200, 201\}$.

$c_4 \triangleq x := x - 1; y := \max(0, y - 1)$ is the loop body. This derivation provides an example of use of the rules (unroll) and (loopinv), discussed in Section 7, in lieu of (rec) and (iterate): In fact, the local completeness for the Boolean guard $x > 1$ of c_2 is not satisfied at each iteration but only when the abstract invariant is computed.

Finally, these two proofs $\vdash_{A_1} [p] c_1 [w]$ and $\vdash_{A_2} [w] c_2 [q]$ can be combined using (refine) twice and then (seq) to infer $\vdash_A^{\leq} [p] c [q]$ in $\text{LCL}_A^{\leq}$ as follows:

$$\frac{A_1 \leq A \quad A_1(p) = A(p) = p \quad \vdash_{A_1}^{\leq} [p] c_1 [w] \quad A_1(w) = A(w) = y \in [1, 201]}{\vdash_A^{\leq} [p] c_1 [w]} \quad (\text{refine}) \quad \frac{A_2 \leq A \quad A_2(w) = A(w) = y \in [1, 201] \quad \vdash_{A_2}^{\leq} [w] c_2 [q] \quad A_2(q) = A(q) = q}{\vdash_A^{\leq} [w] c_2 [q]} \quad (\text{refine}) \\
\vdash_A^{\leq} [p] c [q] \quad (\text{seq})$$

Note that, while the bounds of the interval $p = [-101, 100]$ have been chosen so to ease the derivation, the same example would apply to any (finite) input p that contains at least a negative and a positive number.

Let us remark how the above example shows the ability of rule (refine) of carrying out different parts of a proof in some abstraction refinements of a domain A in a situation where local completeness would fail in the original abstraction A . Here, the combined use of two refinements A_1 and A_2 turns out to be more convenient than carrying out the whole proof in a more concrete domain that could be designed, e.g., as reduced product of A_1 and A_2 , because this latter domain would not necessarily be locally complete for the whole program.

9 RELATED WORK

As mentioned, de Vries and Koutavas [2011] were the first to introduce under-approximation triples by proposing the backward consequence rule in reverse Hoare logic for the analysis of non-deterministic algorithms, e.g., for array shuffle. Later, the idea of defining a logic of under-approximation has been fully developed into the design of Incorrectness Logic [O’Hearn 2020], whose comparison with our work has been fully investigated in Section 6. Incorrectness logic attracted a lot of research interest and originated a recent strand of work that aims to exploit incorrectness triples for catching memory errors by moving under-approximating reasoning to separation logic for pointer analysis [Le et al. 2022; Maksimovic et al. 2022; Poskitt 2021; Poskitt and Plump 2023; Raad et al. 2020, 2022; Yan et al. 2022]. The ability of incorrectness triples for compositional bug finding strategies has been practically shown by Le et al. [2022], where 15 new real bugs in OpenSSL were discovered and reported to OpenSSL maintainers thanks to Pulse-X, an automatic program analysis tool based on incorrectness separation logic (ISL) [Raad et al. 2020]. Of course, the main benefit of Pulse-X is that all reported errors are true positives. The work by Raad et al. [2022] extends the above bug catching theory to concurrent programs by defining a parametric framework able to deal with race detection, deadlock detection, and memory safety error detection. Exact separation logic (ESL) [Maksimovic et al. 2022] relies on similar ideas but defines exact triples, in the sense that the consequences are at the same time an under- and over-approximation of the semantics: while the ISL quadruple $[p] \text{ r } [\text{ok} : q][\text{er} : w]$ asserts that any state satisfying either the success post-condition or the error post-condition is the outcome of executing the command r from some state satisfying the pre-condition p , the ESL quadruple $(p) \text{ r } (\text{ok} : q)(\text{er} : w)$ additionally guarantees that any terminating execution of r starting from a state satisfying the pre-condition p either leads to a success state that satisfies q or raises some fault in a state that satisfies w . A major difference is therefore that valid ISL quadruples $[p] \text{ r } [\text{ok} : q][\text{er} : w]$ can always be split into valid triples of the form $[p] \text{ r } [\text{ok} : q]$ and $[p] \text{ r } [\text{er} : w]$, while ESL quadruples $(p) \text{ r } (\text{ok} : q)(\text{er} : w)$ are atomic because they must account for all behaviours. Our conjecture is that ESL accounts for the case of LCL_A where the trivial identity abstraction is considered, but to confirm this intuition we first plan to explore whether and how the relationship of LCL_A with IL studied in Section 6 could be extended to ISL [Raad et al. 2020]. The main challenge for achieving this extension will be to engineer a suitable frame rule for heap assertions that is able to preserve local completeness for the abstraction A .

Recently, Zilberstein et al. [2023] put forward Outcome Logic (OL), a generalization of Hoare Logic that is able to find true bugs while preserving the ability of proving programs correct. OL is parametric on a monoidal structure for predicates and on a so-called outcome assertion logic, and this enables correctness and incorrectness reasoning within the same program logic. OL does not consider the chance of abstracting the predicates as we do in LCL_A , so that we envisage that this challenge can be an appealing subject for future work.

Kleene algebras with tests (KAT) enriched by forward and backward box and diamond operators, so-called modal KATs, have been exploited to unify correctness and incorrectness logics in a single algebraic framework [Möller et al. 2021]. Möller and Struth [2006] already formalized the symmetries of forward and backward box and diamond operators as Galois connections, complementarities and dualities, and provided algebraic soundness and completeness proofs for Hoare logic. Later, Möller et al. [2021] showed that modal KATs with countable joins of tests can also embed incorrectness logic and can be used to draw some links between correctness and incorrectness specifications as well as to deal with backward under-approximations. Recently, Zhang et al. [2022] have shown that O’Hearn incorrectness logic cannot be formulated within a conventional KAT, but, at the same time, a full fledged modal KAT is not needed. In fact, Zhang et al. [2022] prove that a KAT including a greatest element, so-called TopKAT, is capable to encode both Hoare and

O'Hearn program logics in a purely equational fashion. Milanese and Ranzato [2022] gave a further contribution within this stream of works by showing how KATs extended either with a modal diamond operator or with a top element are able to encode our local completeness logic LCL_A . Thus, this latter result generalizes both the modal KAT [Möller et al. 2021] and TopKAT [Zhang et al. 2022] approaches for encoding Hoare correctness and O'Hearn incorrectness logics.

Regarding the use of under-approximation in abstract interpretation, Cousot and Cousot introduced the general framework that could be used to study both over- and under-approximations already in their seminal work [Cousot and Cousot 1977]. However, to the extent of our knowledge, there are no abstract domains thought from the ground up for under-approximation in forward program analysis. The main problems in designing significant abstract domains for under-approximation have been recently studied by Ascari et al. [2022].

10 CONCLUSION AND FUTURE WORK

We presented a program logic for locally complete abstract interpretations, called LCL_A and parametric on an abstract domain A . LCL_A can *prove the presence as well as the absence of true alarms*, meaning that proofs in LCL_A are potentially able to infer *both the correctness and incorrectness* of some program specification. As far as we know, LCL_A combines for the first time over- and under-approximations in a logical proof system for programs based on abstract interpretation. We think that this work opens up many promising lines of research for the automatic verification of program correctness and incorrectness.

When some proof obligation $\mathbb{C}_p^A(f)$ about the local completeness in the abstract domain A of some basic transfer function f fails, a natural question is whether it is possible to transform A into some A' to satisfy $\mathbb{C}_p^{A'}(f)$ and conclude the derivation in $\vdash_{A'}$. In particular, following [Filé et al. 1996; Giacobazzi and Ranzato 1997; Giacobazzi et al. 2000], we are interested in the problem of *minimally transforming* the abstract domain A to A' through refinements (i.e., by adding concrete values) and simplifications (i.e., by removing abstract values) to make A' locally complete for a set of basic transfer functions. This problem has been studied recently in [Bruni et al. 2022] where we proved that in general there is no unique minimal solution to the problem of abstract domain refinement for local completeness and a forward/backward strategy for optimally refining the abstract domain to achieve local completeness has been introduced. As sketched in Example 5.9, in program verification the strategy would be to iteratively transform the original abstract domain A_0 stepping through a sequences of abstract domains $A_1, \dots, A_n$, until a derivation $\vdash_{A_n} [p] \ r \ [q]$ can be completed in A_n . Each domain A_{i+1} can be designed by looking at the proof obligations of local completeness that fail in the attempt to prove $\vdash_{A_i} [p] \ r \ [q]$ using the current abstract domain A_i . Notably, different abstraction refinements could be used by applying the rule (refine) of the extended proof system $LCL_A^\leq$, introduced in Section 8, in different subtrees of the same derivation.

We also aim at investigating extensions of our proof system with *non-compositional rules* for programs. Here, the incompleteness for a statement r may turn to completeness for an extensionally equivalent statement r' such that $\llbracket r \rrbracket = \llbracket r' \rrbracket$. For example, a command such as $x := xy + 1; x := x - 1$ is incomplete for a simple sign analysis in $\text{Sign}^\pm \triangleq \{\emptyset, \mathbb{Z}_{<0}, \mathbb{Z}_{=0}, \mathbb{Z}_{>0}, \mathbb{Z}\}$ because

$$\llbracket x := x - 1 \rrbracket_{\text{Sign}^\pm}^\# (\llbracket x := xy + 1 \rrbracket_{\text{Sign}^\pm}^\#) \langle x/\mathbb{Z}_{>0}, y/\mathbb{Z}_{>0} \rangle = \llbracket x := x - 1 \rrbracket_{\text{Sign}^\pm}^\# \langle x/\mathbb{Z}_{>0}, y/\mathbb{Z}_{>0} \rangle = \langle x/\mathbb{Z}, y/\mathbb{Z}_{>0} \rangle.$$

Nevertheless, when considering an equivalent statement such as $x := xy$, we achieve a complete sign analysis because $\llbracket x := xy \rrbracket_{\text{Sign}^\pm}^\# \langle x/\mathbb{Z}_{>0}, y/\mathbb{Z}_{>0} \rangle = \langle x/\mathbb{Z}_{>0}, y/\mathbb{Z}_{>0} \rangle$. Patterns of this kind could be handled by a noncompositional rule for manipulating assignments such as

$$\frac{\vdash_A [p] \ x := (a_2[a_1/x]) \ [q]}{\vdash_A [p] \ x := a_1; x := a_2 \ [q]}.$$

Because $x := (x - 1)[x^{y+1}/x] \equiv x := xy + 1 - 1 \equiv x := xy$, from $\vdash_{\text{Sign}^+} [\langle x/\mathbb{Z}_{>0}, y/\mathbb{Z}_{>0} \rangle] x := xy [\langle x/\mathbb{Z}_{>0}, y/\mathbb{Z}_{>0} \rangle]$, we would be able to derive $\vdash_{\text{Sign}^+} [\langle x/\mathbb{Z}_{>0}, y/\mathbb{Z}_{>0} \rangle] x := xy + 1; x := x - 1 [\langle x/\mathbb{Z}_{>0}, y/\mathbb{Z}_{>0} \rangle]$.

While completeness of abstract transfer functions is preserved by function composition, as encoded by the rule (seq) of LCL_A , one major issue of abstract interpretation is that best correct approximations are not compositional, i.e., the composition of abstract predicate transformers may not be as precise as the abstraction of their concrete composition [Reps et al. 2004; Yorsh et al. 2004]. The lack of composition for bcas has practical consequences, because the precision of a program analysis strictly depends on the granularity of program decomposition into atomic operations. Finer decompositions commonly induce more imprecise analyses, while coarser decompositions may enhance the chance of designing bcas for larger code blocks. We plan to investigate a proof system for the *property of being a bca*, notably for proving when the composition of bcas is a bca.

ACKNOWLEDGMENTS

We are grateful to the anonymous reviewers for their helpful comments.

REFERENCES

- Flavio Ascari, Roberto Bruni, and Roberta Gori. 2022. Limits and difficulties in the design of under-approximation abstract domains. In *Proceedings of 25th International Conference on Foundations of Software Science and Computation Structures, (FOSSACS'22), Lecture Notes in Computer Science*, Patricia Bouyer and Lutz Schröder (Eds.), Vol. 13242. Springer, 21–39. https://doi.org/10.1007/978-3-030-99253-8_2
- Thomas Ball, Todd D. Millstein, and Sriram K. Rajamani. 2005. Polymorphic predicate abstraction. *ACM Trans. Program. Lang. Syst.* 27, 2 (2005), 314–343. <https://doi.org/10.1145/1057387.1057391>
- François Bourdoncle. 1993. Abstract debugging of higher-order imperative languages. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI'93)*. ACM, 46–55. <https://doi.org/10.1145/155090.155095>
- Roberto Bruni, Roberto Giacobazzi, Roberta Gori, Isabel Garcia-Contreras, and Dusko Pavlovic. 2020. Abstract extensionality: On the properties of incomplete abstract interpretations. In *Proceedings of the ACM Symposium on Principles of Programming Languages*. 28:1–28:28. <https://doi.org/10.1145/3371096>
- Roberto Bruni, Roberto Giacobazzi, Roberta Gori, and Francesco Ranzato. 2021. A logic for locally complete abstract interpretations. In *Proceedings of the 36th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS'21)*, Distinguished Paper. IEEE, 1–13. <https://doi.org/10.1109/LICS52264.2021.9470608>
- Roberto Bruni, Roberto Giacobazzi, Roberta Gori, and Francesco Ranzato. 2022. Abstract interpretation repair. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI'22)*, Ranjit Jhala and Isil Dillig (Eds.). ACM, 426–441. <https://doi.org/10.1145/3519939.3523453>
- Cristiano Calcagno, Dino Distefano, Jérémy Dubreil, Dominik Gabi, Pieter Hooimeijer, Martino Luca, Peter W. O'Hearn, Irene Papakonstantinou, Jim Purbrick, and Dulma Rodriguez. 2015. Moving fast with software verification. In *Proceedings of the NASA Formal Methods Symposium (NFM'15)*, LNCS, Vol. 9058. Springer, 3–11. https://doi.org/10.1007/978-3-319-17524-9_1
- Patrick Cousot. 2021. *Principles of Abstract Interpretation*. MIT Press.
- Patrick Cousot and Radhia Cousot. 1977. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the ACM Symposium on Principles of Programming Languages (POPL'77)*. ACM, 238–252. <https://doi.org/10.1145/512950.512973>
- Patrick Cousot and Radhia Cousot. 1979. Systematic design of program analysis frameworks. In *Proceedings of the ACM Symposium on Principles of Programming Languages (POPL'79)*. ACM, 269–282. <https://doi.org/10.1145/567752.567778>
- Patrick Cousot and Radhia Cousot. 1992. Abstract interpretation frameworks. *J. Logic Comput.* 2, 4 (1992), 511–547. <https://doi.org/10.1093/logcom/2.4.511>
- Patrick Cousot, Roberto Giacobazzi, and Francesco Ranzato. 2018. Program analysis is harder than verification: A computability perspective. In *Proceedings of the 30th International Conference on Computer Aided Verification (CAV'18) Lecture Notes in Computer Science*, Vol. 10982. Springer, 75–95. https://doi.org/10.1007/978-3-319-96142-2_8
- Edsko de Vries and Vasileios Koutavas. 2011. Reverse Hoare logic. In *Proceedings of the International Conference on Software Engineering and Formal Methods (SEFM'11)*. Springer, 155–171. https://doi.org/10.1007/978-3-642-24690-6_12
- Edsger W. Dijkstra. 1972a. *Chapter I: Notes on Structured Programming*. Academic Press Ltd., GBR, 1–82.

- Edsger W. Dijkstra. 1972b. The humble programmer. *Commun. ACM* 15, 10 (1972), 859–866. <https://doi.org/10.1145/355604.361591>
- Edsger W. Dijkstra. 1972c. Turing Award Lecture. Retrieved from <https://www.youtube.com/watch?v=6sIlKP2LzbA>.
- Dino Distefano, Manuel Fähndrich, Francesco Logozzo, and Peter W. O’Hearn. 2019. Scaling static analyses at Facebook. *Commun. ACM* 62, 8 (2019), 62–70. <https://doi.org/10.1145/3338112>
- G. Filé, R. Giacobazzi, and F. Ranzato. 1996. A unifying view of abstract domain design. *ACM Comput. Surv.* 28, 2 (1996), 333–336. <https://doi.org/10.1145/234528.234742>
- Robert W. Floyd. 1967. Assigning meanings to programs. In *Proceedings of the Symposium on Applied Mathematics*, Vol. 19, 19–32.
- Roberto Giacobazzi, Francesco Logozzo, and Francesco Ranzato. 2015. Analyzing program analyses. In *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL’15)*. 261–273. <https://doi.org/10.1145/2676726.2676987>
- Roberto Giacobazzi and Francesco Ranzato. 1996. Compositional optimization of disjunctive abstract interpretations. In *Proceedings of 6th European Symposium on Programming (ESOP’96), Lecture Notes in Computer Science*, Vol. 1058. Springer, 141–155. https://doi.org/10.1007/3-540-61055-3_34
- Roberto Giacobazzi and Francesco Ranzato. 1997. Completeness in abstract interpretation: A domain perspective. In *Proceedings of the 6th International Conference on Algebraic Methodology and Software Technology (AMAST’97), Lecture Notes in Computer Science*, Vol. 1349. Springer, 231–245. <https://doi.org/10.1007/BFb0000474>
- Roberto Giacobazzi and Francesco Ranzato. 1998. Optimal domains for disjunctive abstract interpretation. *Sci. Comput. Program.* 32, 1-3 (1998), 177–210. [https://doi.org/10.1016/S0167-6423\(97\)00034-8](https://doi.org/10.1016/S0167-6423(97)00034-8)
- Roberto Giacobazzi and Francesco Ranzato. 2022. History of abstract interpretation. *IEEE Ann. Hist. Comput.* 44, 2 (2022), 33–43. <https://doi.org/10.1109/MAHC.2021.3133136>
- Roberto Giacobazzi, Francesco Ranzato, and Francesca Scozzari. 1998. Complete abstract interpretations made constructive. In *Proceedings of the 23rd International Symposium on Mathematical Foundations of Computer Science (MFCS’98), Lecture Notes in Computer Science*, Vol. 1450. Springer, 366–377. <https://doi.org/10.1007/BFb0055786>
- Roberto Giacobazzi, Francesco Ranzato, and Francesca Scozzari. 2000. Making abstract interpretation complete. *J. ACM* 47, 2 (March 2000), 361–416. <https://doi.org/10.1145/333979.333989>
- Charles A. R. Hoare. 1969. An axiomatic basis for computer programming. *Commun. ACM* 12, 10 (1969), 576–580. <https://doi.org/10.1145/363235.363259>
- Charles A. R. Hoare. 2003. The verifying compiler: A grand challenge for computing research. *J. ACM* 50, 1 (2003), 63–69. <https://doi.org/10.1145/602382.602403>
- Cliff Jones, Peter O’Hearn, and Jim Woodcock. 2006. Verified software: A grand challenge. *IEEE Comput.* 39, 04 (2006), 93–95. <https://doi.org/10.1109/MC.2006.145>
- Jacques-Henri Jourdan, Vincent Laporte, Sandrine Blazy, Xavier Leroy, and David Pichardie. 2015. A formally-verified C static analyzer. In *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL’15)*. 247–259. <https://doi.org/10.1145/2676726.2676966>
- Dexter Kozen. 1997. Kleene algebra with tests. *ACM Trans. Program. Lang. Syst.* 19, 3 (May 1997), 427–443. <https://doi.org/10.1145/256167.256195>
- Quang Loc Le, Azalea Raad, Jules Villard, Josh Berdine, Derek Dreyer, and Peter W. O’Hearn. 2022. Finding real bugs in big programs with incorrectness logic. In *Proceedings of the ACM SIGPLAN International Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA’22)*, Vol. 6, 1–27. <https://doi.org/10.1145/3527325>
- Xavier Leroy. 2006. Formal certification of a compiler back-end or: Programming a compiler with a proof assistant. In *Proceedings of the 33rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL’06)*. 42–54. <https://doi.org/10.1145/1111037.1111042>
- Petar Maksimovic, Caroline Cronjäger, Julian Sutherland, Andreas Löw, Sacha-Élie Ayoun, and Philippa Gardner. 2022. Exact separation logic. [arXiv:2208.07200](https://arxiv.org/abs/2208.07200). Retrieved from <https://arxiv.org/abs/2208.07200>.
- John McCarthy. 1962. Towards a mathematical science of computation. In *IFIP Congress*. 21–28.
- Marco Milanese and Francesco Ranzato. 2022. Local completeness logic on Kleene algebra with tests. In *Proceedings of the 29th International Static Analysis Symposium (SAS’22), LNCS*, Vol. 13790. 350–371. https://doi.org/10.1007/978-3-031-22308-2_16
- Antoine Miné. 2006. The octagon abstract domain. *High. Order Symb. Comput.* 19, 1 (2006), 31–100. <https://doi.org/10.1007/s10990-006-8609-1>
- Antoine Miné. 2017. Tutorial on static inference of numeric invariants by abstract interpretation. *Found. Trends Program. Lang.* 4, 3-4 (2017), 120–372. <https://doi.org/10.1561/25000000034>
- Antoine Miné. 2014. Backward under-approximations in numeric abstract domains to automatically infer sufficient program conditions. *Sci. Comput. Program.* 93 (2014), 154–182. <https://doi.org/10.1016/j.scico.2013.09.014>

- Bernhard Möller, Peter W. O'Hearn, and Charles A. R. Hoare. 2021. On algebra of program correctness and incorrectness. In *Proceedings of the 19th International Conference on Relational and Algebraic Methods in Computer Science (RAMICS'21), Lecture Notes in Computer Science*, Uli Fahrenberg, Mai Gehrke, Luigi Santocanale, and Michael Winter (Eds.), Vol. 13027. Springer, 325–343. https://doi.org/10.1007/978-3-030-88701-8_20
- Bernhard Möller and Georg Struth. 2006. Algebras of modal operators and partial correctness. *Theor. Comput. Sci.* 351, 2 (2006), 221–239. <https://doi.org/10.1016/j.tcs.2005.09.069>
- Peter W. O'Hearn. 2018. Continuous reasoning: Scaling the impact of formal methods. In *Proceedings of the ACM/IEEE Symposium on Logic in Computer Science (LICS'18)*. ACM, 13–25. <https://doi.org/10.1145/3209108.3209109>
- Peter W. O'Hearn. 2020. Incorrectness logic. In *Proceedings of the ACM Annual Symposium on Principles of Programming Languages*, Vol. 4, 10:1–10:32. <https://doi.org/10.1145/3371078>
- Benjamin Pierce. 2002. *Types and Programming Languages*. MIT Press.
- Christopher M. Poskitt. 2021. Incorrectness logic for graph programs. In *Proceedings of the 14th International Conference on Graph Transformation (ICGT'21), Lecture Notes in Computer Science*, Vol. 12741. Springer, 81–101. https://doi.org/10.1007/978-3-030-78946-6_5
- Christopher M. Poskitt and Detlef Plump. 2023. Monadic second-order incorrectness logic for GP 2. *J. Log. Algebr. Methods Program.* 130 (2023), 100825. <https://doi.org/10.1016/j.jlamp.2022.100825>
- Azalea Raad, Josh Berdine, Hoang-Hai Dang, Derek Dreyer, Peter W. O'Hearn, and Jules Villard. 2020. Local reasoning about the presence of bugs: Incorrectness separation logic. In *Proceedings of the International Conference on Computer-Aided Verification (CAV'20), Part II, LNCS*, Vol. 12225. Springer, 225–252. https://doi.org/10.1007/978-3-030-53291-8_14
- Azalea Raad, Josh Berdine, Derek Dreyer, and Peter W. O'Hearn. 2022. Concurrent incorrectness separation logic. In *Proceedings of the ACM Annual Symposium on Principles of Programming Languages*, Vol. 6 (2022), 1–29. <https://doi.org/10.1145/3498695>
- Francesco Ranzato. 2020. Decidability and synthesis of abstract inductive invariants. In *Proceedings of the 31st International Conference on Concurrency Theory (CONCUR'20), LIPIcs*, Vol. 171. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 30:1–30:21. <https://doi.org/10.4230/LIPIcs.CONCUR.2020.30>
- Thomas W. Reps, Shmuel Sagiv, and Greta Yorsh. 2004. Symbolic implementation of the best transformer. In *Proceedings of the 24th International Conference on Verification, Model Checking, and Abstract Interpretation (VMCAI'04), LNCS*, Vol. 2937. Springer, 252–266. https://doi.org/10.1007/978-3-540-24622-0_21
- Henry G. Rice. 1953. Classes of recursively enumerable sets and their decision problems. *Trans. Am. Math. Soc.* 74 (1953), 358–366.
- Xavier Rival and Kwang Yi. 2020. *Introduction to Static Analysis—An Abstract Interpretation Perspective*. MIT Press.
- Caitlin Sadowski, Edward Aftandilian, Alex Eagle, Liam Miller-Cushon, and Ciera Jaspán. 2018. Lessons from building static analysis tools at Google. *Commun. ACM* 61, 4 (March 2018), 58–66. <https://doi.org/10.1145/3188720>
- Alan M. Turing. 1989. Checking a large routine. In *The Early British Computer Conferences*, Martin Campbell-Kelly (Ed.). MIT Press, Cambridge, MA, 70–72.
- Glynn Winskel. 1993. *The Formal Semantics of Programming Languages: An Introduction*. MIT Press.
- Peng Yan, Hanru Jiang, and Nengkun Yu. 2022. On incorrectness logic for Quantum programs. In *Proceedings of the ACM SIGPLAN International Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA'22)*, 1–28. <https://doi.org/10.1145/3527316>
- Greta Yorsh, Thomas W. Reps, and Shmuel Sagiv. 2004. Symbolically computing most-precise abstract operations for shape analysis. In *Proceedings of the International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS'04), LNCS*, Vol. 2988. Springer, 530–545. https://doi.org/10.1007/978-3-540-24730-2_39
- Cheng Zhang, Arthur Azevedo de Amorim, and Marco Gaboardi. 2022. On incorrectness logic and Kleene algebra with top and tests. In *Proceedings of the ACM Annual Symposium on Principles of Programming Languages (POPL'22)*. <https://doi.org/10.1145/3498690>
- Noam Zilberstein, Derek Dreyer, and Alexandra Silva. 2023. Outcome logic: A unifying foundation for correctness and incorrectness reasoning. *Proceedings of the ACM SIGPLAN International Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA'23)*, To appear.

Received 26 January 2022; revised 26 October 2022; accepted 4 January 2023